

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống quản lý ngân hàng câu hỏi và
chấm thi trắc nghiệm tự động sử dụng công nghệ
OMR**

ĐINH MẠNH DŨNG

mahndugn@gmail.com

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: TS. Trần Vĩnh Đức

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống quản lý ngân hàng câu hỏi và
chấm thi trắc nghiệm tự động sử dụng công nghệ
OMR**

ĐINH MẠNH DŨNG

mahndugn@gmail.com

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: TS. Trần Vĩnh Đức _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành tới TS. Trần Vĩnh Đức, giảng viên hướng dẫn của đề án, người đã tận tình chỉ bảo, định hướng nghiên cứu và đóng góp những nhận xét quý báu giúp em hoàn thiện đề án này.

Em cũng xin trân trọng cảm ơn các thầy cô Khoa Khoa học máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã trang bị kiến thức chuyên môn và tạo điều kiện để em thực hiện đề án. Em cảm ơn các bạn cùng khóa đã thường xuyên thảo luận, chia sẻ kinh nghiệm và hỗ trợ em trong quá trình xây dựng và kiểm thử hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc tới gia đình và bạn bè đã luôn động viên, khuyến khích em vượt qua khó khăn trong quá trình học tập và thực hiện đề án. Em cũng tự cảm ơn chính mình đã kiên trì để hoàn thành đề án này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh các kỳ thi trắc nghiệm quy mô lớn ngày càng phổ biến tại các trường phổ thông và đại học, việc chấm thi thủ công trên phiếu điền tay vẫn tồn tại nhiều hạn chế như tốn thời gian, dễ sai sót, thiếu tính minh bạch và khó khăn trong việc thống kê, phân tích kết quả. Một số giải pháp chấm thi tự động hiện có thường phụ thuộc vào phần cứng máy quét chuyên dụng đắt tiền, hoặc chỉ giải quyết được một phần quy trình như tạo đề hay nhận dạng phiếu trả lời mà chưa cung cấp một hệ thống tích hợp đầy đủ từ quản lý ngân hàng câu hỏi, trộn đề, chấm thi bằng nhận dạng OMR trên thiết bị di động, đến xử lý phúc tra và báo cáo thống kê.

Đồ án lựa chọn hướng tiếp cận xây dựng một hệ thống chấm thi trắc nghiệm thông minh đa nền tảng, tích hợp nhận dạng OMR trên thiết bị di động, tự động tạo nhiều phiên bản đề thi từ ngân hàng câu hỏi, hỗ trợ quy trình phúc tra minh bạch và cung cấp các báo cáo thống kê chi tiết cho giáo viên và quản trị viên. Hệ thống sử dụng kiến trúc ba tầng gồm backend Node.js/Express với cơ sở dữ liệu MongoDB, ứng dụng di động Flutter cho thầy cô sử dụng máy ảnh thiết bị để quét và chấm bài, giao diện web React cho quản lý và thống kê, kết hợp gói LaTeX AMC (Auto-Multiple-Choice) để tự động sinh đề thi và phiếu trả lời. Đồng thời, hệ thống tích hợp các mô hình trí tuệ nhân tạo sinh câu hỏi từ Gemini, OpenAI và Claude nhằm hỗ trợ giáo viên trong khâu xây dựng ngân hàng câu hỏi.

Kết quả chính của đồ án là hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR đã được xây dựng hoàn chỉnh với các chức năng quản lý trường học, lớp học, giáo viên và học sinh; quản lý ngân hàng câu hỏi và đề thi có hỗ trợ tạo câu hỏi bằng trí tuệ nhân tạo; tự động trộn đề thi thành nhiều phiên bản khác nhau thông qua AMC kèm tọa độ nhận dạng chính xác; chấm thi tự động bằng nhận dạng OMR trên thiết bị di động với độ chính xác cao, có khả năng phát hiện lỗi tô nhầm, tô nhẹ và hỗ trợ chỉnh sửa thủ công; quy trình tạo đơn phúc tra và phản hồi phúc tra minh bạch với thông báo thời gian thực; cùng với hệ thống báo cáo và thống kê chi tiết cho từng bài kiểm tra. Hệ thống đã được triển khai thử nghiệm tại hai lớp học với khoảng 60 học sinh trong bốn tuần, xử lý thành công 5 bài kiểm tra và khoảng 300 bài làm, đạt độ chính xác OMR trung bình 96,2% và thời gian phản hồi API trung bình 180ms.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)
ĐINH MẠNH DŨNG

MỤC LỤC

MỤC LỤC	i
DANH MỤC HÌNH VẼ	vii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	3
2.1 Khảo sát hiện trạng	3
2.2 Tổng quan chức năng.....	4
2.2.1 Biểu đồ use case phân rã Đăng ký/Đăng nhập.....	5
2.2.2 Biểu đồ use case phân rã Quản lý trường học.....	6
2.2.3 Biểu đồ use case phân rã Quản lý giáo viên	7
2.2.4 Biểu đồ use case phân rã Quản lý học sinh	8
2.2.5 Biểu đồ use case phân rã Quản lý câu hỏi	9
2.2.6 Biểu đồ use case phân rã Quản lý bài kiểm tra	10
2.2.7 Biểu đồ use case phân rã Quản lý các bài thi của học sinh.....	11
2.2.8 Biểu đồ use case phân rã Chấm thi bằng ứng dụng	12
2.2.9 Biểu đồ use case phân rã Phúc khảo bài thi.....	13
2.2.10 Biểu đồ use case phân rã Xem bài thi và điểm số	14
2.2.11 Quy trình nghiệp vụ.....	14
2.3 Đặc tả chức năng	15
2.3.1 Đặc tả use case Quản lý giáo viên.....	15
2.3.2 Đặc tả use case Quản lý câu hỏi.....	17

2.3.3 Đặc tả use case Quản lý bài kiểm tra.....	18
2.3.4 Đặc tả use case Chấm thi bằng ứng dụng.....	20
2.3.5 Đặc tả use case Phúc khảo bài thi	22
2.4 Yêu cầu phi chức năng.....	24
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG..	26
3.1 Kỹ thuật nhận dạng OMR và xử lý ảnh số.....	26
3.2 Kỹ thuật tạo đề nhiều phiên bản với AMC	26
3.3 Công nghệ phát triển backend.....	27
3.4 Công nghệ phát triển giao diện web.....	27
3.5 Công nghệ phát triển ứng dụng di động	27
3.6 Công nghệ trí tuệ nhân tạo tích hợp	27
3.7 Các công nghệ hỗ trợ khác	28
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG.....	29
4.1 Thiết kế kiến trúc.....	29
4.1.1 Lựa chọn kiến trúc phần mềm	29
4.1.2 Thiết kế tổng quan	29
4.1.3 Thiết kế chi tiết các gói.....	31
4.2 Thiết kế chi tiết	38
4.2.1 Thiết kế giao diện	38
4.2.2 Thiết kế chi tiết lớp	42
4.2.3 Thiết kế cơ sở dữ liệu	49
4.3 Xây dựng ứng dụng	57
4.3.1 Thư viện và công cụ sử dụng.....	57
4.3.2 Kết quả đạt được.....	57
4.3.3 Minh họa các chức năng chính	58

4.4 Kiểm thử	59
4.5 Sản phẩm và hướng dẫn triển khai.....	59
4.5.1 Tổng quan sản phẩm	60
4.5.2 Hướng dẫn triển khai.....	60
4.5.3 Kiểm thử trong môi trường cục bộ.....	61
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	62
5.1 Giải pháp tự động sinh đề thi nhiều phiên bản và phiếu OMR kèm tọa độ nhận dạng.....	62
5.1.1 Dẫn dắt bài toán.....	62
5.1.2 Giải pháp chi tiết.....	62
5.1.3 Kết quả đạt được.....	67
5.2 Giải pháp nhận dạng OMR trên thiết bị di động với khả năng phát hiện lỗi	67
5.2.1 Dẫn dắt bài toán.....	67
5.2.2 Giải pháp chi tiết.....	67
5.2.3 Kết quả đạt được.....	74
5.3 Giải pháp quản lý phiên làm việc và phân quyền theo vai trò trên đa nền tảng	75
5.3.1 Dẫn dắt bài toán.....	75
5.3.2 Giải pháp chi tiết.....	75
5.3.3 Kết quả đạt được.....	75
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	76
6.1 Kết luận.....	76
6.2 Hướng phát triển	76
6.2.1 Hoàn thiện các chức năng hiện có.....	76
6.2.2 Mở rộng tính năng mới.....	77
6.2.3 Nâng cấp công nghệ	77
TÀI LIỆU THAM KHẢO	79

PHỤ LỤC	81
A. ĐẶC TẢ USE CASE	81
A.0.1 Đặc tả use case Quản lý trường học	81
A.0.2 Đặc tả use case Quản lý học sinh	82
A.0.3 Đặc tả use case Quản lý các bài thi của học sinh	83
A.0.4 Đặc tả use case Xem bài thi và điểm số	84
A.0.5 Đặc tả use case Đăng ký/Đăng nhập.....	85
B. THIẾT KẾ CHI TIẾT CÁC LỚP.....	87
B.1 Lớp AppealService (Backend – Node.js).....	87
B.2 Lớp NotificationService (Backend – Node.js).....	88
B.3 Lớp AuthService (Backend – Node.js).....	89
B.4 Lớp ClassService (Backend – Node.js)	90
B.5 Lớp AuthStore (Frontend – React/Zustand)	90
B.6 Lớp SubmissionStore (Frontend – React/Zustand)	91
B.7 Lớp ImageProcessor (Mobile – Flutter).....	92
B.8 Lớp BubbleDetector (Mobile – Flutter).....	93
B.9 Lớp AnswerReader (Mobile – Flutter)	94
B.10 Tổng hợp các lớp trong phụ lục	95
C. SINH CÂU HỎI BẰNG AI.....	97
C.1 Tổng quan chức năng sinh câu hỏi bằng AI	97
C.2 Sinh câu hỏi mới.....	97
C.2.1 Mục đích	97
C.2.2 Điều kiện trước	97
C.2.3 Luồng thực thi chính.....	97
C.2.4 Luồng thực thi mở rộng	98
C.2.5 Điều kiện sau	98

C.2.6 Cấu trúc prompt gửi đến AI.....	98
C.2.7 Cấu trúc câu hỏi sinh ra.....	98
C.3 Sinh câu hỏi tương tự	98
C.3.1 Mục đích	98
C.3.2 Điều kiện trước	99
C.3.3 Luồng thực thi chính.....	99
C.3.4 Luồng thực thi mở rộng	99
C.3.5 Điều kiện sau	99
C.3.6 Các kiểu thay đổi khi sinh tương tự.....	100
C.3.7 Cấu trúc prompt cho chức năng sinh tương tự	100
C.4 Xử lý lỗi và giới hạn.....	100
C.4.1 Các lỗi thường gặp	100
C.4.2 Giới hạn của hệ thống	100
C.5 Bảo mật và đạo đức.....	101
C.5.1 Các câu hỏi do AI sinh ra	101
C.5.2 Xác thực nội dung	101

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	5
Hình 2.2	Biểu đồ use case phân rã Đăng ký/Đăng nhập	6
Hình 2.3	Biểu đồ use case phân rã Quản lý trường học	7
Hình 2.4	Biểu đồ use case phân rã Quản lý giáo viên	8
Hình 2.5	Biểu đồ use case phân rã Quản lý học sinh	9
Hình 2.6	Biểu đồ use case phân rã Quản lý câu hỏi	10
Hình 2.7	Biểu đồ use case phân rã Quản lý bài kiểm tra	11
Hình 2.8	Biểu đồ use case phân rã Quản lý các bài thi của học sinh . .	12
Hình 2.9	Biểu đồ use case phân rã Chấm thi bằng ứng dụng	13
Hình 2.10	Biểu đồ use case phân rã Phúc khảo bài thi	13
Hình 2.11	Biểu đồ use case phân rã Xem bài thi và điểm số	14
Hình 4.1	Biểu đồ gói tổng quan của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	30
Hình 4.2	Biểu đồ gói Routers của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	31
Hình 4.3	Biểu đồ gói Controllers của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	32
Hình 4.4	Biểu đồ gói Services của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	32
Hình 4.5	Biểu đồ gói Models của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	33
Hình 4.6	Biểu đồ gói Pages của giao diện web React	34
Hình 4.7	Biểu đồ gói Components của giao diện web React	34
Hình 4.8	Biểu đồ gói Stores và Hooks của giao diện web React	35
Hình 4.9	Biểu đồ gói ApiService của giao diện web React	36
Hình 4.10	Biểu đồ gói Widgets của ứng dụng di động Flutter	37
Hình 4.11	Biểu đồ gói Screens của ứng dụng di động Flutter	37

Hình 4.12	Biểu đồ gói BLoCs của ứng dụng di động Flutter	38
Hình 4.13	Minh họa màn hình chi tiết lớp học trên web	39
Hình 4.14	Minh họa màn hình chi tiết bài kiểm tra trên web	40
Hình 4.15	Minh họa màn hình quét phiếu trên di động	41
Hình 4.16	Minh họa màn hình phúc khảo trên web	42
Hình 4.17	Biểu đồ trình tự use case Chấm thi bằng điện thoại	48
Hình 4.18	Biểu đồ trình tự use case Quản lý đề thi	49
Hình 4.19	Sơ đồ quan hệ thực thể (ER) của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	50

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh tính năng giữa các giải pháp chấm thi hiện có	4
Bảng 2.2	Thông tin chung use case Quản lý giáo viên	15
Bảng 2.3	Luồng thực thi chính của use case Quản lý giáo viên	16
Bảng 2.4	Luồng thực thi mở rộng của use case Quản lý giáo viên . . .	16
Bảng 2.5	Điều kiện kết thúc use case Quản lý giáo viên	16
Bảng 2.6	Thông tin chung use case Quản lý câu hỏi	17
Bảng 2.7	Luồng thực thi chính của use case Quản lý câu hỏi	17
Bảng 2.8	Luồng thực thi mở rộng của use case Quản lý câu hỏi	18
Bảng 2.9	Điều kiện kết thúc use case Quản lý câu hỏi	18
Bảng 2.10	Thông tin chung use case Quản lý bài kiểm tra	18
Bảng 2.11	Luồng thực thi chính của use case Quản lý bài kiểm tra . . .	19
Bảng 2.12	Luồng thực thi mở rộng của use case Quản lý bài kiểm tra .	20
Bảng 2.13	Điều kiện kết thúc use case Quản lý bài kiểm tra	20
Bảng 2.14	Thông tin chung use case Chấm thi bằng ứng dụng	20
Bảng 2.15	Luồng thực thi chính của use case Chấm thi bằng ứng dụng .	21
Bảng 2.16	Luồng thực thi mở rộng của use case Chấm thi bằng ứng dụng	21
Bảng 2.17	Điều kiện kết thúc use case Chấm thi bằng ứng dụng	22
Bảng 2.18	Thông tin chung use case Phúc khảo bài thi	22
Bảng 2.19	Luồng thực thi chính của use case Phúc khảo bài thi	23
Bảng 2.20	Luồng thực thi mở rộng của use case Phúc khảo bài thi . . .	24
Bảng 2.21	Điều kiện kết thúc use case Phúc khảo bài thi	24
Bảng 4.1	Các thuộc tính của lớp SubmissionService	43
Bảng 4.2	Các phương thức của lớp SubmissionService	43
Bảng 4.3	Các thuộc tính của lớp ExamService	44
Bảng 4.4	Các phương thức của lớp ExamService	44
Bảng 4.5	Các thuộc tính của lớp OMREngine	45
Bảng 4.6	Các phương thức của lớp OMREngine	45
Bảng 4.7	Tổng hợp các lớp chủ đạo của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	46
Bảng 4.8	Danh sách thư viện và công cụ sử dụng trong đồ án	57
Bảng 4.9	Thống kê mã nguồn của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR	58

Bảng 4.10	Tổng hợp kết quả kiểm thử các chức năng chính	59
Bảng A.1	Thông tin chung use case Quản lý trường học	81
Bảng A.2	Luồng thực thi chính của use case Quản lý trường học . . .	81
Bảng A.3	Luồng thực thi mở rộng của use case Quản lý trường học . .	82
Bảng A.4	Điều kiện kết thúc use case Quản lý trường học	82
Bảng A.5	Thông tin chung use case Quản lý học sinh	82
Bảng A.6	Luồng thực thi chính của use case Quản lý học sinh	82
Bảng A.7	Luồng thực thi mở rộng của use case Quản lý học sinh . . .	83
Bảng A.8	Điều kiện kết thúc use case Quản lý học sinh	83
Bảng A.9	Thông tin chung use case Quản lý các bài thi của học sinh .	83
Bảng A.10	Luồng thực thi chính của use case Quản lý các bài thi của học sinh	83
Bảng A.11	Luồng thực thi mở rộng của use case Quản lý các bài thi của học sinh	84
Bảng A.12	Điều kiện kết thúc use case Quản lý các bài thi của học sinh	84
Bảng A.13	Thông tin chung use case Xem bài thi và điểm số	84
Bảng A.14	Luồng thực thi chính của use case Xem bài thi và điểm số .	84
Bảng A.15	Luồng thực thi mở rộng của use case Xem bài thi và điểm số	85
Bảng A.16	Điều kiện kết thúc use case Xem bài thi và điểm số	85
Bảng A.17	Thông tin chung use case Đăng ký/Đăng nhập	85
Bảng A.18	Luồng thực thi chính của use case Đăng ký/Đăng nhập . . .	86
Bảng A.19	Luồng thực thi mở rộng của use case Đăng ký/Đăng nhập . .	86
Bảng A.20	Điều kiện kết thúc use case Đăng ký/Đăng nhập	86
Bảng B.1	Các thuộc tính của lớp AppealService	87
Bảng B.2	Các phương thức của lớp AppealService	87
Bảng B.3	Các thuộc tính của lớp NotificationService	88
Bảng B.4	Các phương thức của lớp NotificationService	88
Bảng B.5	Các thuộc tính của lớp AuthService	89
Bảng B.6	Các phương thức của lớp AuthService	89
Bảng B.7	Các thuộc tính của lớp ClassService	90
Bảng B.8	Các phương thức của lớp ClassService	90
Bảng B.9	Các thuộc tính của lớp AuthStore	91
Bảng B.10	Các phương thức của lớp AuthStore	91
Bảng B.11	Các thuộc tính của lớp SubmissionStore	92
Bảng B.12	Các phương thức của lớp SubmissionStore	92
Bảng B.13	Các thuộc tính của lớp ImageProcessor	93

Bảng B.14	Các phương thức của lớp ImageProcessor	93
Bảng B.15	Các thuộc tính của lớp BubbleDetector	94
Bảng B.16	Các phương thức của lớp BubbleDetector	94
Bảng B.17	Các thuộc tính của lớp AnswerReader	95
Bảng B.18	Các phương thức của lớp AnswerReader	95
Bảng B.19	Tổng hợp các lớp trong Phụ lục C	95
Bảng B.20	Phân bố các lớp theo nền tảng	96
Bảng C.1	Cấu trúc prompt cho chức năng sinh câu hỏi mới	98
Bảng C.2	Cấu trúc câu hỏi sinh ra bởi AI	98
Bảng C.3	Các kiểu thay đổi khi sinh câu hỏi tương tự	100
Bảng C.4	Cấu trúc prompt cho chức năng sinh câu hỏi tương tự	100
Bảng C.5	Các lỗi thường gặp và cách xử lý	100

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
AMC	Auto-Multiple-Choice - gói LaTeX hỗ trợ tạo đề và phiếu trả lời trắc nghiệm
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CDN	Mạng phân phối nội dung (Content Delivery Network)
CRUD	Create - Read - Update - Delete: bốn thao tác cơ bản trên dữ liệu
CSS	Cascading Style Sheets - ngôn ngữ định dạng giao diện web
CV	Thị giác máy tính (Computer Vision)
DOM	Document Object Model - mô hình đối tượng tài liệu trong trình duyệt web
DPI	Dots Per Inch - số điểm ảnh trên mỗi inch, đơn vị đo độ phân giải in ấn
JWT	JSON Web Token - chuẩn mã thông báo truy cập dạng JSON
LaTeX	Hệ thống sắp chữ khoa học dựa trên TeX, dùng để soạn thảo tài liệu kỹ thuật
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
MVC	Model-View-Controller - mô hình kiến trúc phần mềm phân tách dữ liệu, giao diện và điều khiển
ODM	Object Document Mapper - lớp ánh xạ giữa đối tượng và tài liệu trong CSDL
OMR	Nhận dạng dấu quang học (Optical Mark Recognition)
OpenCV	Thư viện mã nguồn mở dùng cho thị giác máy tính và xử lý ảnh (Open Source Computer Vision Library)

Thuật ngữ	Ý nghĩa
ORM	Object Relational Mapper - lớp ánh xạ giữa đối tượng và quan hệ trong CSDL
REST	Representational State Transfer - kiểu kiến trúc API phổ biến trên HTTP
SDK	Bộ công cụ phát triển phần mềm (Software Development Kit)
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)
WSL2	Windows Subsystem for Linux phiên bản 2 - môi trường Linux chạy trên Windows

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày bối cảnh thực tiễn của bài toán chấm thi trắc nghiệm, nêu bật các hạn chế của giải pháp hiện có, từ đó xác định mục tiêu, phạm vi và định hướng giải pháp Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR cho đề án.

1.1 Đặt vấn đề

Hình thức thi trắc nghiệm được sử dụng rộng rãi tại các kỳ thi tốt nghiệp, tuyển sinh và kiểm tra định kỳ ở Việt Nam. Tuy nhiên, quy trình chấm thi hiện nay tại nhiều trường còn tồn tại nhiều hạn chế: chấm thủ công tốn thời gian và dễ sai sót, máy quét OMR chuyên dụng chi phí cao và khó triển khai cho trường nhỏ, quy trình phúc khảo chưa số hóa và thiếu minh bạch, cùng với việc thiếu tích hợp giữa quản lý đào tạo và chấm thi.

Từ đó, đề án đặt mục tiêu xây dựng hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR – hệ thống chấm thi trắc nghiệm thông minh đa nền tảng, tích hợp đầy đủ từ tạo đề thi đến chấm bài, phúc khảo và báo cáo thống kê, sử dụng camera điện thoại thay cho máy quét chuyên dụng.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay đã có một số phần mềm thương mại (Remark Office OMR, Gradescope) và mã nguồn mở (OMRChecker) hỗ trợ chấm thi trắc nghiệm. Qua khảo sát, các giải pháp hiện có còn tồn tại các hạn chế chính: phụ thuộc phần cứng máy quét (chi phí cao); chưa hỗ trợ tốt tạo nhiều phiên bản đề thi; quy trình phúc khảo chưa số hóa; thiếu tích hợp giữa quản lý đào tạo với chấm thi; và hầu hết chỉ chạy trên web, chưa có ứng dụng di động cho phép giáo viên chấm bài ngay tại lớp.

Từ đó, mục tiêu chính của đề án là xây dựng hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR với các chức năng: (i) quản lý trường học, lớp học, giáo viên và học sinh; (ii) quản lý ngân hàng câu hỏi và đề thi có hỗ trợ sinh câu hỏi bằng AI; (iii) tự động tạo nhiều phiên bản đề thi; (iv) chấm thi tự động bằng nhận dạng OMR trên thiết bị di động; (v) quy trình phúc khảo minh bạch; (vi) báo cáo thống kê chi tiết; và (vii) cho phép học sinh xem bài thi và điểm số. Phạm vi tập trung vào môi trường thi trên giấy với phiếu OMR, tiếng Việt, phù hợp bậc phổ thông và đại học.

1.3 Định hướng giải pháp

Đề án lựa chọn hướng tiếp cận xây dựng hệ thống đa nền tảng theo kiến trúc ba tầng: giao diện gồm ứng dụng di động Flutter và web React, backend Node.js/Express kết hợp MongoDB, và tầng sinh đề thi bằng gói LaTeX AMC qua WSL2. Hướng tiếp cận này được chọn vì: camera smartphone thay thế máy quét (giảm chi phí); AMC hỗ trợ tạo nhiều phiên bản đề tự động (chống gian lận); Node.js/Express và MongoDB dễ mở rộng với hệ sinh thái thư viện phong phú.

Đóng góp chính của đề án gồm: pipeline tích hợp đầy đủ từ tạo đề, chấm thi, phúc khảo đến báo cáo; phương pháp sinh tự động phiếu OMR kèm tọa độ nhận dạng; thuật toán chấm OMR trên di động có khả năng phát hiện lỗi tô nhầm, tô nhẹ; và tích hợp trợ lý AI hỗ trợ sinh câu hỏi và phân tích bài làm.

1.4 Bố cục đề án

Phần còn lại của đề án gồm: Chương 2 trình bày khảo sát hiện trạng, biểu đồ use case và đặc tả chi tiết các use case chính cùng yêu cầu phi chức năng; Chương 3 giới thiệu các nền tảng lý thuyết và công nghệ sử dụng – OMR, AMC, Node.js/Express, React, Flutter, AI; Chương 4 trình bày chi tiết phân tích thiết kế kiến trúc, giao diện, lớp, cơ sở dữ liệu, kết quả xây dựng, kiểm thử và triển khai; Chương 5 phân tích các giải pháp kỹ thuật nổi bật; và Chương 6 tổng kết, đối chiếu với sản phẩm tương tự và đề xuất hướng phát triển.

Chương 1 đã trình bày bối cảnh thực tiễn, xác định mục tiêu xây dựng hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR và định hướng giải pháp đa nền tảng. Chương 2 sẽ trình bày chi tiết khảo sát hiện trạng và phân tích yêu cầu của hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày kết quả khảo sát hiện trạng từ ba nguồn: nhu cầu người dùng, hệ thống chấm thi hiện tại và các ứng dụng tương tự. Trên cơ sở đó, chương mô tả tổng quan chức năng hệ thống thông qua biểu đồ use case, đặc tả chi tiết các use case quan trọng và xác định các yêu cầu phi chức năng làm cơ sở cho thiết kế và triển khai ở các chương sau.

2.1 Khảo sát hiện trạng

Để xác định rõ yêu cầu cho hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, đề án tiến hành khảo sát hiện trạng từ ba nguồn thông tin chính, bao gồm: (i) khảo sát nhu cầu của giáo viên và học sinh tại một số trường phổ thông và đại học; (ii) khảo sát hệ thống chấm thi đang được sử dụng tại trường; và (iii) khảo sát các ứng dụng tương tự trên thị trường.

Khảo sát nhu cầu người dùng. Qua khảo sát trực tuyến và phỏng vấn sơ bộ khoảng 30 giáo viên và 80 học sinh, có thể nhận thấy các nhu cầu phổ biến như sau: giáo viên mong muốn có công cụ tạo đề thi nhanh từ ngân hàng câu hỏi có sẵn, hỗ trợ sinh nhiều mã đề để hạn chế gian lận, có khả năng chấm bài ngay sau khi kết thúc buổi thi để công bố kết quả sớm; học sinh mong muốn được xem điểm và bài làm của mình một cách minh bạch, có cơ chế gửi phúc khảo rõ ràng khi cho rằng điểm chưa chính xác. Ngoài ra, cả giáo viên và học sinh đều mong muốn hệ thống hỗ trợ thống kê kết quả theo lớp, theo môn, theo chủ đề để phục vụ cho việc đánh giá chất lượng học tập.

Khảo sát hệ thống hiện tại. Tại các trường được khảo sát, hệ thống chấm thi hiện đang sử dụng chủ yếu thuộc hai loại: (i) chấm thủ công bằng cách đối chiếu phiếu trả lời với đáp án, áp dụng phổ biến tại các trường có quy mô nhỏ; và (ii) chấm bằng máy quét OMR chuyên dụng kết hợp phần mềm đi kèm, áp dụng tại một số trường lớn và các hội đồng thi cấp tỉnh. Hệ thống loại thứ nhất có ưu điểm chi phí thấp nhưng tốn thời gian, dễ sai sót và khó thống kê. Hệ thống loại thứ hai cho tốc độ chấm nhanh và độ chính xác cao, nhưng chi phí đầu tư máy quét lớn, phần mềm thường đi kèm không hỗ trợ tốt các nghiệp vụ quản lý đào tạo và phúc khảo.

Khảo sát ứng dụng tương tự. Đề án khảo sát một số giải pháp chấm thi phổ biến hiện nay, gồm: phần mềm thương mại như Remark Office OMR và Gradescope; công cụ mã nguồn mở như OMRChecker; các hệ thống thi trực tuyến như Google Forms, Microsoft Forms và Quizizz. Đặc điểm so sánh giữa các giải pháp được tổng hợp trong Bảng 2.1.

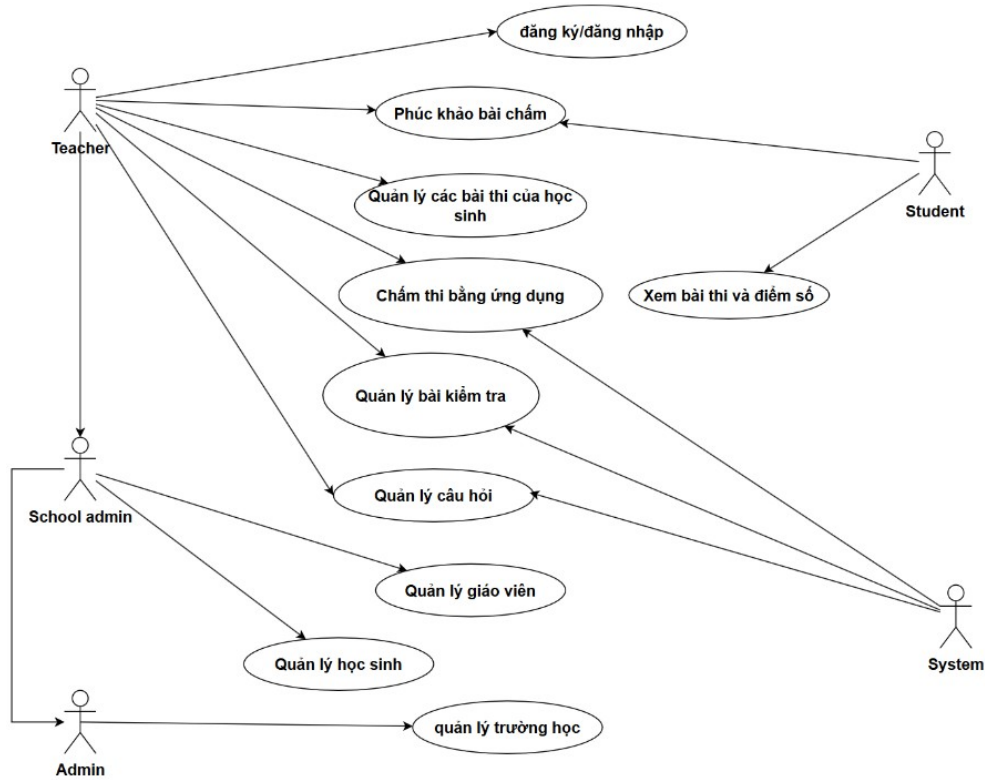
Tính năng	Remark OMR	Gradescope	OMRChecker	Hệ thống
Chấm OMR tự động	Có	Có	Có	Có
Tạo nhiều mã đề	Không	Không	Không	Có
Quản lý lớp, học sinh	Không	Có	Không	Có
Phúc khảo trực tuyến	Không	Có	Không	Có
Chấm trên di động	Không	Có	Có	Có
Hỗ trợ tiếng Việt	Không	Không	Một phần	Có
Tích hợp AI	Không	Có	Không	Có
Chi phí đầu tư	Cao	Cao	Miễn phí	Miễn phí

Bảng 2.1: So sánh tính năng giữa các giải pháp chấm thi hiện có

Qua bảng so sánh có thể nhận thấy hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR được đề xuất trong đồ án có sự kết hợp đầy đủ các tính năng mà các giải pháp hiện có chưa đáp ứng đồng thời, đặc biệt là khả năng tạo nhiều mã đề, quản lý đào tạo tích hợp, hỗ trợ phúc khảo minh bạch, chấm trên di động và hỗ trợ tiếng Việt đầy đủ.

2.2 Tổng quan chức năng

Trên cơ sở kết quả khảo sát, hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR được đề xuất với các nhóm chức năng chính gồm: quản lý người dùng và đào tạo (trường, lớp, giáo viên, học sinh), quản lý ngân hàng câu hỏi và đề thi, chấm thi bằng nhận dạng OMR, quản lý bài thi và phúc khảo, và báo cáo thống kê. Biểu đồ use case tổng quát mô tả các tác nhân và use case chính của hệ thống như trong Hình 2.1.



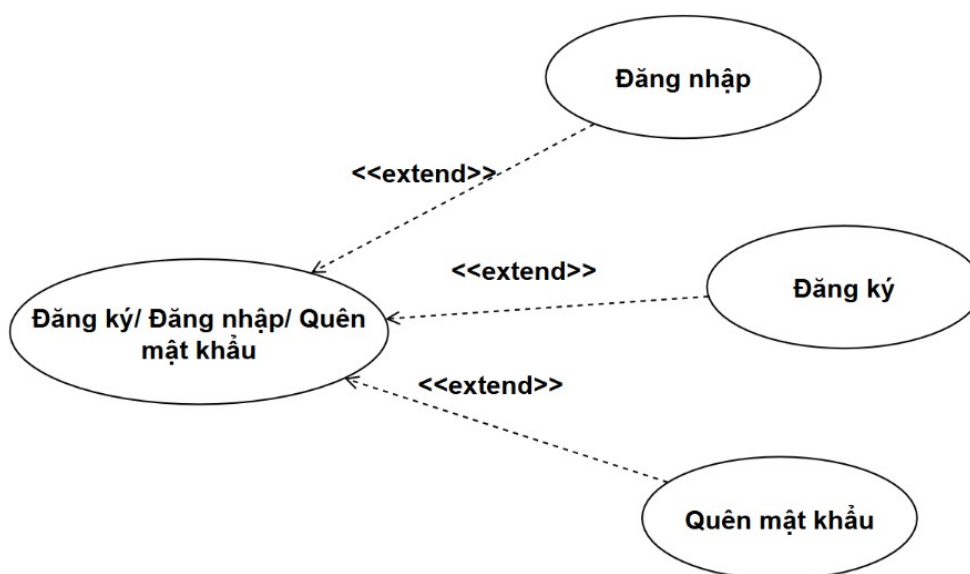
Hình 2.1: Biểu đồ use case tổng quát của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Tác nhân tham gia hệ thống. Hệ thống có bốn tác nhân chính gồm: (i) *Quản trị viên hệ thống* (admin) chịu trách nhiệm quản lý toàn bộ trường, tài khoản và cấu hình hệ thống; (ii) *Quản trị viên trường* (school-admin) quản lý các lớp, giáo viên, học sinh thuộc trường mình phụ trách; (iii) *Giáo viên* (teacher) thực hiện tạo đề thi, quản lý ngân hàng câu hỏi, chấm bài và xử lý phúc khảo; và (iv) *Học sinh* (student) xem đề thi, xem điểm, gửi yêu cầu phúc khảo và sử dụng trợ lý AI.

Các use case chính. Hệ thống có 10 use case chính, được phân thành 3 nhóm chức năng lớn. *Nhóm quản lý đào tạo* gồm Quản lý trường học, Quản lý giáo viên, Quản lý học sinh và Đăng ký/Đăng nhập. *Nhóm ngân hàng câu hỏi và đề thi* gồm Quản lý câu hỏi và Quản lý bài kiểm tra. *Nhóm chấm thi và phúc khảo* gồm Quản lý các bài thi của học sinh, Chấm thi bằng ứng dụng, Phúc khảo bài thi và Xem bài thi và điểm số. Trong đó, use case Phúc khảo bài thi bao trùm toàn bộ quy trình từ phía học sinh (gửi đơn phúc khảo) đến phía giáo viên (phản hồi phúc khảo) nhằm phản ánh đúng bản chất end-to-end của nghiệp vụ này. Các use case này được phân rã chi tiết ở các mục tiếp theo.

2.2.1 Biểu đồ use case phân rã Đăng ký/Đăng nhập

Biểu đồ use case phân rã Đăng ký/Đăng nhập mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Đăng ký/Đăng nhập, được thể hiện trong Hình 2.2.

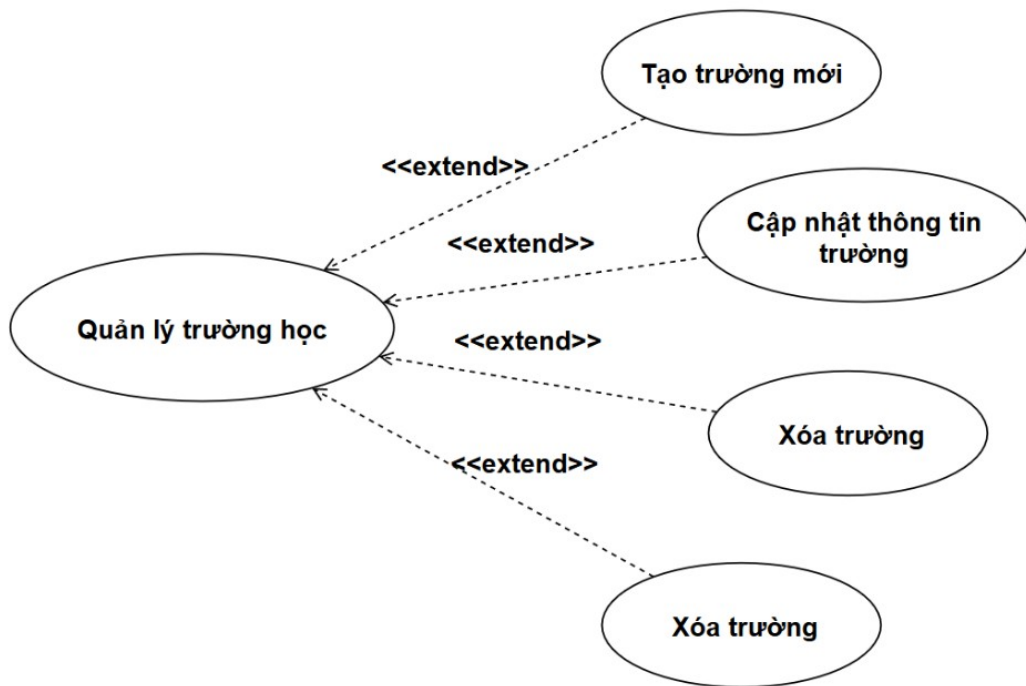


Hình 2.2: Biểu đồ use case phân rã Đăng ký/Đăng nhập

Tác nhân chính là tất cả bốn vai trò người dùng.

2.2.2 Biểu đồ use case phân rã Quản lý trường học

Biểu đồ use case phân rã Quản lý trường học mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Quản lý trường học, được thể hiện trong Hình 2.3.



Hình 2.3: Biểu đồ use case phân rã Quản lý trường học

Tác nhân chính là Quản trị viên hệ thống.

2.2.3 Biểu đồ use case phân rã Quản lý giáo viên

Biểu đồ use case phân rã Quản lý giáo viên mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Quản lý giáo viên, được thể hiện trong Hình 2.4.

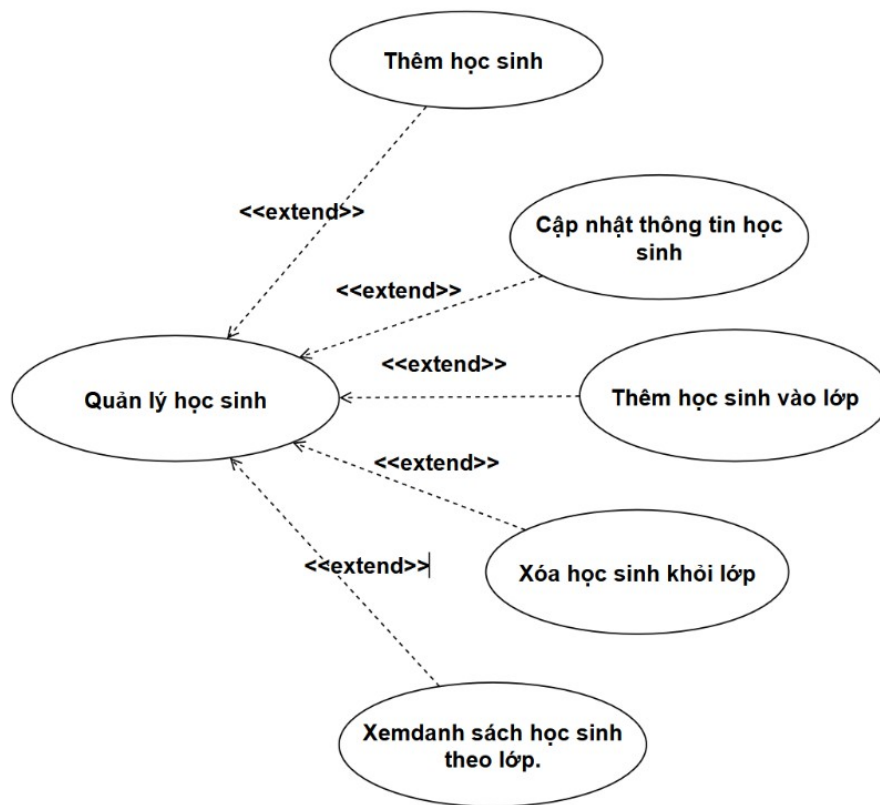


Hình 2.4: Biểu đồ use case phân rã Quản lý giáo viên

Tác nhân chính là Quản trị viên hệ thống và Quản trị viên trường.

2.2.4 Biểu đồ use case phân rã Quản lý học sinh

Biểu đồ use case phân rã Quản lý học sinh mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Quản lý học sinh, được thể hiện trong Hình 2.5.



Hình 2.5: Biểu đồ use case phân rã Quản lý học sinh

Tác nhân chính là Quản trị viên trường và Giáo viên.

2.2.5 Biểu đồ use case phân rã Quản lý câu hỏi

Biểu đồ use case phân rã Quản lý câu hỏi mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Quản lý câu hỏi, được thể hiện trong Hình 2.6.

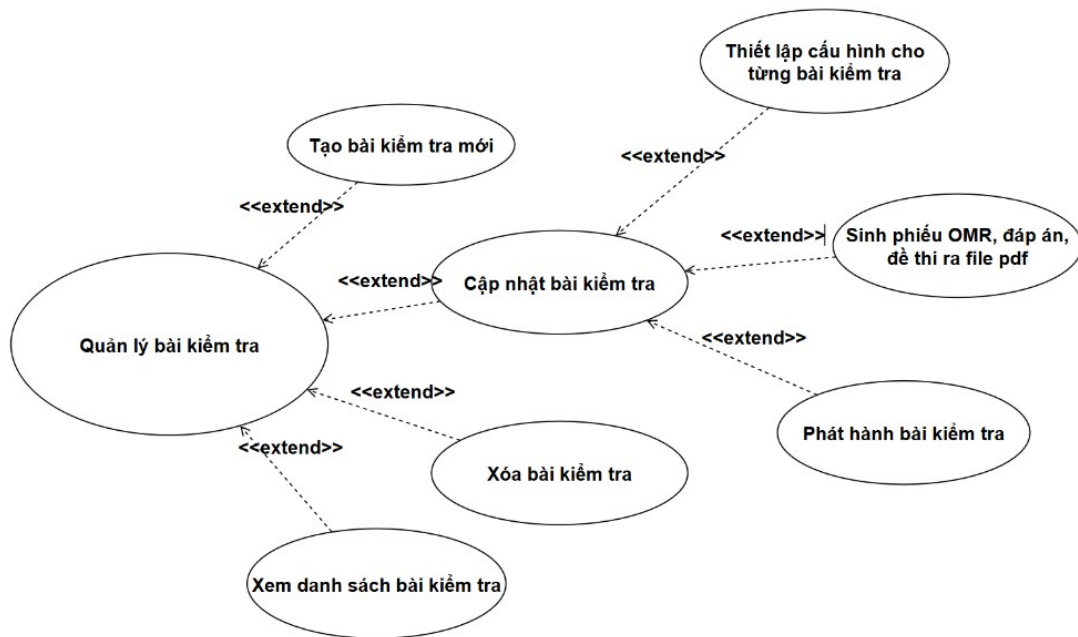


Hình 2.6: Biểu đồ use case phân rã Quản lý câu hỏi

Tác nhân chính là Giáo viên, có sự hỗ trợ của System thông qua các dịch vụ AI.

2.2.6 Biểu đồ use case phân rã Quản lý bài kiểm tra

Biểu đồ use case phân rã Quản lý bài kiểm tra mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Quản lý bài kiểm tra, được thể hiện trong Hình 2.7.



Hình 2.7: Biểu đồ use case phân rã Quản lý bài kiểm tra

Tác nhân chính là Giáo viên.

2.2.7 Biểu đồ use case phân rã Quản lý các bài thi của học sinh

Biểu đồ use case phân rã Quản lý các bài thi của học sinh mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Quản lý các bài thi của học sinh, được thể hiện trong Hình 2.8.

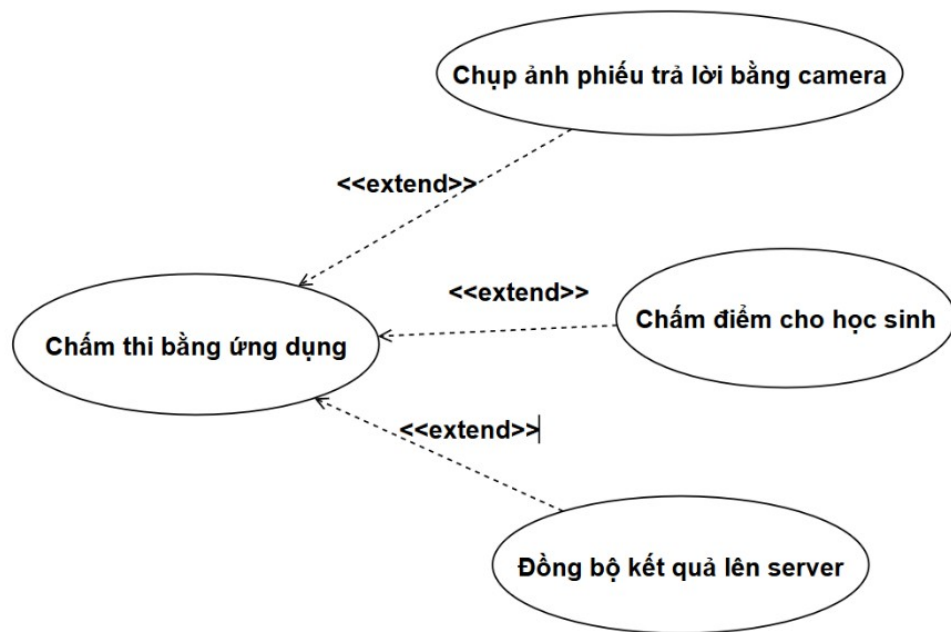


Hình 2.8: Biểu đồ use case phân rã Quản lý các bài thi của học sinh

Tác nhân chính là Giáo viên.

2.2.8 Biểu đồ use case phân rã Chấm thi bằng ứng dụng

Biểu đồ use case phân rã Chấm thi bằng ứng dụng mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Chấm thi bằng ứng dụng, được thể hiện trong Hình 2.9.

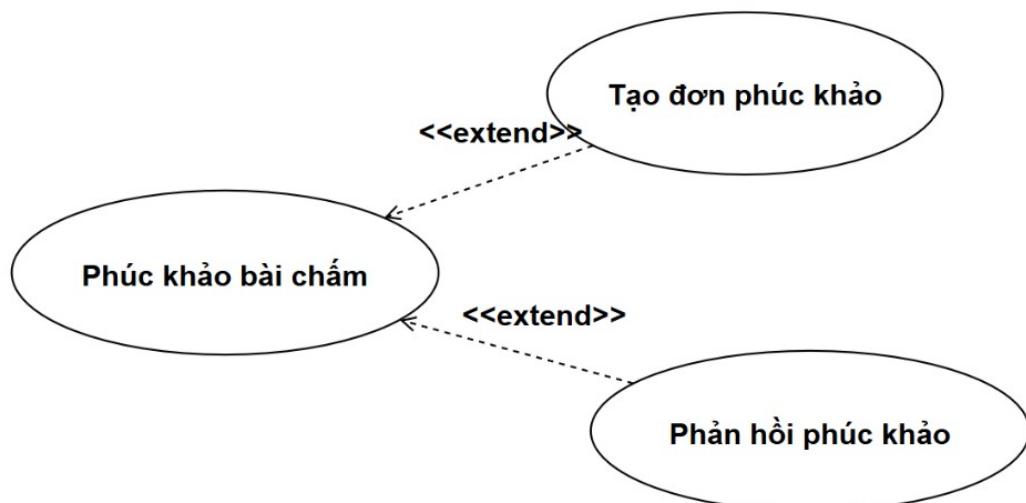


Hình 2.9: Biểu đồ use case phân rã Chấm thi bằng ứng dụng

Tác nhân chính là Giáo viên thông qua thiết bị di động.

2.2.9 Biểu đồ use case phân rã Phúc khảo bài thi

Biểu đồ use case phân rã Phúc khảo bài thi mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Phúc khảo bài thi, được thể hiện trong Hình 2.10.

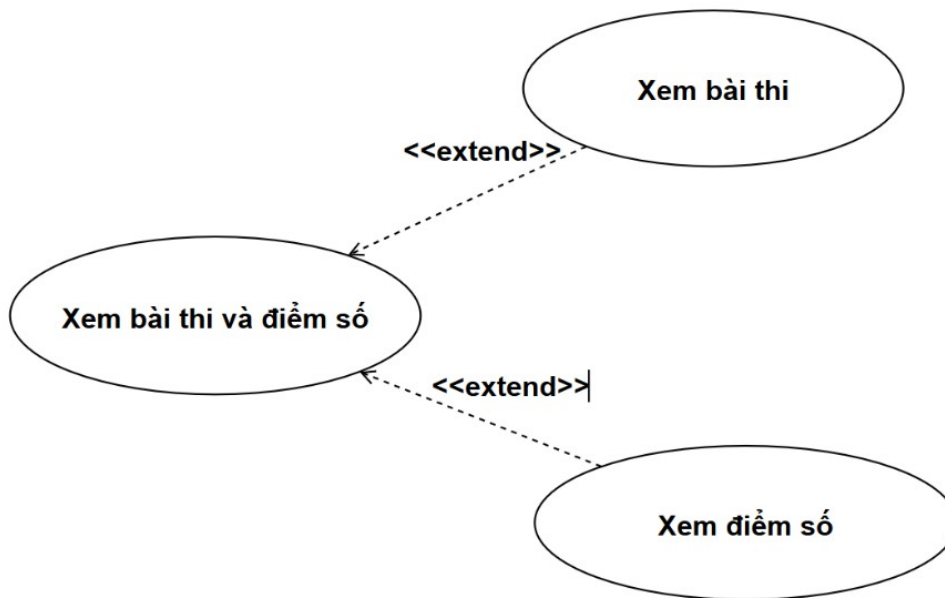


Hình 2.10: Biểu đồ use case phân rã Phúc khảo bài thi

Tác nhân chính là Học sinh (phía gửi đơn) và Giáo viên (phía phản hồi).

2.2.10 Biểu đồ use case phân rã Xem bài thi và điểm số

Biểu đồ use case phân rã Xem bài thi và điểm số mô tả chi tiết các chức năng con và mối quan hệ giữa các tác nhân với use case Xem bài thi và điểm số, được thể hiện trong Hình 2.11.



Hình 2.11: Biểu đồ use case phân rã Xem bài thi và điểm số

Tác nhân chính là Học sinh.

2.2.11 Quy trình nghiệp vụ

Hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR có một số quy trình nghiệp vụ quan trọng kết hợp nhiều use case với nhau. Quy trình nghiệp vụ "Tổ chức kỳ thi và chấm bài" được mô tả tổng quan như sau: đầu tiên giáo viên quản lý câu hỏi trong ngân hàng và tạo bài kiểm tra, hệ thống tự động trộn đề thành nhiều phiên bản và sinh phiếu OMR tương ứng; giáo viên in phiếu và phát cho học sinh làm bài; sau khi thu phiếu, giáo viên sử dụng ứng dụng di động để chấm từng phiếu; kết quả chấm được đồng bộ về server và công bố cho học sinh; học sinh có thể xem điểm và gửi đơn phúc tra nếu cần; giáo viên tiếp nhận và phản hồi phúc tra.

Quy trình nghiệp vụ "Quản lý đào tạo" gồm: admin tạo trường học và tài khoản quản trị trường; quản trị trường tạo năm học, khối lớp, môn học và tài khoản giáo viên; giáo viên được phân công phụ trách lớp và môn học, tạo lớp và thêm học sinh vào lớp; học sinh được cấp tài khoản và đăng nhập vào hệ thống để xem thông tin cá nhân, kết quả học tập.

Quy trình nghiệp vụ ”Phúc tra” gồm: học sinh xem bài thi và phát hiện điểm chưa chính xác, gửi đơn phúc tra kèm lý do và bằng chứng; giáo viên nhận thông báo, mở chi tiết bài làm của học sinh, so sánh với đáp án; giáo viên phê duyệt hoặc từ chối đơn phúc tra kèm phản hồi; nếu phê duyệt, điểm của học sinh được cập nhật lại và thông báo kết quả được gửi cho học sinh.

2.3 Đặc tả chức năng

Căn cứ vào tầm quan trọng và độ phức tạp, đề án lựa chọn đặc tả chi tiết cho 6 use case chính gồm: Quản lý giáo viên, Quản lý câu hỏi, Quản lý bài kiểm tra, Chấm thi bằng ứng dụng, Phúc khảo bài thi và Quản lý các bài thi của học sinh. Mỗi đặc tả gồm mã use case, tên use case, tác nhân, điều kiện trước, luồng thực thi chính, luồng thực thi mở rộng và điều kiện kết thúc. Đặc tả đầy đủ cho các use case còn lại được trình bày trong Phụ lục B. Trong đó, use case Phúc khảo bài thi được trình bày gộp cả phía học sinh (gửi đơn) và phía giáo viên (phản hồi) nhằm thể hiện đúng bản chất end-to-end của nghiệp vụ phúc khảo.

2.3.1 Đặc tả use case Quản lý giáo viên

Thông tin chung	Chi tiết
Mã UC (UC #)	UC003
Tên use case	Quản lý giáo viên
Tác nhân	Quản trị viên hệ thống (admin), Quản trị viên trường (school-admin)
Điều kiện trước	Quản trị viên đã đăng nhập vào hệ thống với vai trò phù hợp.

Bảng 2.2: Thông tin chung use case Quản lý giáo viên

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Quản trị viên	Truy cập chức năng "Quản lý giáo viên" trên giao diện web.
2	Hệ thống	Hiển thị danh sách giáo viên hiện có kèm các điều khiển tìm kiếm, lọc theo môn và trạng thái.
3	Quản trị viên	Chọn chức năng "Thêm giáo viên mới".
4	Hệ thống	Hiển thị form yêu cầu nhập họ tên, email, số điện thoại, môn phụ trách và các thông tin liên quan.
5	Quản trị viên	Nhập thông tin giáo viên và nhấn nút "Lưu".
6	Hệ thống	Kiểm tra tính hợp lệ, kiểm tra email trùng lặp và tạo tài khoản giáo viên với trạng thái "chưa kích hoạt".
7	Hệ thống	Gửi email kích hoạt tài khoản cho giáo viên và hiển thị thông báo thành công trên giao diện.
8	Quản trị viên	Chọn một giáo viên trong danh sách để cập nhật thông tin hoặc vô hiệu hóa tài khoản.
9	Hệ thống	Cập nhật thông tin giáo viên tương ứng và ghi nhận nhật ký thao tác vào cơ sở dữ liệu.

Bảng 2.3: Luồng thực thi chính của use case Quản lý giáo viên

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
6a	Hệ thống	Thông báo lỗi: Email đã được sử dụng. Vui lòng nhập email khác.
6b	Hệ thống	Thông báo lỗi: Vui lòng nhập đầy đủ các trường bắt buộc (họ tên, email, môn phụ trách).
9a	Hệ thống	Thông báo cảnh báo: Giáo viên đang phụ trách lớp học. Bạn có chắc chắn muốn vô hiệu hóa tài khoản?

Bảng 2.4: Luồng thực thi mở rộng của use case Quản lý giáo viên

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, tài khoản giáo viên được tạo hoặc cập nhật trong hệ thống, email kích hoạt đã được gửi. Nếu thất bại, trạng thái danh sách giáo viên không thay đổi và giáo viên chưa được tạo.

Bảng 2.5: Điều kiện kết thúc use case Quản lý giáo viên

2.3.2 Đặc tả use case Quản lý câu hỏi

Thông tin chung	Chi tiết
Mã UC (UC #)	UC005
Tên use case	Quản lý câu hỏi
Tác nhân	Giáo viên (teacher); hỗ trợ bởi System (dịch vụ AI)
Điều kiện trước	Giáo viên đã đăng nhập vào hệ thống và có quyền quản lý ngân hàng câu hỏi.

Bảng 2.6: Thông tin chung use case Quản lý câu hỏi

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Giáo viên	Truy cập chức năng "Ngân hàng câu hỏi" trên giao diện web.
2	Hệ thống	Hiển thị danh sách câu hỏi hiện có kèm các điều khiển tìm kiếm, lọc theo môn, chủ đề, độ khó.
3	Giáo viên	Chọn "Tạo câu hỏi mới" và chọn hình thức: tạo thủ công, sinh bằng AI hoặc nhập khẩu từ file.
4	Giáo viên	Đối với tạo thủ công: nhập nội dung, đáp án, mức độ khó, chủ đề, môn học trong form soạn thảo.
5	Giáo viên	Đối với sinh bằng AI: nhập chủ đề hoặc tài liệu tham khảo; nhấn "Sinh câu hỏi".
6	Hệ thống	Đối với sinh bằng AI: gọi dịch vụ AI để sinh danh sách câu hỏi gợi ý và hiển thị cho giáo viên.
7	Giáo viên	Đối với nhập khẩu: tải lên file theo định dạng hỗ trợ (Excel/CSV).
8	Hệ thống	Đối với nhập khẩu: đọc file và tạo câu hỏi hàng loạt, hiển thị danh sách để giáo viên kiểm tra.
9	Giáo viên	Kiểm tra, chỉnh sửa nội dung câu hỏi và nhấn nút "Lưu".
10	Hệ thống	Kiểm tra tính hợp lệ (đủ đáp án, đáp án đúng được chọn) và lưu câu hỏi vào ngân hàng.
11	Giáo viên	Chọn một câu hỏi bất kỳ để cập nhật hoặc xóa.
12	Hệ thống	Cập nhật hoặc xóa câu hỏi tương ứng; cảnh báo nếu câu hỏi đang được sử dụng trong đề thi.

Bảng 2.7: Luồng thực thi chính của use case Quản lý câu hỏi

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
6a	Hệ thống	Thông báo lỗi: Dịch vụ AI gặp sự cố. Vui lòng thử lại hoặc chuyển sang tạo câu hỏi thủ công.
8a	Hệ thống	Thông báo lỗi: File không đúng định dạng. Vui lòng kiểm tra lại và tải lại.
10a	Hệ thống	Thông báo lỗi: Câu hỏi thiếu đáp án đúng. Vui lòng bổ sung trước khi lưu.
12a	Hệ thống	Thông báo cảnh báo: Câu hỏi đang được sử dụng trong đề thi. Bạn có chắc chắn muốn xóa?

Bảng 2.8: Luồng thực thi mở rộng của use case Quản lý câu hỏi

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, ngân hàng câu hỏi được cập nhật với câu hỏi mới hoặc thay đổi tương ứng. Nếu thất bại, trạng thái ngân hàng câu hỏi không thay đổi.

Bảng 2.9: Điều kiện kết thúc use case Quản lý câu hỏi

2.3.3 Đặc tả use case Quản lý bài kiểm tra

Thông tin chung	Chi tiết
Mã UC (UC #)	UC006
Tên use case	Quản lý bài kiểm tra
Tác nhân	Giáo viên (teacher)
Điều kiện trước	Giáo viên đã đăng nhập vào hệ thống và đã có ngân hàng câu hỏi thuộc môn học tương ứng.

Bảng 2.10: Thông tin chung use case Quản lý bài kiểm tra

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Giáo viên	Truy cập chức năng "Quản lý bài kiểm tra" trên giao diện web.
2	Hệ thống	Hiển thị danh sách bài kiểm tra hiện có kèm các điều khiển lọc theo lớp, môn, trạng thái.
3	Giáo viên	Chọn "Tạo bài kiểm tra mới".
4	Hệ thống	Hiển thị form tạo bài kiểm tra với các trường: tiêu đề, mô tả, lớp áp dụng, môn học, thời gian làm bài, tổng điểm, số lượng phiên bản cần tạo.
5	Giáo viên	Nhập thông tin chung và chọn câu hỏi từ ngân hàng câu hỏi theo chủ đề hoặc độ khó.
6	Giáo viên	Cấu hình tham số trộn đề: trộn thứ tự câu hỏi (Có/Không), trộn thứ tự đáp án (Có/Không), số lượng phiên bản cần tạo.
7	Giáo viên	Nhấn nút "Sinh đề thi" để yêu cầu hệ thống tạo các phiên bản đề.
8	Hệ thống	Tạo các phiên bản đề thi bằng cách hoán vị câu hỏi và đáp án theo cấu hình, đảm bảo đáp án đúng được cập nhật tương ứng.
9	Hệ thống	Sinh mã nguồn LaTeX sử dụng gói AMC cho từng phiên bản đề, biên dịch sang PDF và tải lên dịch vụ lưu trữ đám mây.
10	Hệ thống	Sinh phiếu trả lời OMR kèm tọa độ nhận dạng cho từng phiên bản, lưu thông tin tọa độ vào cơ sở dữ liệu.
11	Giáo viên	Xem lại danh sách các phiên bản đề đã tạo, tải về file PDF đề thi và phiếu trả lời.
12	Giáo viên	Nhấn nút "Phát hành" để công bố bài kiểm tra cho học sinh.
13	Hệ thống	Gửi thông báo đến học sinh thuộc các lớp áp dụng và cập nhật trạng thái bài kiểm tra thành "đã phát hành".
14	Giáo viên	Chọn một bài kiểm tra để cập nhật thông tin hoặc xóa.

Bảng 2.11: Luồng thực thi chính của use case Quản lý bài kiểm tra

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
5a	Hệ thống	Thông báo lỗi: Thiếu thông tin bắt buộc. Vui lòng bổ sung tiêu đề, môn học và số lượng phiên bản.
5b	Hệ thống	Thông báo cảnh báo: Ngân hàng câu hỏi không đủ số lượng cần thiết. Đề xuất giảm số lượng phiên bản hoặc bổ sung câu hỏi.
8a	Hệ thống	Thông báo lỗi: Có lỗi khi trộn đề. Hệ thống giữ nguyên bản nháp. Vui lòng thử lại.
9a	Hệ thống	Thông báo lỗi: Biên dịch LaTeX thất bại. Hệ thống ghi nhận log lỗi và cho phép giáo viên thử lại.
14a	Hệ thống	Thông báo cảnh báo: Bài kiểm tra đã có bài làm của học sinh. Bạn có chắc chắn muốn xóa?

Bảng 2.12: Luồng thực thi mở rộng của use case Quản lý bài kiểm tra

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, bài kiểm tra được tạo với nhiều phiên bản, phiếu OMR tương ứng được sinh và học sinh nhận được thông báo. Nếu thất bại, trạng thái bài kiểm tra giữ nguyên.

Bảng 2.13: Điều kiện kết thúc use case Quản lý bài kiểm tra

2.3.4 Đặc tả use case Chấm thi bằng ứng dụng

Thông tin chung	Chi tiết
Mã UC (UC #)	UC008
Tên use case	Chấm thi bằng ứng dụng
Tác nhân	Giáo viên (teacher), thông qua thiết bị di động
Điều kiện trước	Giáo viên đã đăng nhập trên ứng dụng di động, có bài thi đã được phát hành và phiếu OMR tương ứng.

Bảng 2.14: Thông tin chung use case Chấm thi bằng ứng dụng

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Giáo viên	Mở ứng dụng di động Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR và chọn chức năng "Quét phiếu".
2	Ứng dụng	Yêu cầu quyền truy cập camera và hiển thị giao diện chụp ảnh.
3	Giáo viên	Đặt phiếu trả lời trên bề mặt phẳng, căn chỉnh vào khung hình và chụp ảnh.
4	Ứng dụng	Tải thông tin mẫu OMR tương ứng với bài thi từ server.
5	Ứng dụng	Tiền xử lý ảnh: chuyển sang thang xám, làm mờ Gaussian, nhị phân hóa bằng phương pháp Otsu, phát hiện và chỉnh nghiêng phiếu, cắt vùng phiếu.
6	Ứng dụng	Phát hiện các ô trả lời bằng cách so sánh vị trí với tọa độ đã lưu trong mẫu OMR.
7	Ứng dụng	Đánh giá mức độ tô của từng ô, xác định ô được chọn dựa trên ngưỡng diện tích tô, phát hiện lỗi tô nhầm (nhiều ô trong cùng một câu) hoặc tô nhẹ.
8	Ứng dụng	Nhận dạng mã đề và mã số sinh viên từ vùng đầu phiếu.
9	Ứng dụng	Đối chiếu câu trả lời của học sinh với đáp án của phiên bản đề tương ứng, tính điểm cho từng câu và tổng điểm.
10	Ứng dụng	Hiển thị kết quả chấm cho giáo viên kèm ảnh phiếu đã được đánh dấu.
11	Giáo viên	Kiểm tra và chỉnh sửa thủ công các câu bị nhận dạng sai hoặc đánh dấu là không hợp lệ.
12	Giáo viên	Nhấn nút "Lưu kết quả" để đồng bộ kết quả lên server.

Bảng 2.15: Luồng thực thi chính của use case Chấm thi bằng ứng dụng

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
5a	Ứng dụng	Thông báo lỗi: Ảnh chụp mờ hoặc nghiêng quá nhiều. Vui lòng chụp lại.
8a	Ứng dụng	Thông báo lỗi: Không nhận dạng được mã đề. Cho phép giáo viên chọn thủ công mã đề từ danh sách.
7a	Ứng dụng	Thông báo cảnh báo: Phát hiện câu tô nhầm hoặc tô nhẹ. Yêu cầu giáo viên xác nhận.
12a	Ứng dụng	Thông báo: Mất kết nối mạng. Kết quả được lưu cục bộ và tự động đồng bộ khi có kết nối.

Bảng 2.16: Luồng thực thi mở rộng của use case Chấm thi bằng ứng dụng

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, bài làm của học sinh được lưu trên server với điểm số đã tính, học sinh có thể xem điểm sau khi giáo viên phát hành kết quả. Nếu thất bại, ứng dụng giữ kết quả ở trạng thái chờ xử lý.

Bảng 2.17: Điều kiện kết thúc use case Chấm thi bằng ứng dụng

2.3.5 Đặc tả use case Phúc khảo bài thi

Use case Phúc khảo bài thi bao trùm toàn bộ quy trình phúc khảo từ phía học sinh (gửi đơn phúc khảo) đến phía giáo viên (phản hồi phúc khảo), thể hiện đúng bản chất end-to-end của nghiệp vụ phúc khảo trong hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR.

Thông tin chung	Chi tiết
Mã UC (UC #)	UC009
Tên use case	Phúc khảo bài thi
Tác nhân	Học sinh (student), Giáo viên (teacher)
Điều kiện trước	Học sinh đã đăng nhập vào hệ thống, đã có kết quả bài thi được công bố và còn trong thời hạn phúc khảo.

Bảng 2.18: Thông tin chung use case Phúc khảo bài thi

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Học sinh	Truy cập chức năng "Bài thi của tôi" và chọn bài thi muốn phúc khảo.
2	Hệ thống	Hiển thị chi tiết bài thi gồm: điểm tổng, điểm từng câu, ảnh phiếu đã chấm và đáp án đúng.
3	Học sinh	Chọn chức năng "Tạo đơn phúc khảo" và chọn một hoặc nhiều câu cần phúc khảo.
4	Hệ thống	Hiển thị form yêu cầu nhập lý do phúc khảo cho từng câu, cho phép đính kèm hình ảnh minh chứng (nếu có).
5	Học sinh	Nhập lý do, đính kèm bằng chứng (nếu có) và nhấn nút "Gửi đơn".
6	Hệ thống	Kiểm tra thời hạn phúc khảo và tính hợp lệ của thông tin.
7	Hệ thống	Tạo đơn phúc khảo mới với trạng thái "đang chờ", gửi thông báo cho giáo viên phụ trách.
8	Hệ thống	Gửi thông báo xác nhận cho học sinh và hiển thị trạng thái đơn phúc khảo trong danh sách "Đơn phúc khảo của tôi".
9	Giáo viên	Truy cập chức năng "Phúc khảo" và xem danh sách đơn phúc khảo đang chờ.
10	Giáo viên	Chọn một đơn phúc khảo để xem chi tiết.
11	Hệ thống	Hiển thị thông tin bài thi của học sinh, câu được phúc khảo, lý do và hình ảnh minh chứng (nếu có).
12	Giáo viên	So sánh câu trả lời của học sinh với đáp án, đối chiếu với hình ảnh phiếu gốc.
13	Giáo viên	Quyết định phê duyệt hoặc từ chối và nhập nội dung phản hồi cho học sinh.
14	Giáo viên	Nhấn nút "Gửi phản hồi" để hoàn tất.
15	Hệ thống	Cập nhật trạng thái đơn phúc khảo thành "đã phê duyệt" hoặc "đã từ chối".
16	Hệ thống	Nếu phê duyệt, tính lại điểm cho câu được phúc khảo và cập nhật tổng điểm của học sinh.
17	Hệ thống	Gửi thông báo kết quả phúc khảo cho học sinh.

Bảng 2.19: Luồng thực thi chính của use case Phúc khảo bài thi

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
6a	Hệ thống	Thông báo lỗi: Đã hết thời hạn phúc khảo. Không thể gửi đơn.
6b	Hệ thống	Thông báo lỗi: Bạn đã gửi đơn phúc khảo cho câu này. Vui lòng xem trạng thái đơn cũ.
4a	Hệ thống	Thông báo lỗi: Lý do phúc khảo quá ngắn. Vui lòng nhập chi tiết hơn (tối thiểu 20 ký tự).
13a	Hệ thống	Thông báo: Đơn đang được đánh dấu "đang xử lý". Giáo viên có thể tiếp tục xử lý sau.
16a	Hệ thống	Thông báo lỗi: Có lỗi khi tính lại điểm. Hệ thống giữ nguyên điểm cũ và thông báo cho quản trị hệ thống.
14a	Hệ thống	Thông báo cảnh báo: Đơn đã hết thời hạn xử lý. Bạn có chắc chắn muốn gửi phản hồi?

Bảng 2.20: Luồng thực thi mở rộng của use case Phúc khảo bài thi

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, đơn phúc khảo được đóng và điểm được cập nhật (nếu phê duyệt). Học sinh nhận được thông báo kết quả phúc khảo. Nếu thất bại ở bất kỳ bước nào, trạng thái đơn và điểm không bị thay đổi.

Bảng 2.21: Điều kiện kết thúc use case Phúc khảo bài thi

2.4 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng đã nêu, hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR cần đáp ứng các yêu cầu phi chức năng sau:

Yêu cầu về hiệu năng. Hệ thống phải đảm bảo thời gian phản hồi của các API dưới 2 giây đối với các thao tác thông thường và dưới 5 giây đối với các thao tác tổng hợp dữ liệu lớn như tạo báo cáo. Thời gian chấm một phiếu trả lời bằng ứng dụng di động không quá 3 giây với điều kiện ảnh đầu vào đạt chất lượng.

Yêu cầu về độ tin cậy. Hệ thống phải đảm bảo độ chính xác của thuật toán nhận dạng OMR đạt tối thiểu 95% trên các điều kiện ảnh chụp thông thường. Hệ thống phải có cơ chế ghi log đầy đủ để truy vết lỗi và phải có khả năng khôi phục sau sự cố mà không mất dữ liệu đã chấm.

Yêu cầu về tính dễ dùng. Giao diện người dùng phải được Việt hóa hoàn toàn, sử dụng phông chữ dễ đọc và có độ tương phản phù hợp. Các thao tác chính phải được thực hiện trong vòng không quá 3 lần nhấp chuột hoặc chạm. Ứng dụng di động phải hoạt động ổn định trên các thiết bị có cấu hình trung bình với hệ điều hành Android từ 7.0 trở lên.

Yêu cầu về tính bảo trì. Mã nguồn phải được tổ chức theo kiến trúc phân lớp rõ

ràng, có tài liệu hướng dẫn cài đặt và vận hành. Hệ thống phải có bộ kiểm thử tự động cho các thành phần cốt lõi với độ phủ kiểm thử tối thiểu 60%.

Yêu cầu về bảo mật. Hệ thống phải sử dụng JWT cho xác thực, mật khẩu phải được lưu trữ dưới dạng mã băm bcrypt. Các điểm cuối API phải được phân quyền theo vai trò người dùng. Ảnh phiếu thi và dữ liệu cá nhân của học sinh phải được bảo vệ khỏi truy cập trái phép.

Yêu cầu về cơ sở dữ liệu và công nghệ. Hệ thống sử dụng cơ sở dữ liệu MongoDB phiên bản 5.0 trở lên để lưu trữ dữ liệu nghiệp vụ. Hệ thống phải hỗ trợ triển khai trên cả môi trường phát triển (local) và môi trường đám mây (cloud). Backend chạy trên Node.js v18 trở lên, frontend web xây dựng trên React 18 với TypeScript, ứng dụng di động xây dựng trên Flutter 3.x.

Chương 2 đã trình bày kết quả khảo sát hiện trạng, mô tả tổng quan chức năng hệ thống thông qua biểu đồ use case, đặc tả chi tiết các use case quan trọng và xác định các yêu cầu phi chức năng. Chương 3 sẽ giới thiệu các nền tảng lý thuyết và công nghệ được lựa chọn để hiện thực hóa các yêu cầu đã nêu.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này giới thiệu các nền tảng lý thuyết và công nghệ được lựa chọn để xây dựng Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR. Mỗi công nghệ được phân tích vai trò giải quyết yêu cầu cụ thể và so sánh với lựa chọn thay thế để giải thích lý do lựa chọn.

3.1 Kỹ thuật nhận dạng OMR và xử lý ảnh số

Kỹ thuật nhận dạng dấu quang học (Optical Mark Recognition – OMR) là nền tảng cốt lõi để chấm thi tự động. OMR phát hiện sự có mặt hoặc vắng mặt của dấu tại vị trí xác định trên giấy, từ đó suy ra lựa chọn của người trả lời. Quy trình OMR gồm: thu nhận ảnh, tiền xử lý, phát hiện vùng phiếu và chỉnh nghiêng, phát hiện ô trả lời, phân loại trạng thái tô, và ánh xạ câu trả lời.

Trong đồ án, các bước này được cài đặt trong module xử lý ảnh của ứng dụng Flutter, sử dụng thư viện OpenCV [1] qua package `opencv_dart`. Các phép biến đổi như chuyển ảnh xám, lọc Gaussian, nhị phân hóa Otsu và phát hiện cạnh Canny được áp dụng dựa trên lý thuyết xử lý ảnh số [2]. Đồ án chọn hướng sử dụng camera di động thay vì máy quét chuyên dụng nhờ giảm chi phí và cho phép giáo viên chấm bài ngay tại lớp. Dự án mã nguồn mở OMRChecker [3] cung cấp pipeline tham khảo quý giá.

3.2 Kỹ thuật tạo đề nhiều phiên bản với AMC

Yêu cầu tạo nhiều phiên bản đề thi được giải quyết bằng gói LaTeX AMC (Auto-Multiple-Choice) [4]. AMC là dự án mã nguồn mở cung cấp công cụ toàn diện để tạo đề trắc nghiệm, sinh phiếu trả lời, chấm bài và phân tích kết quả, hỗ trợ trộn ngẫu nhiên thứ tự câu hỏi và đáp án.

Trong hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, AMC được tích hợp theo cơ chế: backend tạo mã nguồn LaTeX từ dữ liệu câu hỏi và cấu hình trộn đề; mã nguồn được biên dịch sang PDF bằng XeLaTeX qua WSL2; đồng thời sinh phiếu OMR và gọi chế độ `prepare --mode s --out-calage` của AMC để trích xuất file tọa độ `.calage.xy` chứa tọa độ chính xác các ô trên phiếu, phục vụ nhận dạng OMR. Các tọa độ được chuẩn hóa về pixel ở độ phân giải 300 DPI và lưu trong bảng OMRTemplate.

Hai hướng thay thế đã xem xét nhưng không chọn: tự viết module sinh PDF bằng `pdfkit` (khó đảm bảo tọa độ chính xác) và dùng thư viện Python `reportlab` (thêm phụ thuộc ngôn ngữ, khó tích hợp Node.js). AMC được chọn vì cung cấp giải pháp toàn diện và đã được kiểm chứng trong thực tế giáo dục.

3.3 Công nghệ phát triển backend

Backend Node.js [5] kết hợp Express [6] đáp ứng yêu cầu xử lý nghiệp vụ phức tạp. Backend được tổ chức theo biến thể MVC [7] gồm: routes (định tuyến API REST), controllers (điều phối request/response), services (logic nghiệp vụ) và models (schema Mongoose). Gói `services/` chứa logic theo miền: exam, submission, appeal, user, notification, ai.

Xác thực dùng JWT [8] và Passport.js [9], mật khẩu mã hóa bcrypt. Cơ sở dữ liệu MongoDB [10] qua Mongoose ODM [11] được chọn thay PostgreSQL vì linh hoạt schema phù hợp dữ liệu giáo dục (câu hỏi nhiều dạng, đề thi nhiều phiên bản).

3.4 Công nghệ phát triển giao diện web

Giao diện web xây dựng bằng React [12] 18 với TypeScript, sử dụng mô hình component giúp tổ chức giao diện thành các thành phần độc lập, tái sử dụng. TypeScript bổ sung kiểu tĩnh giúp phát hiện lỗi sớm và tăng tính dễ bảo trì.

Trạng thái client quản lý bằng Zustand [13] (nhẹ, API đơn giản), định tuyến dùng React Router v6 hỗ trợ phân quyền, đồng bộ dữ liệu dùng TanStack Query (caching, cập nhật tự động), biểu đồ thống kê dùng Recharts. React được chọn thay Angular (đường cong học cao) và Vue.js (hệ sinh thái TypeScript hạn chế) nhờ cộng đồng lớn và hệ sinh thái thư viện phong phú.

3.5 Công nghệ phát triển ứng dụng di động

Flutter [14] được sử dụng cho ứng dụng di động nhờ biên dịch đa nền tảng (Android/iOS) từ cùng mã nguồn, hiệu năng gần native nhờ biên dịch AOT, và hỗ trợ xử lý ảnh tốt qua OpenCV. Quản lý trạng thái dùng mô hình BLoC [15] phân tách logic nghiệp vụ khỏi giao diện. Giao tiếp backend dùng Dio, lưu trữ cục bộ dùng `flutter_secure_storage` và `sqflite`.

Flutter được chọn thay React Native (hiệu năng thấp hơn, khó tích hợp OpenCV) và phát triển native (đòi hai đội ngũ riêng, tăng chi phí).

3.6 Công nghệ trí tuệ nhân tạo tích hợp

Hệ thống tích hợp các mô hình ngôn ngữ lớn (LLM) qua nhiều nhà cung cấp: Gemini [16] (chính), OpenAI [17] và Claude (dự phòng). Các mô hình dựa trên kiến trúc Transformer [18].

AI được ứng dụng cho: sinh câu hỏi trắc nghiệm (module `questionGen.service.js` dùng prompt engineering và few-shot examples), sinh nhận xét báo cáo (`aiReport.service.js`) và trợ lý chat cho học sinh (`aiChat.service.js`). Hệ thống áp dụng cache 5 phút và exponential backoff khi gặp lỗi nhà cung cấp. Được chọn thay tự huấn luyện hoặc on-premise nhờ tiết kiệm chi phí, chất lượng cao và hỗ trợ tiếng Việt.

3.7 Các công nghệ hỗ trợ khác

Ngoài các công nghệ chính, hệ thống còn dùng Cloudinary [19] lưu trữ ảnh và PDF, Nodemailer gửi email, Winston và Morgan ghi log. Container hóa Docker với docker-compose cho ba môi trường dev/test/prod. Backend kiểm thử bằng Jest (unit và integration test), tài liệu API sinh tự động qua Swagger.

Chương 3 đã giới thiệu các nền tảng lý thuyết và công nghệ xây dựng Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR: OMR, AMC, Node.js/Express, MongoDB, React, Flutter, AI và các công cụ hỗ trợ. Chương 4 sẽ trình bày chi tiết phân tích thiết kế kiến trúc, giao diện, lớp và cơ sở dữ liệu.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày chi tiết quá trình phân tích thiết kế và xây dựng hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, bao gồm: lựa chọn kiến trúc phần mềm phân lớp kết hợp client-server, thiết kế tổng quan với biểu đồ gói UML theo từng tầng, thiết kế chi tiết các gói thành phần trên backend, web và mobile, thiết kế giao diện, lớp và cơ sở dữ liệu, kết quả xây dựng ứng dụng, kiểm thử và hướng dẫn triển khai.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR lựa chọn kiến trúc phân lớp kết hợp với mô hình client-server, phù hợp với đặc thù của ứng dụng có nhiều thành phần tương tác qua mạng. Toàn bộ hệ thống được chia thành ba tầng chính:

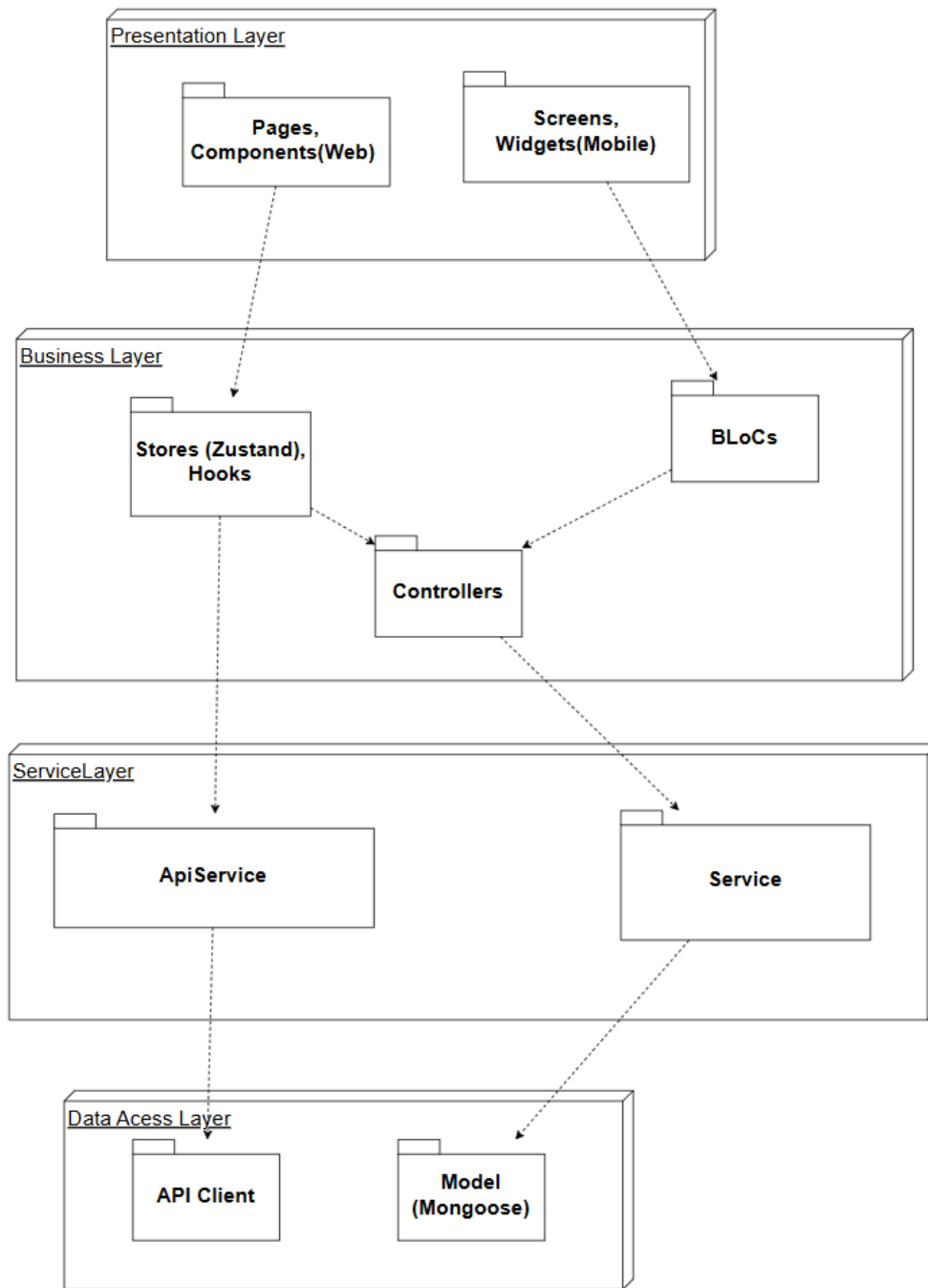
- tầng trình bày gồm ứng dụng di động Flutter và giao diện web React;
- tầng xử lý nghiệp vụ là backend Node.js/Express;
- tầng dữ liệu gồm MongoDB và dịch vụ lưu trữ đám mây Cloudinary.

Trong tầng backend, hệ thống áp dụng biến thể của mô hình MVC gồm bốn lớp: lớp định tuyến (routes) tiếp nhận và phân loại yêu cầu; lớp điều khiển (controllers) điều phối luồng xử lý và định dạng phản hồi; lớp dịch vụ (services) chứa toàn bộ logic nghiệp vụ; và lớp mô hình (models) quản lý tương tác với cơ sở dữ liệu. Các thành phần này được tổ chức thành các gói (module) độc lập theo từng miền nghiệp vụ như auth, user, exam, submission, appeal, question và notification.

Trong tầng giao diện web, kiến trúc áp dụng mô hình component với React và phân tách rõ ràng giữa ba lớp: lớp trình bày (pages và components), lớp quản lý trạng thái (Zustand stores) và lớp dịch vụ (services) để giao tiếp với backend thông qua REST API. Trong tầng di động, kiến trúc BLoC kết hợp với Repository pattern được áp dụng, trong đó BLoC xử lý logic nghiệp vụ, Repository đóng vai trò trung gian giữa BLoC và các nguồn dữ liệu (API server, lưu trữ cục bộ).

4.1.2 Thiết kế tổng quan

Biểu đồ gói UML của toàn bộ hệ thống được mô tả như trong Hình 4.1, thể hiện sự phụ thuộc một chiều giữa các gói theo từng tầng.



Hình 4.1: Biểu đồ gói tổng quan của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Nhóm gói tầng trình bày (Presentation Layer). Gồm hai gói con: web chứa các trang và component React, mobile chứa các màn hình và widget Flutter. Các gói này chỉ phụ thuộc vào tầng dịch vụ chung và không phụ thuộc trực tiếp vào tầng dữ liệu.

Nhóm gói tầng dịch vụ chung (Service Layer). Gồm ba gói: api-client (web) và network (mobile) cung cấp các hàm tiện ích để gọi REST API; state-management quản lý trạng thái phía client; routing định tuyến và phân quyền truy cập theo vai trò.

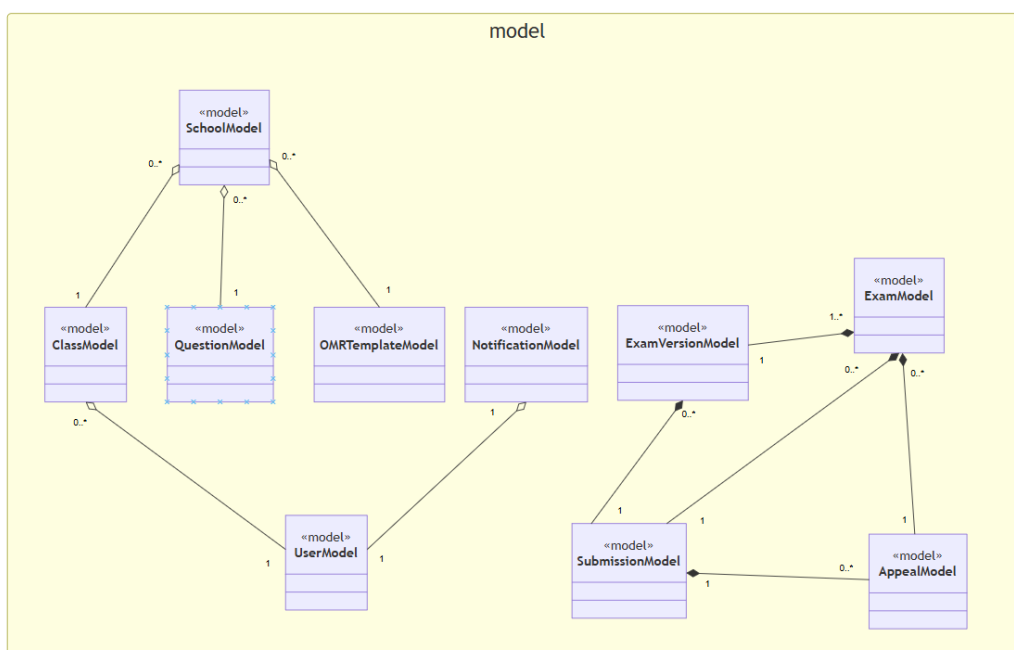
Nhóm gói tầng nghiệp vụ (Business Layer). Gồm các gói backend: routes định nghĩa điểm cuối API; controllers xử lý request; services chứa logic nghiệp vụ được chia theo miền; validations xác thực đầu vào; middlewares xử lý xác thực, phân quyền và ghi log.

Nhóm gói tầng dữ liệu (Data Layer). Gồm các gói models (định nghĩa schema MongoDB thông qua Mongoose), amc (sinh và xử lý đề thi bằng AMC), ai-services (tích hợp các nhà cung cấp AI), storage (tương tác với Cloudinary), utils (các hàm tiện ích chung).

4.1.3 Thiết kế chi tiết các gói

Phần này trình bày biểu đồ chi tiết cho từng gói thành phần trong kiến trúc hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, bao gồm cả backend Node.js, giao diện web React và ứng dụng di động Flutter.

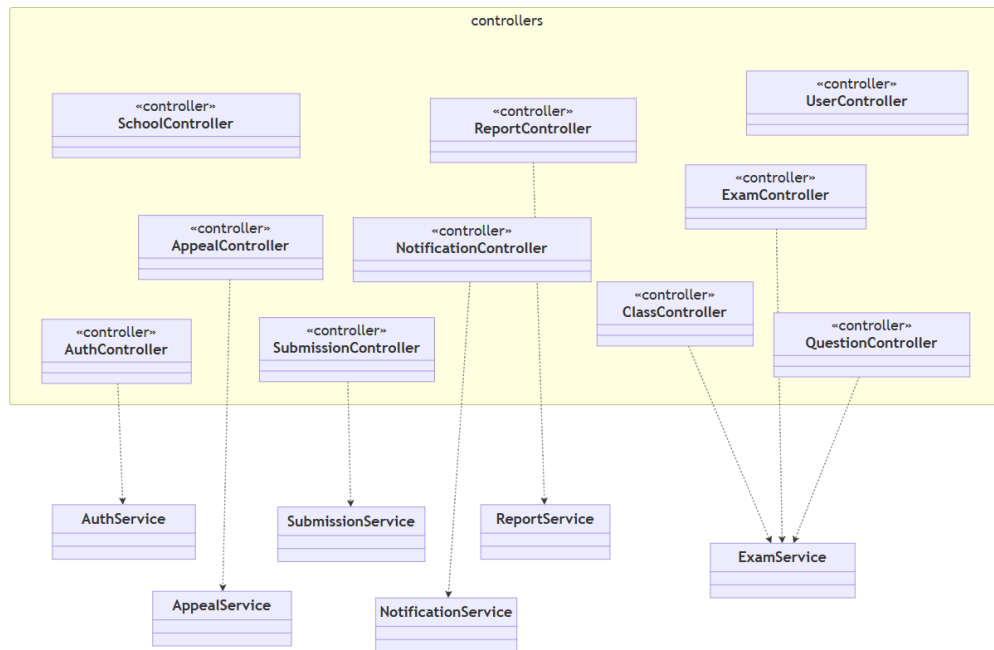
Gói Routers (Backend). Hệ thống backend sử dụng kiến trúc MVC với lớp routes đóng vai trò tiếp nhận và phân loại yêu cầu HTTP từ client. Mỗi routes được tổ chức theo miền nghiệp vụ và sử dụng Express Router. Biểu đồ các route chính được mô tả trong Hình 4.2.



Hình 4.2: Biểu đồ gói Routers của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

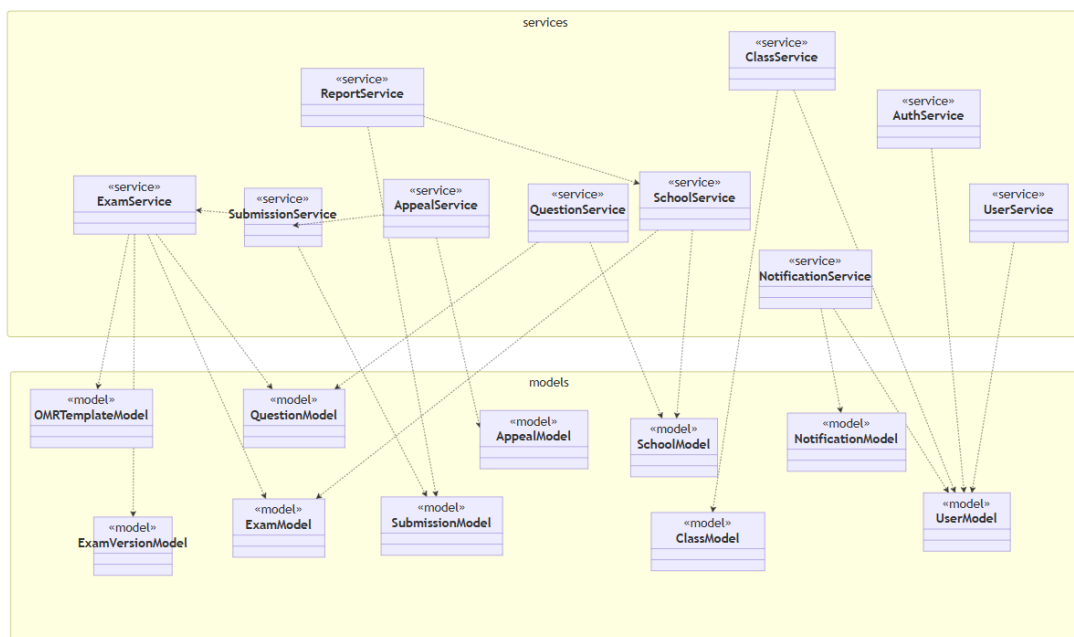
Gói Controllers (Backend). Controllers nhận yêu cầu từ routes, xử lý định dạng request/response và gọi các service tương ứng. Mỗi controller được thiết kế theo nguyên tắc đơn trách nhiệm (single responsibility). Biểu đồ các controller được mô

tả trong Hình 4.3.



Hình 4.3: Biểu đồ gói Controllers của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Gói Services (Backend). Services chứa toàn bộ logic nghiệp vụ, là lớp trung tâm của kiến trúc backend. Các service được chia theo miền nghiệp vụ và giao tiếp trực tiếp với các Mongoose model. Biểu đồ các service được mô tả trong Hình 4.4.

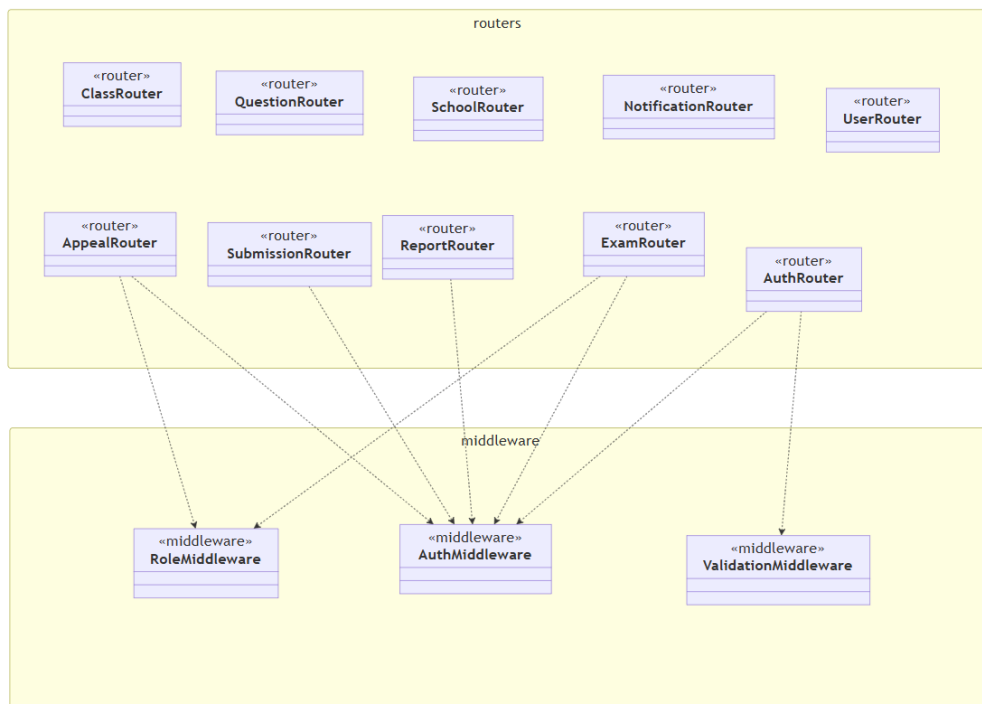


Hình 4.4: Biểu đồ gói Services của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Gói Models (Backend). Models định nghĩa schema MongoDB thông qua

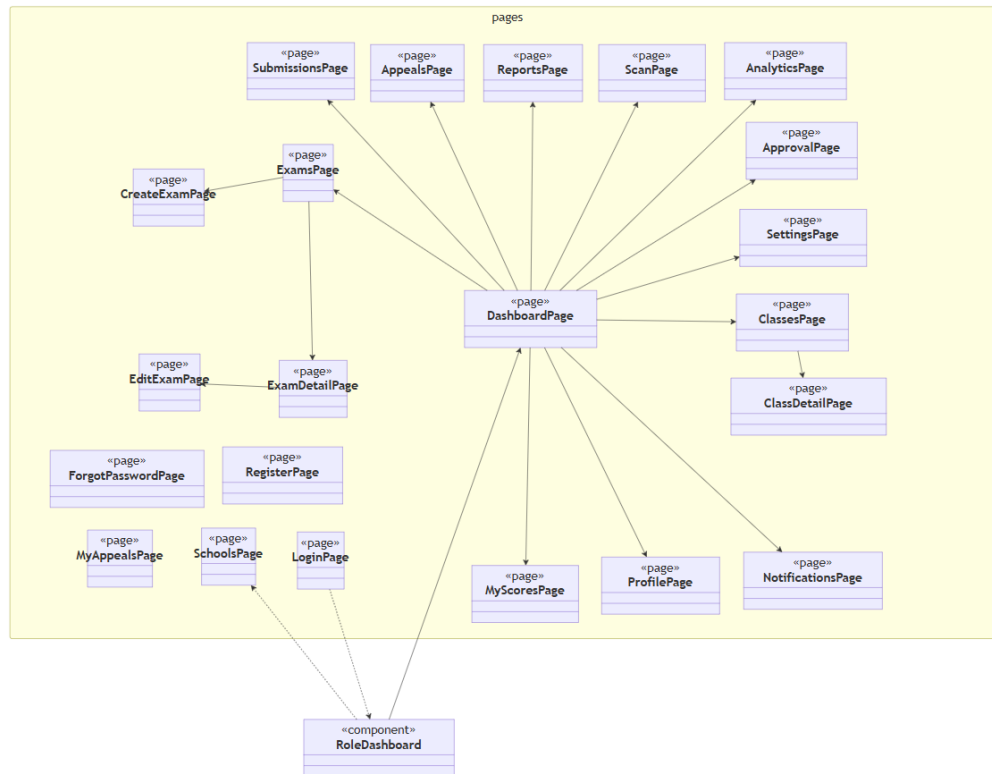
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mongoose ODM, mỗi model tương ứng với một collection trong cơ sở dữ liệu. Biểu đồ các model được mô tả trong Hình 4.5.

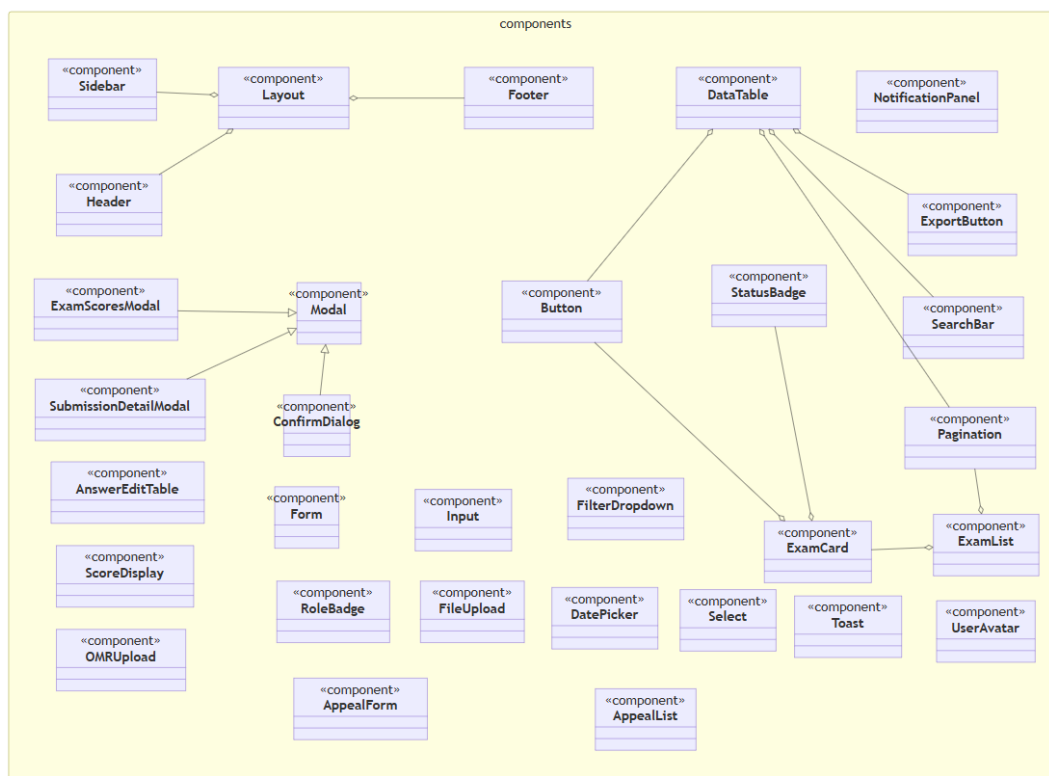


Hình 4.5: Biểu đồ gói Models của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Gói Pages và Components (Web React). Giao diện web được xây dựng theo mô hình component với React, trong đó pages chứa các trang chính và components chứa các thành phần giao diện có thể tái sử dụng. Biểu đồ các pages và components được mô tả trong Hình 4.6 và Hình 4.7.



Hình 4.6: Biểu đồ gói Pages của giao diện web React

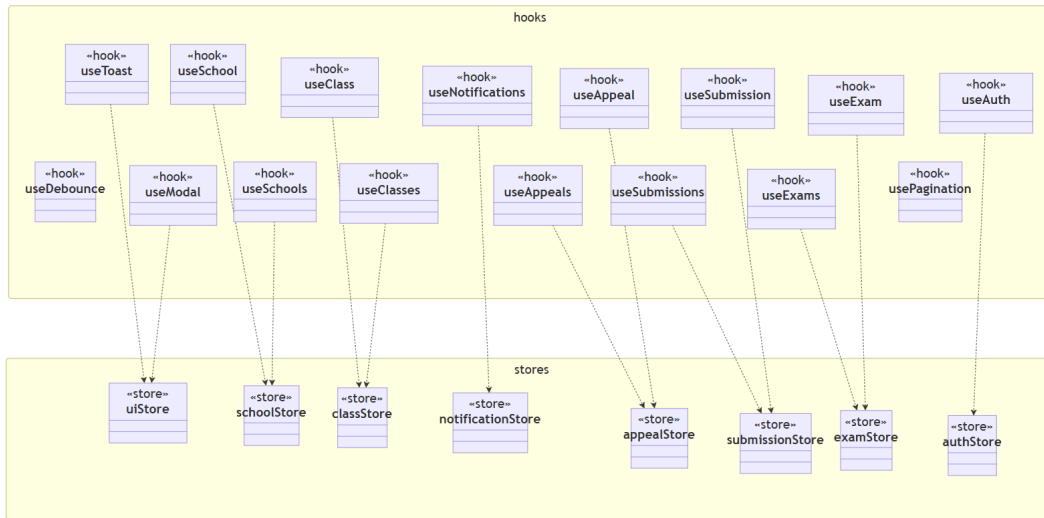


Hình 4.7: Biểu đồ gói Components của giao diện web React

Gói Stores và Hooks (Web React). Trạng thái ứng dụng web được quản lý thông

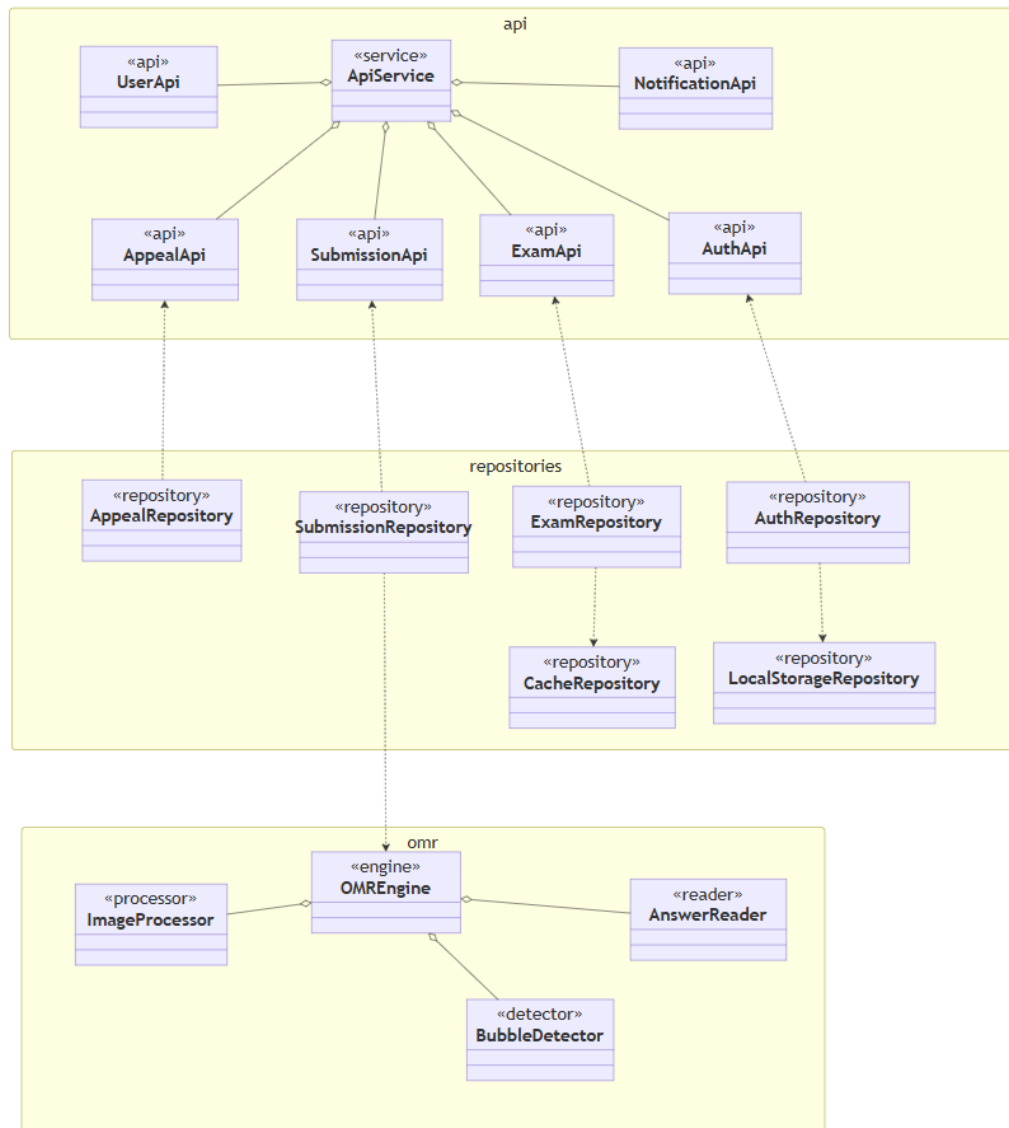
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

qua Zustand stores, kết hợp với các custom hooks để chia sẻ logic và dữ liệu giữa các component. Biểu đồ các stores và hooks được mô tả trong Hình 4.8.



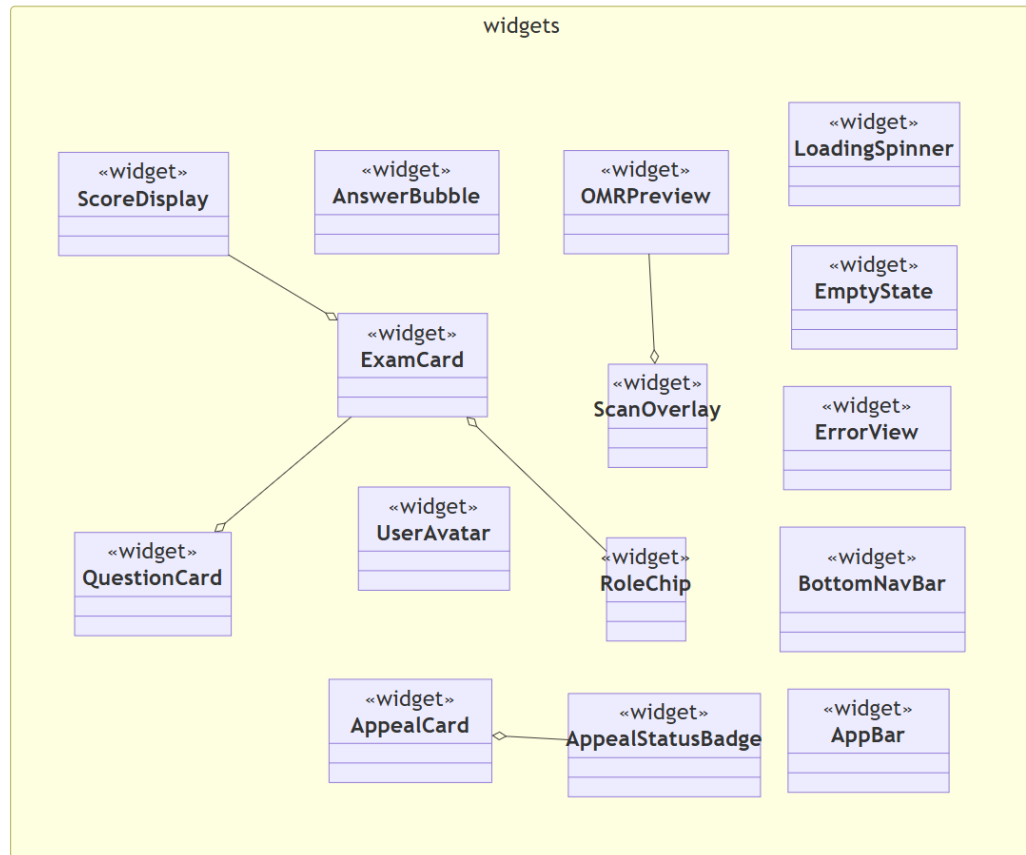
Hình 4.8: Biểu đồ gói Stores và Hooks của giao diện web React

Gói ApiService (Web React). ApiService cung cấp lớp giao tiếp REST API với backend, bao gồm các hàm tiện ích để thực hiện các yêu cầu HTTP như GET, POST, PUT, DELETE. Biểu đồ ApiService được mô tả trong Hình 4.9.

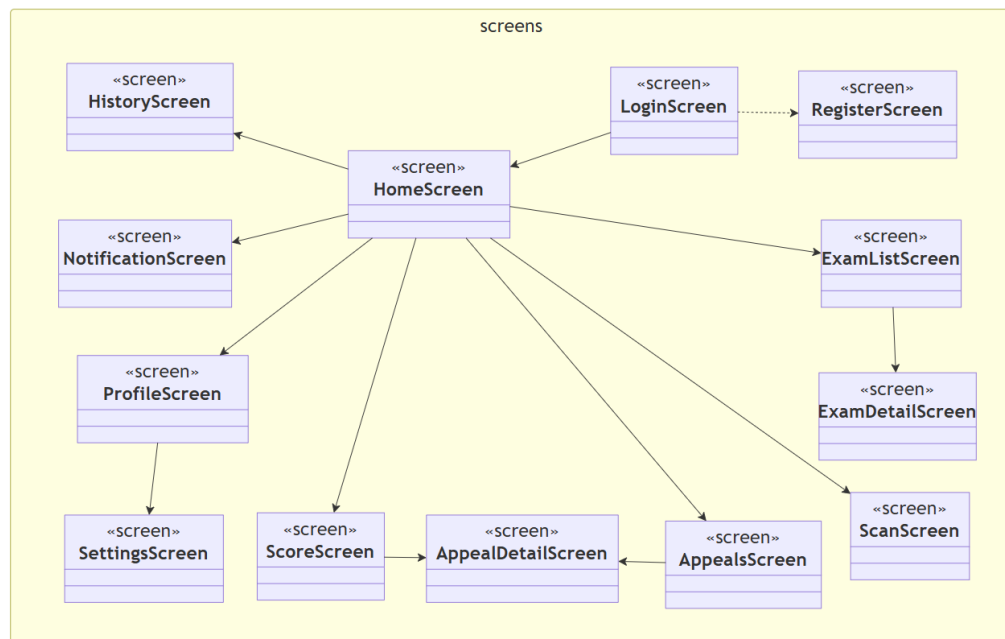


Hình 4.9: Biểu đồ gói ApiService của giao diện web React

Gói Screens và Widgets (Mobile Flutter). Ứng dụng di động Flutter sử dụng mô hình BLoC kết hợp Repository pattern, trong đó screens chứa các màn hình chính và widgets chứa các thành phần giao diện có thể tái sử dụng. Biểu đồ các screens và widgets được mô tả trong Hình 4.10 và Hình 4.11.



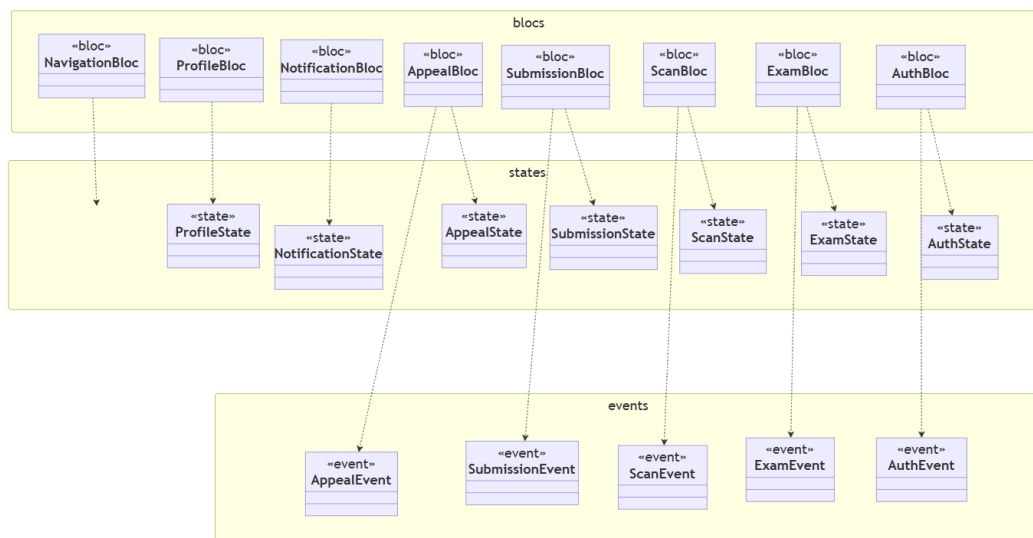
Hình 4.10: Biểu đồ gói Widgets của ứng dụng di động Flutter



Hình 4.11: Biểu đồ gói Screens của ứng dụng di động Flutter

Gói BLoCs (Mobile Flutter). BLoC (Business Logic Component) quản lý trạng

thái ứng dụng di động Flutter theo nguyên tắc phản ứng, mỗi BLoC xử lý một luồng sự kiện và trạng thái riêng biệt. Biểu đồ các BLoC được mô tả trong Hình 4.12.



Hình 4.12: Biểu đồ gói BLoCs của ứng dụng di động Flutter

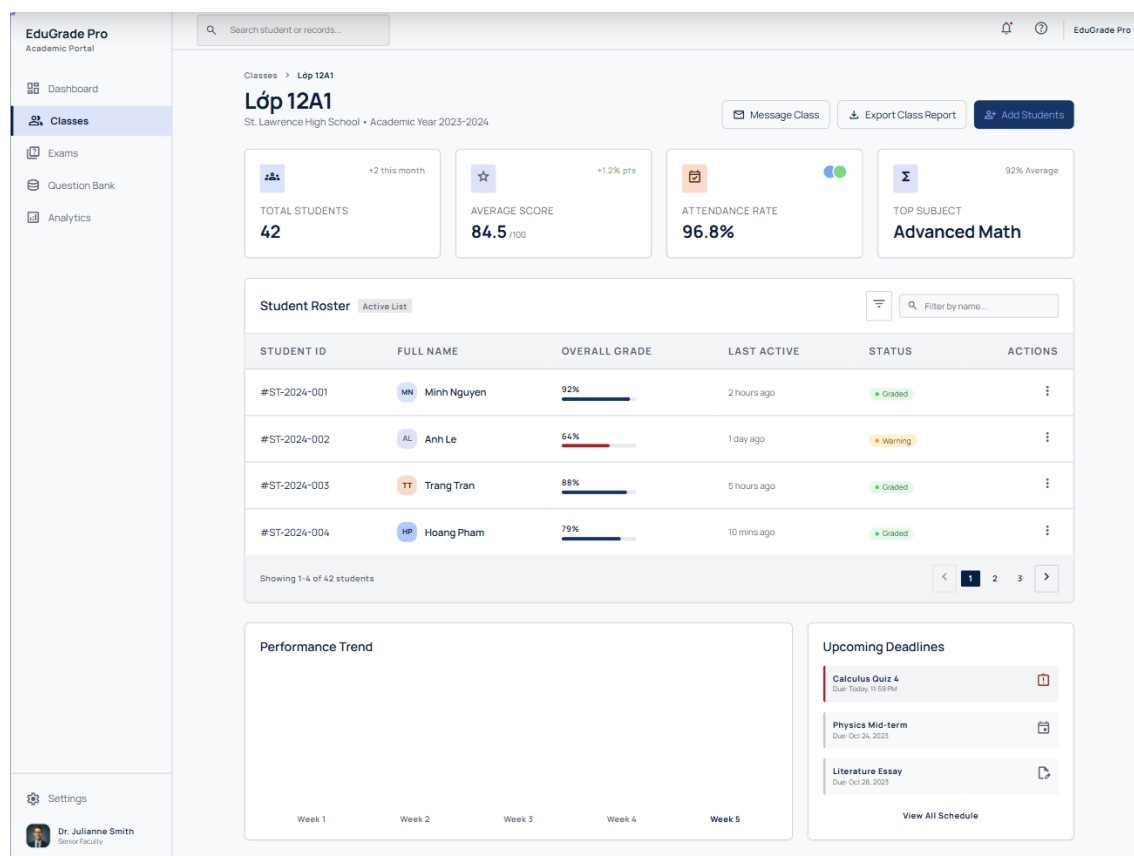
4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR hỗ trợ hai nhóm nền tảng giao diện với các chuẩn thiết kế thống nhất. Đối với ứng dụng di động Flutter, hệ thống hướng tới các thiết bị Android từ phiên bản 7.0 trở lên và iOS từ phiên bản 13 trở lên, kích thước màn hình tối thiểu 5.0 inch, độ phân giải tối thiểu HD (1280×720). Đối với giao diện web, hệ thống hỗ trợ các trình duyệt phổ biến (Chrome, Edge, Firefox, Safari) ở độ phân giải tối thiểu 1366×768 và tối ưu cho độ phân giải 1920×1080.

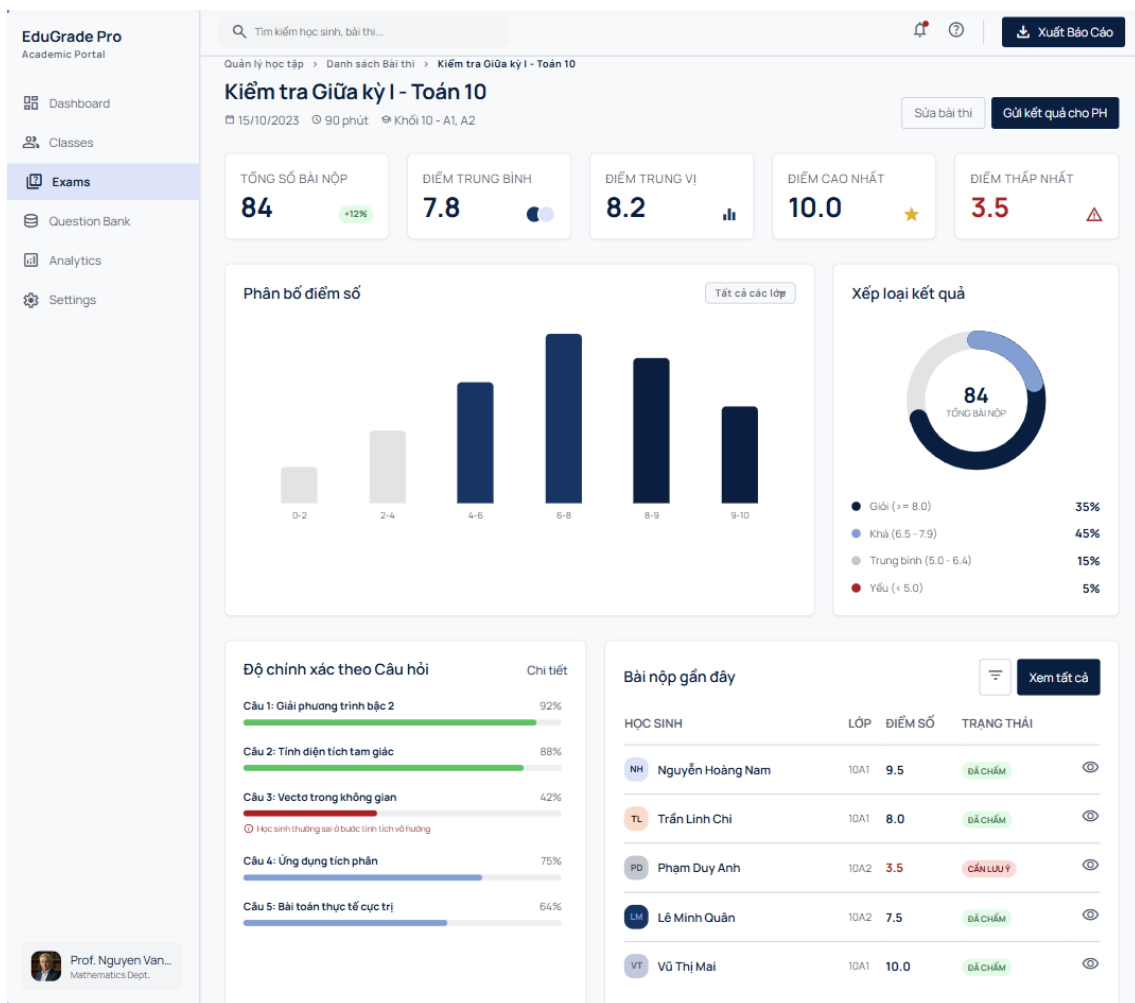
Chuẩn phối màu được sử dụng xuyên suốt hệ thống gồm: màu chủ đạo xanh đậm #0b2240 cho tiêu đề và thanh điều hướng; màu phụ xanh dương #2563eb cho các nút hành động chính; màu thành công xanh lá #059669 cho trạng thái tích cực; màu cảnh báo đỏ #dc2626 cho trạng thái lỗi; màu nền xám nhạt #f3f4f6 cho vùng nền. Phong chữ chính là phong không chân (sans-serif) để đảm bảo dễ đọc trên màn hình, với kích thước tiêu chuẩn 14pt cho nội dung, 16pt cho tiêu đề phụ và 20pt cho tiêu đề chính. Thông báo phản hồi được đặt ở góc trên bên phải màn hình với biểu tượng phù hợp, tự động biến mất sau 3 giây.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

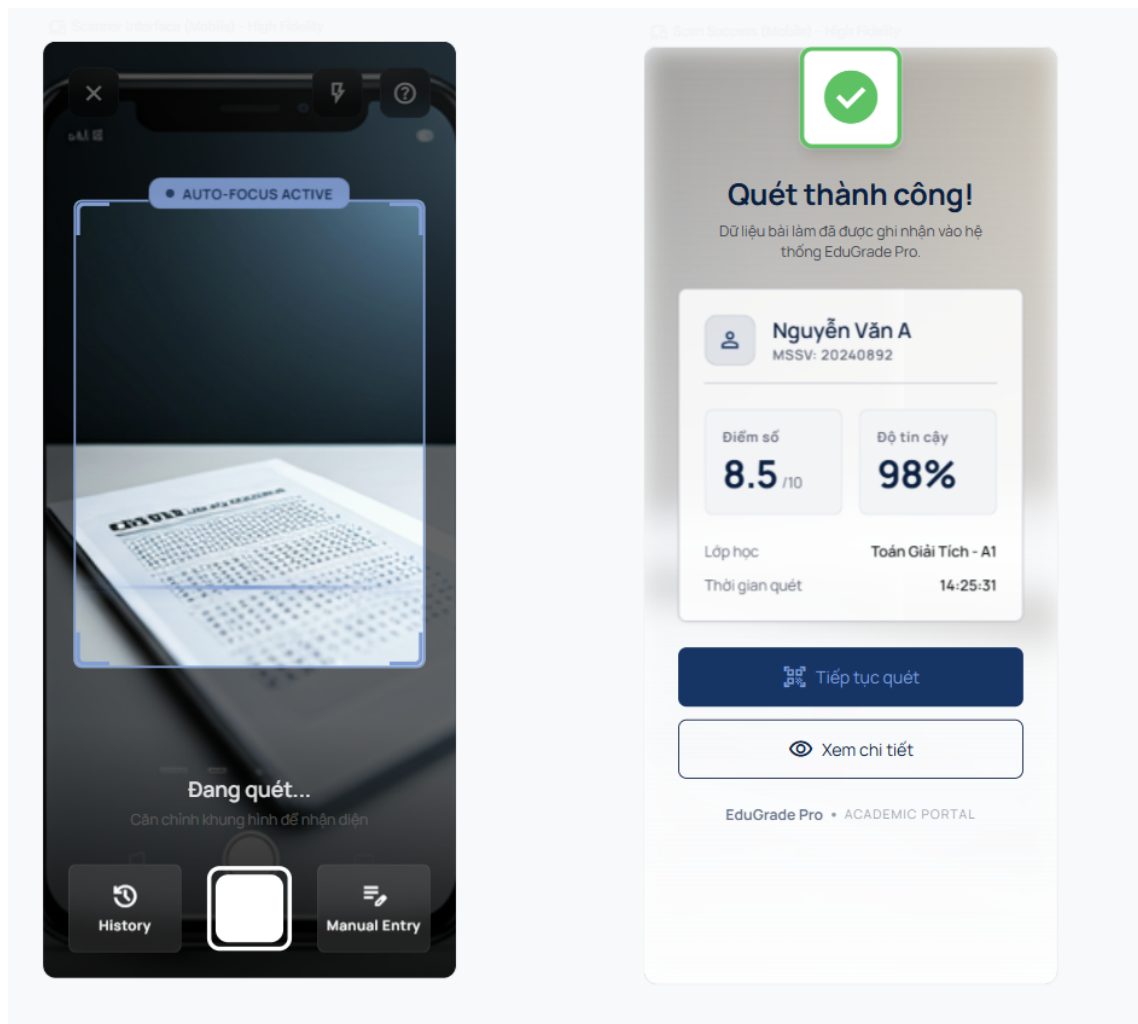


Hình 4.13: Minh họa màn hình chi tiết lớp học trên web

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

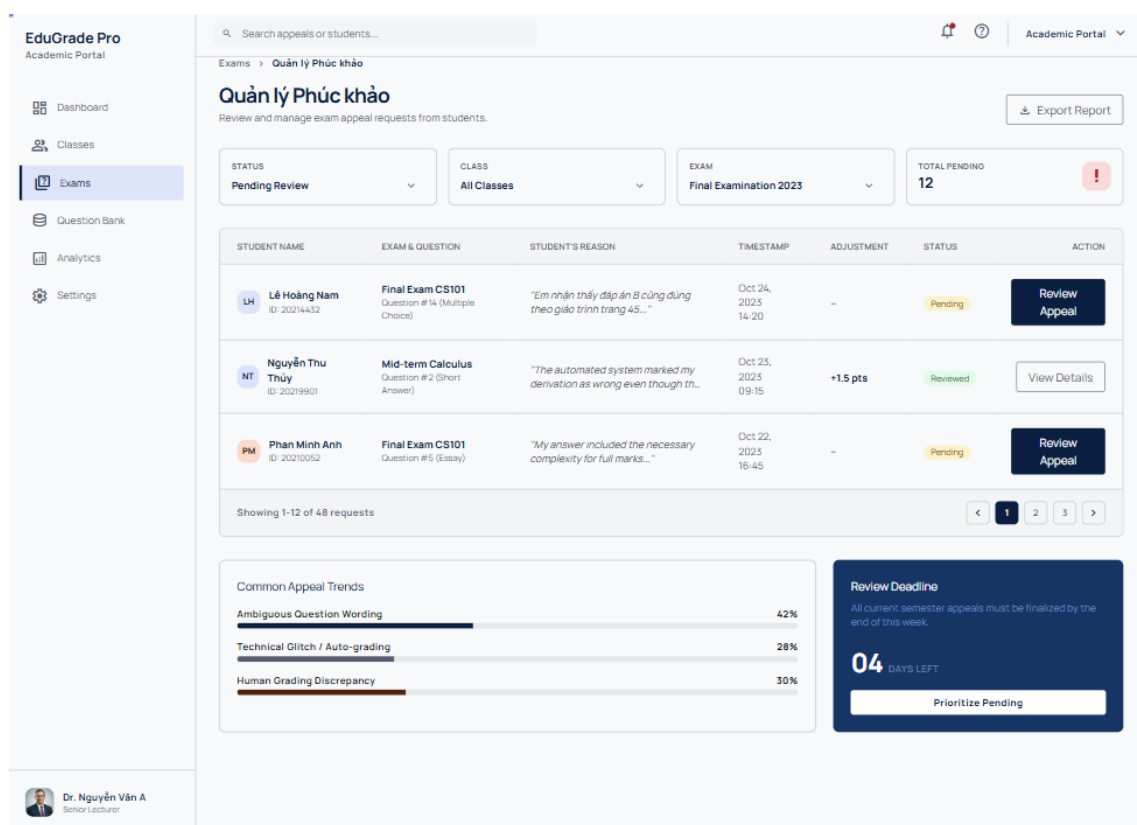


Hình 4.14: Minh họa màn hình chi tiết bài kiểm tra trên web



Hình 4.15: Minh họa màn hình quét phiếu trên di động

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.16: Minh họa màn hình phúc khảo trên web

4.2.2 Thiết kế chi tiết lớp

Phần này trình bày chi tiết ba lớp chủ đạo nhất của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, bao gồm đầy đủ thuộc tính và phương thức. Các lớp được lựa chọn dựa trên vai trò cốt lõi trong hệ thống: ExamService quản lý vòng đời đề thi, SubmissionService là lớp trung tâm của chức năng chấm thi OMR, và OMREngine xử lý nhận dạng OMR trên thiết bị di động.

a, Lớp SubmissionService (Backend – Node.js)

Vị trí. Gói services/, tầng Business Logic.

Mô tả. SubmissionService là lớp cốt lõi của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, đảm nhiệm toàn bộ logic xử lý bài nộp từ tiếp nhận kết quả quét OMR, chấm điểm tự động đến lưu trữ và thống kê kết quả thi.

Các thuộc tính. Bảng 4.1 mô tả chi tiết các thuộc tính của lớp.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thuộc tính	Kiểu	Mô tả
submissionModel	SubmissionModel	Tham chiếu đến model bài nộp trong MongoDB
examService	ExamService	Service quản lý đề thi, dùng để lấy đáp án
notificationService	NotificationService	Service gửi thông báo điểm số
userModel	UserModel	Model người dùng để tra cứu thông tin học sinh
logger	Logger	Ghi nhật ký hoạt động

Bảng 4.1: Các thuộc tính của lớp SubmissionService

Các phương thức. Bảng 4.2 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
create	req: CreateSubmissionDTO	Promise<Submission>	Tạo bài nộp mới từ dữ liệu thủ công
createFromMobileGraded	data: MobileGradedData	Promise<Submission>	Tạo bài nộp từ kết quả chấm trên mobile (AMC Mode)
createFromServerScanned	imageUrl, examId, studentCode	Promise<Submission>	Tạo bài nộp từ ảnh quét gửi lên server (Legacy Mode)
gradeSubmission	submissionId: string	Promise<Submission>	Chấm điểm bài nộp dựa trên đáp án
regradeSubmission	submissionId: string	Promise<Submission>	Chấm lại bài nộp sau khi cập nhật đáp án đề thi
updateScore	submissionId, score	Promise<Submission>	Cập nhật điểm thủ công
overrideAnswer	submissionId, questionNumber, newAnswer, reviewedBy	Promise<Submission>	Ghi đè đáp án khi phúc khảo được chấp nhận
findById	id: string	Promise<Submission>	Tìm bài nộp theo ID
findByExam	examId, filters	Promise<Submission[]>	Tìm tất cả bài nộp theo đề thi, có lọc
findByStudent	studentId, filters	Promise<Submission[]>	Tìm bài nộp theo học sinh
getStatistics	examId: string	Promise<ExamStatistics>	Lấy thống kê điểm của đề thi
bulkCreateFromScanned	examId, scannedData	Promise<Submission[]>	Tạo nhiều bài nộp từ danh sách dữ liệu quét
processImage	imageUrl, templateId	Promise<OMRResult>	Xử lý ảnh OMR trên server
_buildSubmissionFromOMR	omrResult, examId	Submission	Xây dựng object Submission từ kết quả OMR
_calculateScore	answers, answerKey	number	Tính điểm dựa trên đáp án
_validateStudentCode	studentCode, examId	boolean	Kiểm tra mã học sinh có hợp lệ

Bảng 4.2: Các phương thức của lớp SubmissionService

Mối quan hệ. SubmissionService có composition với SubmissionModel (tạo và lưu bài nộp), dependency với ExamService (lấy đáp án để chấm điểm), dependency với NotificationService (gửi thông báo kết quả), và sử dụng UserModel để tra cứu thông tin học sinh.

b, Lớp ExamService (Backend – Node.js)

Vị trí. Gói `services/`, tầng Business Logic.

Mô tả. ExamService quản lý toàn bộ vòng đời của đề thi từ giai đoạn tạo mới, cập nhật câu hỏi, xuất bản đề thi, đến việc sinh các phiên bản đề (với đáp án khác nhau) phục vụ chống gian lận thi cử.

Các thuộc tính. Bảng 4.3 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
examModel	ExamModel	Model đề thi trong MongoDB
examVersionModel	ExamVersionModel	Model phiên bản đề
questionModel	QuestionModel	Model câu hỏi trong ngân hàng đề
omrTemplateModel	OMRTemplateModel	Model template OMR
submissionService	SubmissionService	Service bài nộp để lấy thống kê
paperEngine	IPaperEngine	Engine tạo đề thi PDF (AMC)

Bảng 4.3: Các thuộc tính của lớp ExamService

Các phương thức. Bảng 4.4 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
create	userId, data	Promise<Exam>	Tạo đề thi mới với thông tin cơ bản
findById	id: string	Promise<Exam>	Tìm đề thi theo ID
findBySchool	schoolId, filters	Promise<Exam[]>	Tìm đề thi theo trường học
findByTeacher	teacherId: string	Promise<Exam[]>	Tìm đề thi do giáo viên tạo
update	id, data	Promise<Exam>	Cập nhật thông tin đề thi
delete	id: string	Promise<void>	Xóa đề thi và các phiên bản
publish	id: string	Promise<Exam>	Xuất bản đề thi, sinh các phiên bản
unpublish	id: string	Promise<Exam>	Hủy xuất bản, không cho phép nộp bài
duplicate	id, newTitle	Promise<Exam>	Tạo bản sao đề thi để tái sử dụng
generateVersions	examId: string	Promise<ExamVersion[]>	Sinh các phiên bản đề với đáp án khác nhau
getVersion	examId, versionCode	Promise<ExamVersion>	Lấy phiên bản đề cụ thể
getAnswerKey	examId, versionCode	Promise<AnswerKey>	Lấy đáp án của phiên bản
addQuestions	examId, questionIds	Promise<Exam>	Thêm câu hỏi vào đề thi
removeQuestions	examId, questionIds	Promise<Exam>	Xóa câu hỏi khỏi đề thi
shuffleQuestions	examId, config	Promise<Exam>	Cấu hình xáo trộn câu hỏi
getStatistics	id: string	Promise<ExamStatistics>	Lấy thống kê điểm và tỷ lệ
_generatePaperPDF	exam, version	Promise<string>	Sinh file PDF đề thi bằng AMC
_validateExamData	data	void	Kiểm tra dữ liệu đề thi hợp lệ

Bảng 4.4: Các phương thức của lớp ExamService

Mối quan hệ. ExamService có composition với ExamModel và ExamVersionModel (quản lý trực tiếp), dependency với QuestionModel và OMRTemplateModel (truy vấn khi sinh đề), dependency với SubmissionService (lấy dữ liệu thống kê), và dependency với AMCService (sinh đề thi PDF qua giao diện IPaperEngine).

c, Lớp OMREngine (Mobile – Flutter)

Vị trí. Gói engine/, tầng Business Logic trên ứng dụng di động.

Mô tả. OMREngine là lớp xử lý nhận dạng OMR trên thiết bị di động, thực hiện chấm thi ngay trên smartphone mà không cần gửi ảnh lên server. Lớp này sử dụng các thuật toán xử lý ảnh để phát hiện bong bóng đã tô và đọc đáp án.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Các thuộc tính. Bảng 4.5 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
imageProcessor	ImageProcessor	Xử lý ảnh: tăng tương phản, xóa nhiễu, chỉnh nghiêng
bubbleDetector	BubbleDetector	Phát hiện bong bóng đã tô
answerReader	AnswerReader	Đọc mã sinh viên, mã phiên bản, đáp án
template	OMRTemplate?	Template OMR hiện tại đang sử dụng
answerKey	Map<int, String>?	Đáp án đúng để so sánh khi chấm
isInitialized	boolean	Trạng thái đã khởi tạo với template

Bảng 4.5: Các thuộc tính của lớp OMREngine

Các phương thức. Bảng 4.6 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
initialize	template: OMRTemplate	void	Khởi tạo engine với template OMR tương ứng
setAnswerKey	answerKey: Map<int, String>	void	Đặt đáp án đúng để chấm điểm
processImage	image: Uint8List	OMRResult	Xử lý ảnh đầy đủ: tiền xử lý → phát hiện → đọc → chấm
calibrate	image: Uint8List	CalibrationResult	Hiệu chỉnh góc nghiêng, độ sáng
detectBubbles	image: Uint8List	List<Bubble>	Phát hiện tất cả bong bóng trong ảnh
readAnswers	bubbles: List<Bubble>	Map<int, String>	Đọc đáp án từ danh sách bong bóng
gradeAnswers	userAnswers, correctAnswers	GradeResult	So sánh và chấm điểm
preprocessImage	image: Uint8List	Uint8List	Tiền xử lý ảnh: chuẩn hóa, xóa nhiễu
enhanceContrast	image: Uint8List	Uint8List	Tăng độ tương phản để dễ nhận diện
detectEdges	image: Uint8List	Uint8List	Phát hiện cạnh để tìm vùng bong bóng
findContours	image: Uint8List	List<Contour>	Tìm đường viền của các đối tượng trong ảnh
validateTemplate	image: Uint8List	boolean	Kiểm tra ảnh có đúng template OMR
_normalizeImage	image: Uint8List	Uint8List	Chuẩn hóa kích thước, màu sắc
_removeNoise	image: Uint8List	Uint8List	Lọc nhiễu khỏi ảnh
_detectStudentCode	bubbles: List<Bubble>	String	Đọc mã học sinh từ vùng mã trên phiếu
_detectVersionCode	bubbles: List<Bubble>	String	Đọc mã phiên bản đề thi

Bảng 4.6: Các phương thức của lớp OMREngine

Mối quan hệ. OMREngine có composition với ImageProcessor, BubbleDetector và AnswerReader (sử dụng ba lớp phụ trợ này trong pipeline xử lý), uses với OMRTemplate (template định nghĩa cấu trúc phiếu), produces OMRResult (kết quả nhận dạng gồm mã học sinh, mã đề, đáp án, độ tin cậy). BubbleDetector tạo ra các đối tượng Bubble, và AnswerReader sử dụng Bubble để đọc đáp án.

d, Tổng hợp mối quan hệ giữa các lớp

Biểu đồ tổng hợp dưới đây thể hiện mối quan hệ giữa ba lớp chủ đạo và các lớp phụ trợ trong hệ thống.

Lớp SubmissionService (Backend – Node.js). Lớp này phụ thuộc vào ExamService để lấy đáp án khi chấm điểm, phụ thuộc vào NotificationService để gửi thông báo kết quả, và lưu trữ dữ liệu thông qua SubmissionModel. ExamService có composition với ExamModel (quản lý đề thi) và ExamVersionModel (sinh phiên bản đề).

Lớp OMREngine (Mobile – Flutter). Lớp này sử dụng ba lớp phụ trợ: ImageProcessor để tiền xử lý ảnh đầu vào (chuẩn hóa, tăng tương phản, nhị phân hóa, chỉnh nghiêng), BubbleDetector để phát hiện các bong bóng đã tô trên phiếu (tìm đường viền, lọc theo diện tích, nhóm theo hàng), và AnswerReader để đọc mã học sinh, mã phiên bản đề và đáp án từ các bong bóng.

Tương tác giữa Backend và Mobile. Khi giáo viên chụp ảnh phiếu trên ứng dụng di động, OMREngine xử lý ảnh cục bộ và trả kết quả về ứng dụng. Ứng dụng gửi kết quả này lên backend thông qua SubmissionService. SubmissionService kiểm tra mã học sinh, tính điểm, lưu vào cơ sở dữ liệu và gửi thông báo cho học sinh.

Bảng tổng hợp các lớp. Bảng 4.7 tổng hợp thông tin về ba lớp chủ đạo và các lớp phụ trợ.

Lớp	Nền tảng	Gói	Số phương thức	Chức năng chính
SubmissionService	Backend (Node.js)	services	15	Tiếp nhận, chấm điểm, lưu bài nộp
ExamService	Backend (Node.js)	services	17	Quản lý vòng đời đề thi
OMREngine	Mobile (Flutter)	engine	14	Xử lý ảnh OMR, chấm thi trên thiết bị
ImageProcessor	Mobile (Flutter)	engine	7	Tiền xử lý ảnh đầu vào
BubbleDetector	Mobile (Flutter)	engine	6	Phát hiện bong bóng đã tô
AnswerReader	Mobile (Flutter)	engine	5	Đọc mã và đáp án từ bong bóng

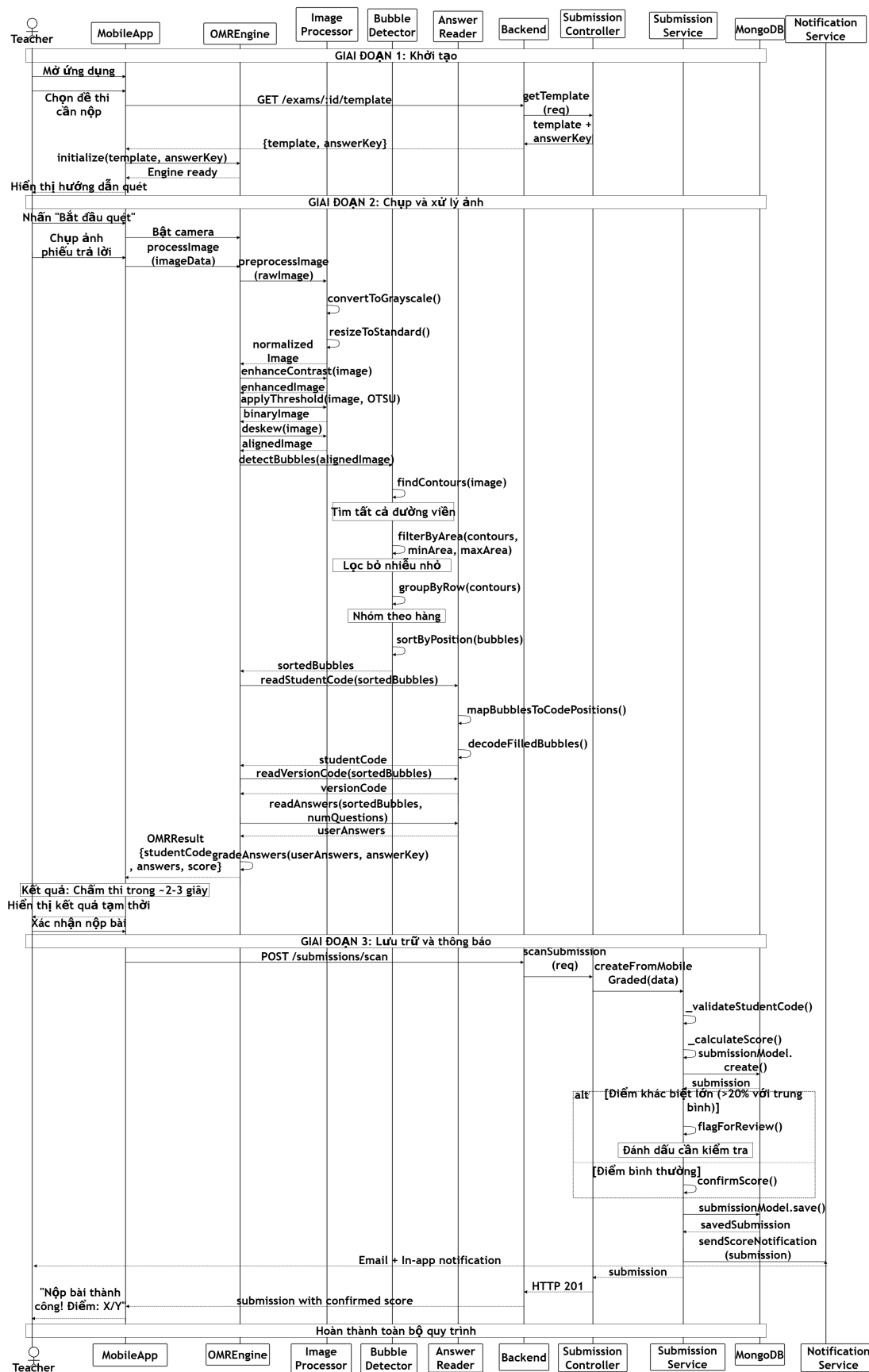
Bảng 4.7: Tổng hợp các lớp chủ đạo của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Luồng truyền thông điệp – Chấm thi OMR. Quy trình chấm thi OMR tự động gồm ba giai đoạn chính. *Giai đoạn 1 (Khởi tạo):* giáo viên mở ứng dụng,

chọn đề thi; ứng dụng gọi API lấy template OMR và đáp án từ server; OMREngine được khởi tạo với template. *Giai đoạn 2 (Chụp và xử lý ảnh)*: giáo viên chụp ảnh phiếu trả lời; ImageProcessor thực hiện chuyển ảnh xám, chuẩn hóa kích thước, tăng tương phản, nhị phân hóa Otsu và chỉnh nghiêng; BubbleDetector tìm đường viền, lọc theo diện tích, nhóm theo hàng và sắp xếp; AnswerReader đọc mã học sinh, mã phiên bản và đáp án; OMREngine so sánh với đáp án và tính điểm; kết quả được hiển thị tạm thời trên ứng dụng (thời gian xử lý khoảng 2–3 giây). *Giai đoạn 3 (Lưu trữ và thông báo)*: ứng dụng gửi kết quả lên server; SubmissionService kiểm tra mã học sinh, tính điểm, lưu vào MongoDB; nếu điểm khác biệt lớn so với trung bình, bài nộp được đánh dấu cần kiểm tra; NotificationService gửi thông báo kết quả cho học sinh qua email và in-app notification.

Hình 4.17 mô tả biểu đồ trình tự của use case Chấm thi bằng điện thoại, thể hiện chi tiết các bước từ khi giáo viên chọn đề thi, khởi tạo engine OMR, chụp và xử lý ảnh phiếu trả lời, nhận diện bong bóng, chấm điểm tự động cho đến khi lưu kết quả và gửi thông báo cho học sinh.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

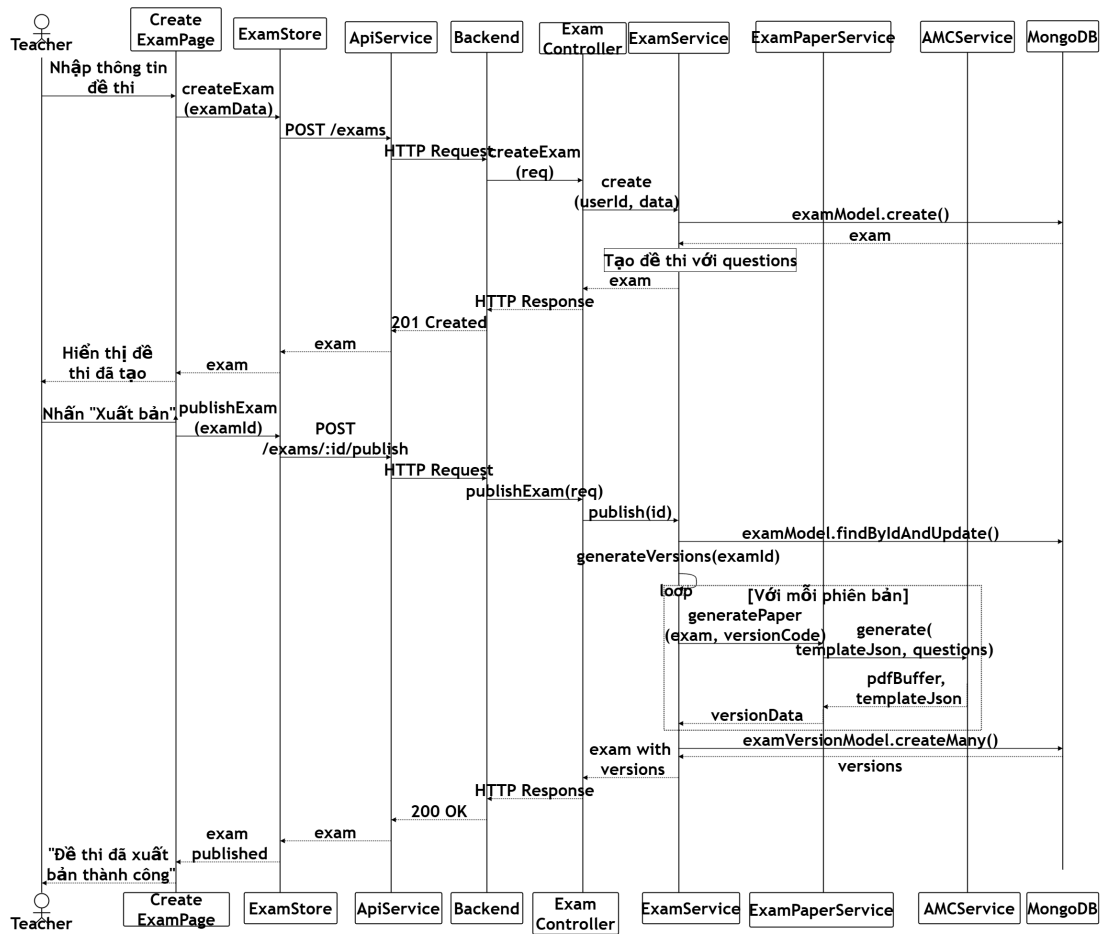


Hình 4.17: Biểu đồ trình tự use case Chấm thi bằng điện thoại

Hình 4.18 mô tả biểu đồ trình tự của use case Quản lý đề thi, thể hiện chi tiết các bước từ khi giáo viên tạo đề thi mới, thiết lập câu hỏi và đáp án, xuất bản đề thi,

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

sinh các phiên bản đề với đáp án khác nhau phục vụ chống gian lận, đến khi phát hành đề thi cho học sinh.

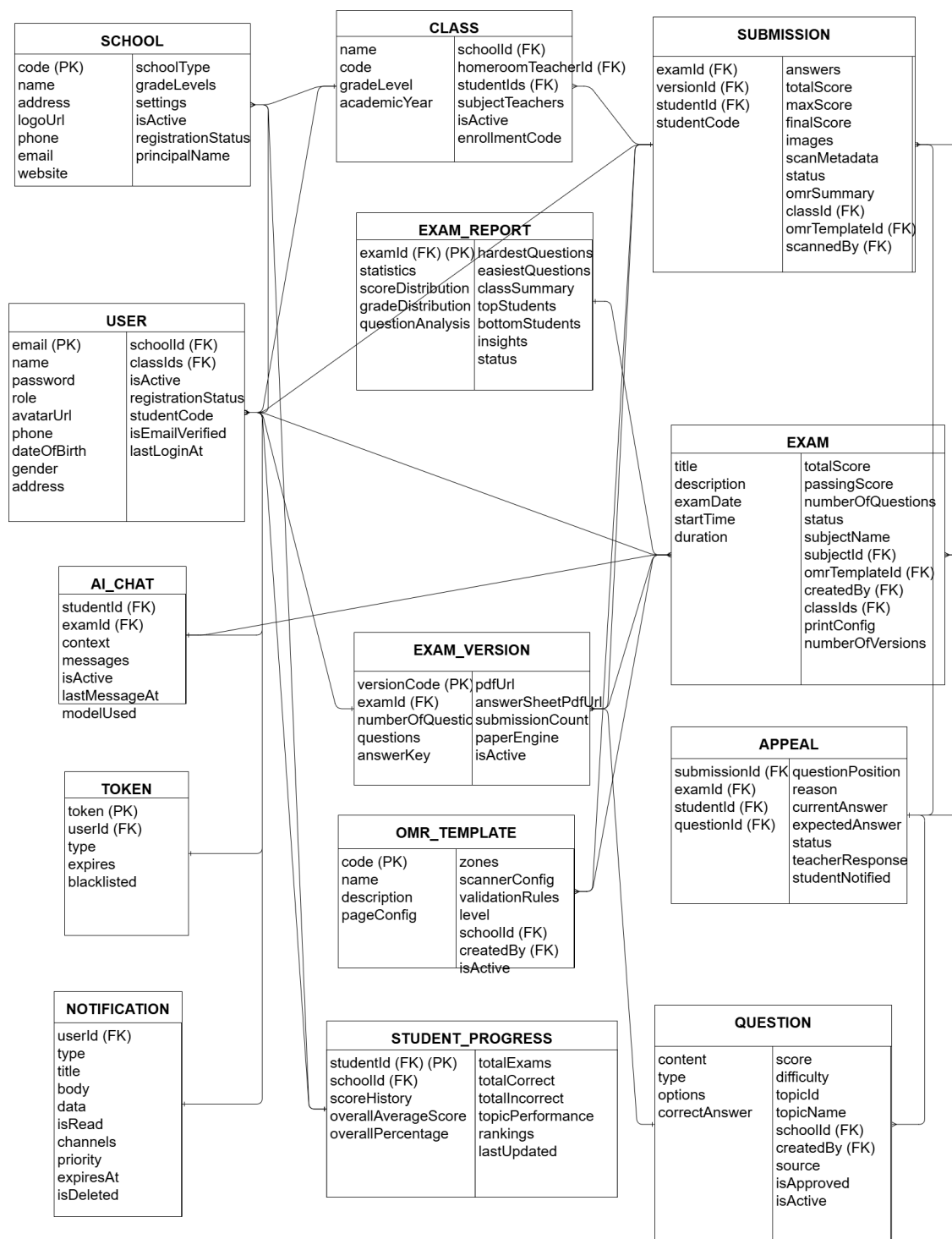


Hình 4.18: Biểu đồ trình tự use case Quản lý đề thi

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu MongoDB phiên bản 5.0 trở lên với 17 collection chính. Sơ đồ quan hệ thực thể (ER) của hệ thống được thể hiện trong Hình 4.19.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.19: Sơ đồ quan hệ thực thể (ER) của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Dưới đây là chi tiết cấu trúc của các collection cốt lõi trong hệ thống.

Bảng 4.2 — Collection users.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
name	String	Họ và tên người dùng
email	String (unique)	Địa chỉ email, duy nhất
password	String	Mật khẩu đã băm (bcrypt)
role	Enum	Vai trò: admin, school-admin, teacher, student, parent
schoolId	ObjectId	Tham chiếu đến collection schools
classIds	Array ObjectId	Danh sách các lớp mà người dùng thuộc về
studentCode	String	Mã số sinh viên / mã giáo viên
isEmailVerified	Boolean	Trạng thái xác thực email
avatar	String	URL ảnh đại diện
createdAt	Date	Thời điểm tạo tài khoản
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.3 — Collection schools.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
name	String	Tên trường học
address	String	Địa chỉ trường
phone	String	Số điện thoại liên hệ
email	String	Email trường
logo	String	URL logo trường
isActive	Boolean	Trạng thái hoạt động của trường
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.4 — Collection classes.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
name	String	Tên lớp (VD: 10A1)
schoolId	ObjectId	Tham chiếu đến collection schools
gradeLevel	Number	Khối lớp (10, 11, 12)
teacherIds	Array ObjectId	Danh sách giáo viên chủ nhiệm/phụ trách
studentIds	Array ObjectId	Danh sách học sinh trong lớp
academicYear	String	Năm học (VD: 2025–2026)
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.5 — Collection subjects.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
name	String	Tên môn học (VD: Toán, Văn, Anh)
code	String	Mã môn học, duy nhất
schoolId	ObjectId	Tham chiếu đến collection schools
description	String	Mô tả môn học
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.6 — Collection questions.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
subjectId	ObjectId	Tham chiếu đến collection subjects
schoolId	ObjectId	Tham chiếu đến collection schools
content	String	Nội dung câu hỏi
type	Enum	Loại: multiple-choice, true-false, fill-blank
options	Array Object	Danh sách các lựa chọn (label, content, isCorrect)
correctAnswer	String	Đáp án đúng
score	Number	Điểm số của câu hỏi
difficulty	Enum	Mức độ: easy, medium, hard
tags	Array String	Các nhãn phân loại câu hỏi
image	String	URL ảnh đính kèm (nếu có)
isActive	Boolean	Trạng thái sử dụng
createdBy	ObjectId	Giáo viên tạo câu hỏi
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.7 — Collection exams.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
title	String	Tiêu đề đề thi
description	String	Mô tả nội dung đề thi
subjectId	ObjectId	Tham chiếu đến collection subjects
classIds	Array ObjectId	Danh sách lớp áp dụng
primaryClassId	ObjectId	Lớp chính của đề thi
omrTemplateId	ObjectId	Tham chiếu đến mẫu OMR
questionIds	Array ObjectId	Danh sách câu hỏi trong đề
examDate	Date	Ngày thi
duration	Number	Thời gian làm bài (phút)
totalScore	Number	Tổng điểm đề thi
numberOfVersions	Number	Số phiên bản đề (để trộn)
status	Enum	Trạng thái: draft, scheduled, published, completed
shuffleConfig	Object	Cấu hình trộn (shuffleQuestions, shuffleOptions)
paperEngine	Enum	Công cụ sinh đề: amc hoặc pdftk
createdBy	ObjectId	Giáo viên tạo đề thi
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.8 — Collection examVersions.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
examId	ObjectId	Tham chiếu đến collection exams
versionNumber	Number	Số phiên bản (1, 2, 3...)
questionOrder	Array ObjectId	Thứ tự câu hỏi đã trộn của phiên bản này
optionOrders	Object	Thứ tự đáp án đã trộn cho từng câu
answerKey	Array Object	Đáp án đúng theo thứ tự câu hỏi của phiên bản
paperUrl	String	URL file PDF đề thi đã sinh
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.9 — Collection omrTemplates.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
name	String	Tên mẫu OMR (VD: Mẫu A4 50 câu)
schoolId	ObjectId	Tham chiếu đến collection schools
fields	Array Object	Danh sách các vùng nhận dạng trên phiếu
markers	Object	Tọa độ các dấu thập phân để căn chỉnh
questionConfig	Object	Cấu hình số câu, số cột, kích thước ô
templateImage	String	URL ảnh mẫu OMR gốc
isDefault	Boolean	Là mẫu mặc định của trường
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.10 — Collection submissions.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
examId	ObjectId	Tham chiếu đến collection exams
versionId	ObjectId	Tham chiếu đến phiên bản đề học sinh nhận được
studentId	ObjectId	Tham chiếu đến collection users (học sinh)
answers	Array Object	Danh sách câu trả lời (questionId, selectedAnswer, isCorrect, score)
images	Array String	Danh sách URL ảnh phiếu đã quét
scanMetadata	Object	Thông tin metadata quét (device, timestamp, confidence)
omrSummary	Object	Tóm tắt kết quả nhận dạng OMR
status	Enum	Trạng thái: scanning, scored, reviewed, published
manualOverrides	Array Object	Các chỉnh sửa thủ công của giáo viên
totalScore	Number	Tổng điểm sau chấm
scoredAt	Date	Thời điểm chấm xong
publishedAt	Date	Thời điểm công bố điểm
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.11 — Collection appeals.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
submissionId	ObjectId	Tham chiếu đến collection submissions
examId	ObjectId	Tham chiếu đến collection exams
studentId	ObjectId	Tham chiếu đến collection users (học sinh)
questionId	ObjectId	Câu hỏi được phúc khảo
reason	String	Lý do nộp đơn phúc khảo
expectedAnswer	String	Đáp án mà học sinh mong đợi
evidenceImages	Array String	Danh sách URL ảnh minh chứng
status	Enum	Trạng thái: pending, approved, rejected
teacherResponse	String	Phản hồi của giáo viên
processedBy	ObjectId	Giáo viên xử lý đơn
processedAt	Date	Thời điểm xử lý
createdAt	Date	Thời điểm tạo đơn
updatedAt	Date	Thời điểm cập nhật gần nhất

Bảng 4.12 — Collection notifications.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
userId	ObjectId	Người nhận thông báo
type	Enum	Loại: exam-published, score-published, appeal-update, system
title	String	Tiêu đề thông báo
content	String	Nội dung thông báo
data	Object	Dữ liệu bổ sung (examId, submissionId...)
isRead	Boolean	Trạng thái đã đọc
readAt	Date	Thời điểm đánh dấu đã đọc
createdAt	Date	Thời điểm tạo

Bảng 4.13 — Collection aiReports.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính tự động tạo
examId	ObjectId	Tham chiếu đến collection exams
reportData	Object	Dữ liệu phân tích báo cáo từ AI
generatedBy	ObjectId	Giáo viên yêu cầu tạo báo cáo
createdAt	Date	Thời điểm tạo
updatedAt	Date	Thời điểm cập nhật gần nhất

Mối quan hệ giữa các bảng được thiết kế theo nguyên tắc tham chiếu thông qua ObjectId, giúp dễ dàng cập nhật và mở rộng. Các chỉ mục (index) được tạo trên các trường thường xuyên truy vấn như email trong users, examId trong submissions, schoolId trong classes, nhằm tối ưu hiệu năng truy vấn.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Danh sách các công cụ, ngôn ngữ, thư viện chính được sử dụng trong đồ án được tổng hợp trong Bảng 4.8.

Thành phần	Công cụ / Thư viện	Phiên bản	Mục đích sử dụng
Runtime	Node.js	18.x+	Môi trường chạy backend
Framework backend	Express	4.17.1	Xây dựng REST API
Cơ sở dữ liệu	MongoDB	5.0+	Lưu trữ dữ liệu
ODM	Mongoose	5.7.7	Ảnh xạ đối tượng-tài liệu (MongoDB)
Xác thực	Passport.js + JWT	0.4 / 9.0	Xác thực và phân quyền người dùng
Kiểm tra dữ liệu	Joi	17.3.0	Kiểm tra đầu vào API
Upload file	Cloudinary SDK	2.10	Lưu trữ ảnh và file PDF trên đám mây
AI	@google/generative-ai	0.24.1	Tích hợp mô hình Gemini vào hệ thống
Email	Nodemailer	6.3.1	Gửi email thông báo đến người dùng
Kiểm thử backend	Jest	26.x	Kiểm thử tự động phía backend
Frontend	React	18.x	Xây dựng giao diện web SPA
Ngôn ngữ web	TypeScript	5.x	Kiểu tĩnh cho JavaScript
Build tool	Vite	5.x	Công cụ đóng gói và phát triển web
State management	Zustand	5.x	Quản lý trạng thái ứng dụng web
Di động	Flutter	3.x	Phát triển ứng dụng đa nền tảng
State di động	flutter_bloc	9.1.1	Quản lý trạng thái theo mô hình BLoC
HTTP client	Dio	5.8.0	Giao tiếp REST API từ di động
Camera	camera (Flutter)	0.11.1	Truy cập camera thiết bị
Xử lý ảnh OMR	opencv_dart	2.2.1	Nhận dạng ký tự quang học trên phiếu trả lời
IDE	Visual Studio Code	Mới nhất	Môi trường phát triển tích hợp
Sinh đề thi	AMC (Auto-Multiple-Choice)	Mới nhất	Sinh đề thi và mẫu phiếu OMR
Biên dịch LaTeX	XeLaTeX (TeX Live 2023)	2023	Biên dịch tài liệu đồ án

Bảng 4.8: Danh sách thư viện và công cụ sử dụng trong đồ án

4.3.2 Kết quả đạt được

Sau quá trình phát triển, hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR đã được xây dựng hoàn chỉnh gồm các sản phẩm đóng gói:

- backend API triển khai được trên Node.js, có thể chạy trực tiếp hoặc đóng gói Docker;

- giao diện web dạng SPA (Single Page Application) có thể build ra các file tĩnh và triển khai trên bất kỳ web server nào;
- ứng dụng di động Flutter có thể build ra file APK cho Android và IPA cho iOS.

Thông kê tổng quan về mã nguồn của hệ thống được tổng hợp trong Bảng 4.9.

Thành phần	Số file mã nguồn	Số dòng mã (ước tính)	Ghi chú
Backend (Node.js)	190	17.000+	Gồm services, controllers, models
Mobile (Flutter)	199	28.000+	Gồm screens, blocs, repositories
Web (React)	151	21.000+	Gồm pages, components, stores
Tổng cộng	540+	66.000+	Toàn bộ dự án

Bảng 4.9: Thông kê mã nguồn của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR

Các lớp chính trong hệ thống gồm: 17 lớp model Mongoose tương ứng với 17 bảng dữ liệu; 16 service xử lý nghiệp vụ; 8 BLoC cho ứng dụng di động; 15 store Zustand cho giao diện web. Tổng dung lượng mã nguồn khoảng 4,5 MB, dung lượng ứng dụng di động sau khi build khoảng 25 MB cho Android.

4.3.3 Minh họa các chức năng chính

Màn hình đăng nhập. Người dùng truy cập hệ thống và đăng nhập bằng email và mật khẩu. Hệ thống hỗ trợ đăng nhập trên cả web và di động với giao diện thống nhất. Sau khi đăng nhập thành công, hệ thống chuyển hướng người dùng đến dashboard tương ứng với vai trò.

Màn hình dashboard. Dashboard hiển thị các chỉ số tổng quan phù hợp với vai trò người dùng: tổng số lớp, học sinh, đề thi, bài nộp, điểm trung bình, tỉ lệ đạt, danh sách bài nộp gần đây và số đơn phúc khảo đang chờ xử lý.

Màn hình quản lý câu hỏi và đề thi. Giáo viên có thể tạo câu hỏi thủ công hoặc sử dụng AI để sinh câu hỏi từ chủ đề. Màn hình tạo đề thi cho phép chọn câu hỏi từ ngân hàng, cấu hình tham số trộn và sinh nhiều phiên bản đề tự động.

Màn hình quét phiếu trên di động. Giáo viên sử dụng camera của thiết bị để chụp phiếu trả lời. Hệ thống tự động nhận dạng các ô tô và hiển thị kết quả chấm ngay trên ứng dụng với khả năng chỉnh sửa thủ công.

Màn hình quản lý phúc khảo. Giáo viên xem danh sách đơn phúc khảo, mở chi tiết để so sánh với đáp án và phản hồi cho học sinh. Học sinh theo dõi trạng thái đơn phúc khảo của mình trong mục "Phúc khảo của tôi".

Màn hình báo cáo thống kê. Hệ thống hiển thị các biểu đồ phổ điểm, tỉ lệ đạt theo lớp, độ khó và độ phân biệt của từng câu hỏi. Giáo viên có thể xuất báo cáo ra file PDF hoặc Excel để sử dụng ngoài hệ thống.

4.4 Kiểm thử

Hệ thống được kiểm thử theo hai kỹ thuật chính là kiểm thử hộp đen (black-box testing) và kiểm thử hộp trắng (white-box testing). Kiểm thử hộp trắng được áp dụng cho backend với framework Jest, bao gồm kiểm thử đơn vị (unit test) cho từng service và controller, kiểm thử tích hợp (integration test) cho luồng API end-to-end. Kiểm thử hộp đen được áp dụng cho cả web và di động thông qua kiểm thử thủ công và kiểm thử giao diện tự động với React Testing Library và Flutter Test.

Đồ án tập trung thiết kế các ca kiểm thử cho ba chức năng chính gồm:

- Tạo đề thi và trộn câu cho nhiều đề thi;
- Chấm thi bằng ứng dụng;
- Phản hồi phúc khảo.

Tổng số ca kiểm thử được thiết kế cho ba chức năng này là 45 ca, trong đó có 18 ca cho chức năng tạo đề thi, 15 ca cho chức năng chấm thi và 12 ca cho chức năng phúc khảo. Kết quả kiểm thử cho thấy 42/45 ca đạt kết quả như mong đợi, 3 ca không đạt đã được phân tích và sửa lỗi. Bảng 4.10 tổng hợp kết quả kiểm thử tổng quan.

Chức năng	Số ca kiểm thử	Số ca đạt	Tỉ lệ đạt
Tạo đề thi, trộn câu	18	17	94.4%
Chấm thi bằng ứng dụng	15	13	86.7%
Phản hồi phúc khảo	12	12	100%
Tổng cộng	45	42	93.3%

Bảng 4.10: Tổng hợp kết quả kiểm thử các chức năng chính

Ba ca kiểm thử không đạt tập trung ở chức năng chấm thi bằng ứng dụng, nguyên nhân chính là do ảnh chụp trong điều kiện ánh sáng yếu hoặc bị nghiêng quá nhiều. Sau khi điều chỉnh thuật toán tiền xử lý ảnh với bước tăng cường độ tương phản thích ứng và bước chỉnh nghiêng cải tiến, các ca này đã đạt trong lần kiểm thử lại.

4.5 Sản phẩm và hướng dẫn triển khai

Hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR đã hoàn thành xây dựng với đầy đủ các thành phần backend, giao diện web và ứng dụng di động. Phần này trình bày tổng quan sản phẩm đã xây dựng, cấu hình môi trường chạy thử nghiệm và hướng dẫn triển khai hệ thống lên môi trường thực tế.

4.5.1 Tổng quan sản phẩm

Sản phẩm Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR được đóng gói thành ba phần chính:

- **Backend API (Node.js):** Có thể chạy trực tiếp với Node.js hoặc đóng gói Docker. Backend cung cấp các API RESTful cho toàn bộ chức năng của hệ thống.
- **Giao diện web (React SPA):** Có thể build ra các file tĩnh và triển khai trên bất kỳ web server nào (nginx, Apache, Vercel, Netlify) hoặc chạy trực tiếp với Vite dev server.
- **Ứng dụng di động (Flutter):** Có thể build ra file APK cho nền tảng Android và IPA cho nền tảng iOS. File cài đặt có thể phân phối qua Google Play Store, Firebase App Distribution hoặc cài trực tiếp qua file APK.

4.5.2 Hướng dẫn triển khai

Để triển khai hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR lên môi trường thực tế, cần thực hiện các bước sau theo thứ tự.

Bước 1: Chuẩn bị môi trường. Cần chuẩn bị một máy chủ (hoặc cloud server) cài đặt Node.js phiên bản 18.x trở lên, MongoDB phiên bản 5.0 trở lên, và tài khoản Cloudinary để lưu trữ file. Nếu sử dụng Docker, chỉ cần cài đặt Docker Engine và Docker Compose.

Bước 2: Cấu hình biến môi trường. Tạo file `.env` trong thư mục backend với các thông số cần thiết: chuỗi kết nối MongoDB (`MONGODB_URI`), khóa bí mật JWT (`JWT_SECRET`), thông tin tài khoản Cloudinary (`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`), cổng chạy server (`PORT`), và địa chỉ client web (`CLIENT_URL`).

Bước 3: Triển khai backend. Chạy lệnh `npm install` để cài đặt các thư viện, sau đó chạy `npm start` để khởi động server. Với Docker, chạy `docker-compose up -d` để khởi động toàn bộ backend và MongoDB trong container.

Bước 4: Triển khai giao diện web. Chỉnh sửa file cấu hình API endpoint trong mã nguồn web trở đến địa chỉ backend đã triển khai, sau đó chạy `npm run build` để sinh file tĩnh. Copy toàn bộ thư mục `dist` lên web server và cấu hình web server phục vụ file tĩnh.

Bước 5: Build ứng dụng di động. Chạy `flutter build apk --release` để sinh file APK hoặc `flutter build ios --release` để sinh file IPA. File cài đặt có thể được phân phối qua kênh phù hợp với nhu cầu thực tế.

4.5.3 Kiểm thử trong môi trường cục bộ

Hệ thống đã được kiểm thử trong môi trường cục bộ (localhost) nhằm xác nhận tất cả các chức năng hoạt động đúng thiết kế. Quá trình kiểm thử bao gồm: chạy bộ unit test trên backend với Jest, kiểm tra toàn bộ luồng nghiệp vụ từ đăng nhập đến chấm điểm và phúc khảo, và thử nghiệm quét phiếu trên thiết bị di động Android với bộ phiếu mẫu. Kết quả cho thấy:

- thời gian chấm trung bình cho một phiếu trên ứng dụng di động là khoảng 2,3 giây;
- độ chính xác của thuật toán nhận dạng OMR đạt trên 95% với ảnh chụp đạt chất lượng tốt;
- thời gian phản hồi API trung bình trên localhost là dưới 100ms;
- giao diện web hoạt động ổn định trên các trình duyệt Chrome, Firefox, Edge và Safari.

Sản phẩm đã sẵn sàng để triển khai thực tế khi hoàn tất các thủ tục về hạ tầng và phê duyệt của đơn vị sử dụng.

Chương 4 đã trình bày phân tích thiết kế kiến trúc, giao diện, lớp, cơ sở dữ liệu, kết quả xây dựng và kiểm thử hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR. Chương 5 sẽ tập trung trình bày chi tiết các giải pháp kỹ thuật nổi bật và đóng góp chính của đồ án.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này tập trung trình bày chi tiết các giải pháp kỹ thuật nổi bật: sinh đề nhiều phiên bản kèm tọa độ OMR, nhận dạng OMR trên di động với phát hiện lỗi, và quản lý phiên làm việc và phân quyền.

5.1 Giải pháp tự động sinh đề thi nhiều phiên bản và phiếu OMR kèm tọa độ nhận dạng

5.1.1 Dẫn dắt bài toán

Trong Chương 2, mục 2.2.6, đề án đã xác định yêu cầu tạo bài kiểm tra với khả năng tự động trộn câu hỏi thành nhiều phiên bản để hạn chế gian lận và sinh phiếu trả lời OMR tương ứng. Đây là bài toán khó vì mỗi phiên bản đề phải có đáp án đúng khác nhau do thứ tự câu hỏi và đáp án bị hoán vị, đồng thời tọa độ các ô trên phiếu phải được biết chính xác để ứng dụng di động có thể nhận dạng. Mục 4.3 chỉ mô tả kết quả đạt được; chi tiết giải pháp được trình bày tại đây.

5.1.2 Giải pháp chi tiết

Pipeline đồng bộ gồm năm giai đoạn chính, mỗi giai đoạn được thiết kế để chạy độc lập và có thể tái chạy khi cần sinh lại phiên bản đề.

a, Giai đoạn 1 – Sinh phiên bản đề và hoán vị đáp án

Để hạn chế gian lận, mỗi học sinh trong cùng một lớp thi sẽ nhận được một phiên bản đề khác nhau. Thuật toán đảm bảo ba tính chất: (a) các phiên bản là hoán vị của nhau, (b) đáp án đúng của mỗi câu bị hoán vị theo thứ tự câu hỏi mới, (c) việc sinh là xác định nên có thể tái tạo kết quả.

Bước 1.1, lấy danh sách câu hỏi của đề thi gốc từ database. Mỗi câu hỏi bao gồm: nội dung, bốn lựa chọn (A, B, C, D), đáp án đúng, loại câu hỏi, độ khó.

Bước 1.2, áp dụng thuật toán Fisher–Yates shuffle để trộn thứ tự câu hỏi. Thuật toán duyệt mảng ngẫu nhiên và hoán đổi phần tử hiện tại với một phần tử ngẫu nhiên phía sau nó:

$$\text{for } i = n - 1 \text{ downto } 1 : \quad j \leftarrow \text{random}(0, i); \quad \text{swap}(A[i], A[j]) \quad (5.1)$$

Hạt giống (seed) của bộ sinh ngẫu nhiên được lưu lại để đảm bảo tính tái tạo.

Bước 1.3, với mỗi câu hỏi sau khi trộn, hoán vị thứ tự bốn lựa chọn (A, B, C, D) theo một hoán vị ngẫu nhiên khác nhau. Đáp án đúng được cập nhật tương ứng để phản ánh vị trí mới.

Bước 1.4, kết quả được lưu thành đối tượng `ExamVersion` gồm: `examId`, `versionIndex` (0 đến $N - 1$), `seed` (hạt giống), `questionOrder` (danh sách ID

câu theo thứ tự mới), `answerKey` (đáp án đúng dạng $\{\text{questionId: correctChoice}\}$), `shuffleMap` (ánh xạ vị trí câu gốc sang vị trí mới).

Pseudocode 1 tóm tắt quy trình sinh phiên bản đề.

Algorithm 1: Sinh phiên bản đề thi

Input: Đề thi gốc E , số phiên bản N

Output: Danh sách $\mathcal{V} = \{V_0, V_1, \dots, V_{N-1}\}$

$\mathcal{V} \leftarrow \emptyset$;

for $i \leftarrow 0$ **to** $N - 1$ **do**

$seed_i \leftarrow \text{generateSeed}(E.id, i)$;

$rng \leftarrow \text{createRNG}(seed_i)$;

$Q_{shuffled} \leftarrow \text{fisherYatesShuffle}(E.questions, rng)$;

$A_{shuffled} \leftarrow \emptyset$;

foreach $q \in Q_{shuffled}$ **do**

$\pi_q \leftarrow \text{randomPermutation}(4, rng)$;

$q.choices \leftarrow \text{applyPermutation}(q.choices, \pi_q)$;

$q.correctChoice \leftarrow \pi_q^{-1}(q.correctChoice)$;

$A_{shuffled}[q.id] \leftarrow q.correctChoice$;

end

$V_i \leftarrow \text{ExamVersion}(E.id, i, seed_i, Q_{shuffled}, A_{shuffled})$;

$\mathcal{V} \leftarrow \mathcal{V} \cup \{V_i\}$;

end

return $\mathcal{V}$;

b, Giai đoạn 2 – Sinh mã nguồn LaTeX cho đề và phiếu OMR

Mỗi phiên bản đề cần hai file PDF: (a) file đề thi cho học sinh làm bài, (b) file phiếu trả lời OMR để chấm thi tự động. Hệ thống sử dụng thư viện AMC (Auto Multiple Choice) làm backend sinh LaTeX nhờ khả năng tạo cả hai loại file từ cùng một cấu hình.

Bước 2.1, module `amcSourceGenerator` nhận đầu vào là `ExamVersion` và cấu hình template (số câu, số lựa chọn, tiêu đề, logo trường). Generator duyệt danh sách câu hỏi theo thứ tự đã trộn và sinh mã LaTeX cho đề thi:

- Môi trường `question` cho mỗi câu hỏi, bọc nội dung và bốn lựa chọn trong `choices`
- Macro `\explication` cho giải thích (nếu có)
- Lệnh `\multiplechoices` để đánh dấu đáp án đúng (không hiển thị cho học sinh)

Bước 2.2, module `amcAnswerSheetTexGenerator` sinh mã LaTeX cho phiếu

trả lời OMR bằng môi trường `answerbox` của AMC. Cấu trúc phiếu gồm:

- Vùng mã số sinh viên: n cột, mỗi cột 4 bubble tương ứng ký tự số
- Vùng mã đề: m cột, mỗi cột 4 bubble tương ứng ký tự chữ/số
- Vùng câu trả lời: ma trận $Q \times 4$ bubble, mỗi hàng một câu, mỗi cột một lựa chọn
- Khung viền bao quanh toàn bộ phiếu để phát hiện contour

Bước 2.3, mã LaTeX được ghi ra file `exam_<version_index>.tex` và `sheet_<version_index>.tex`. Mỗi file kèm theo header chứa metadata: mã đề, phiên bản, số câu, ngày tạo.

Pseudocode 2 mô tả quy trình sinh mã nguồn.

Algorithm 2: Sinh mã nguồn LaTeX

Input: Danh sách phiên bản $\mathcal{V}$, cấu hình template T

Output: Danh sách file LaTeX

`files` $\leftarrow \emptyset$;

foreach $V \in \mathcal{V}$ **do**

`examTex` $\leftarrow$ `amcExamHeader`(T) ;

foreach $q \in V.questionOrder$ **do**

`examTex` $\leftarrow$ `examTex` + `formatQuestion`($q, V.answerKey[q.id]$) ;

end

`examTex` $\leftarrow$ `examTex` + `amcExamFooter`() ;

`writeFile`(`exam_` + $V.versionIndex$ + `.tex`, `examTex`) ;

`sheetTex` $\leftarrow$ `amcSheetHeader`(T) ;

`sheetTex` $\leftarrow$ `sheetTex` + `formatStudentIdBox`(T) ;

`sheetTex` $\leftarrow$ `sheetTex` + `formatExamCodeBox`(T) ;

`sheetTex` $\leftarrow$ `sheetTex` + `formatAnswerGrid`($T, V.versionIndex$) ;

`sheetTex` $\leftarrow$ `sheetTex` + `amcSheetFooter`() ;

`writeFile`(`sheet_` + $V.versionIndex$ + `.tex`, `sheetTex`) ;

`files` $\leftarrow$

`files` $\cup \{(\text{exam_} + V.versionIndex, \text{sheet_} + V.versionIndex)\}$;

end

return `files` ;

c, Giai đoạn 3 – Trích xuất tọa độ các ô từ AMC

Tọa độ chính xác của các bubble là thành phần then chốt để ứng dụng di động có thể nhận dạng. Hệ thống tận dụng chế độ `prepare` của AMC để sinh file tọa độ tự động.

Bước 3.1, biên dịch file LaTeX phiếu OMR bằng XeLaTeX trong môi trường WSL2 để đảm bảo tương thích với các gói LaTeX của AMC:

$$\text{sheet_} < i > .\text{tex} \xrightarrow{\text{XeLaTeX}} \text{sheet_} < i > .\text{pdf} \quad (5.2)$$

Bước 3.2, gọi lệnh AMC với chế độ `prepare` để phân tích cấu trúc phiếu:

$$\text{auto-multiple-choice prepare --mode s --out-calage sheet_} < i > .\text{tex} \quad (5.3)$$

Chế độ `s` (separate) sinh file `sheet\_} < i > .calage.xy` chứa tọa độ pixel của tất cả các vùng được đánh dấu trong phiếu.

Bước 3.3, module `amcCalageParser` đọc file `.calage.xy` theo định dạng AMC. File có cấu trúc:

- Mỗi dòng tương ứng một vùng (box), có định dạng: `Box name x y width height`
- Các tọa độ được lưu ở đơn vị điểm (point) với độ phân giải nguồn

Bước 3.4, chuẩn hóa tọa độ về pixel ở 300 DPI:

$$\text{pixel} = \text{point} \times \frac{300}{72} = \text{point} \times 4,1667 \quad (5.4)$$

Sau làm tròn đến số nguyên gần nhất, sai số dưới 0,5 pixel.

Bước 3.5, phân loại các box thành các nhóm chức năng: `studentIdBoxes`, `examCodeBoxes`, `answerBoxes`. Mỗi nhóm được sắp xếp theo thứ tự hàng và cột để tạo ánh xạ bubble – câu hỏi – lựa chọn.

Kết quả giai đoạn này là danh sách tọa độ bubble chuẩn hóa, được lưu vào cấu trúc `OMRTemplate` để ứng dụng Flutter sử dụng khi chấm thi.

d, Giai đoạn 4 – Tạo template OMR và ánh xạ câu hỏi

Template OMR là cấu trúc trung tâm kết nối giữa backend (sinh đề) và frontend (nhận dạng OMR). Nó chứa tất cả thông tin không đổi của một dạng phiếu: vị trí các vùng đọc mã, cấu trúc lưới câu trả lời, và cách ánh xạ bubble sang câu hỏi.

Bước 4.1, từ kết quả giai đoạn 3, xây dựng `OMRTemplate` gồm các trường:

- `examId`: liên kết với đề thi gốc
- `versionIndex`: phiên bản đề (0 đến $N - 1$)
- `dpi`: độ phân giải chuẩn (300)
- `imageWidth`, `imageHeight`: kích thước ảnh chuẩn hóa
- `studentIdRegion`: tọa độ khung mã sinh viên, gồm danh sách `columns`

(mỗi cột là list bubble với $\{x, y, width, height\}$)

- `examCodeRegion`: tọa độ khung mã đề, cấu trúc tương tự
- `answerGrid`: danh sách hàng, mỗi hàng chứa `questionId`, `questionIndex`, danh sách bubbles (4 bubble tương ứng A, B, C, D)

Bước 4.2, xây dựng ánh xạ câu hỏi – phiên bản – đáp án. Vì thứ tự câu hỏi đã bị trộn, cần lưu ánh xạ để server biết đáp án đúng của câu thứ k trong phiếu trả lời là gì:

$$\text{answerKey}[k] = \text{correctChoice of question at position } k \text{ in version } V_i \quad (5.5)$$

Bước 4.3, template được lưu vào database dưới dạng JSON trong collection `OMRTemplates`, kèm chỉ mục đơn (index) trên trường $\{\text{examId}, \text{versionIndex}\}$ để truy vấn nhanh.

Bước 4.4, các file PDF (đề thi và phiếu OMR) được tải lên Cloudinary, lưu URL công khai. Backend lưu URL này trong `ExamVersion.pdfUrl` và `ExamVersion.sheetUrl` để giáo viên có thể in hoặc phát hành cho học sinh.

Pseudocode 3 mô tả quy trình tạo template OMR.

Algorithm 3: Xây dựng template OMR

Input: File `.calage.xy` $\mathcal{B}$, phiên bản đề V

Output: Đối tượng `OMRTemplate`

`boxes` $\leftarrow$ `parseCalage`($\mathcal{B}$) ;

`stdBoxes` $\leftarrow$ `normalizeTo300DPI`(`boxes`) ;

`template` $\leftarrow$ `OMRTemplate`(`examId` = $V.\text{examId}$, `versionIndex` = $V.\text{versionIndex}$) ;

`template.studentIdRegion` $\leftarrow$ `extractRegion`(`stdBoxes`, `type=studentId`) ;

`template.examCodeRegion` $\leftarrow$ `extractRegion`(`stdBoxes`, `type=examCode`) ;

`template.answerGrid` $\leftarrow$ `buildAnswerGrid`($V.\text{questionOrder}$, `stdBoxes`) ;

`template.answerKey` $\leftarrow$ $V.\text{answerKey}$;

return `template` ;

e, Giai đoạn 5 – Lưu trữ và phân phối đề thi

Sau khi tất cả file đã được sinh và tải lên, hệ thống thực hiện lưu trữ có cấu trúc để phục vụ các nghiệp vụ sau: phát hành đề, chấm thi, thống kê.

Bước 5.1, lưu thông tin phiên bản vào database:

- `ExamVersion`: lưu seed, thứ tự câu, đáp án, URL PDF, URL sheet
- `OMRTemplate`: lưu `templateJson` đầy đủ tọa độ bubble

- Exam: cập nhật trạng thái `published` và số phiên bản đã sinh

Bước 5.2, tạo quan hệ giữa học sinh và phiên bản đề. Hệ thống phân bổ ngẫu nhiên phiên bản cho từng học sinh trong lớp thi, lưu vào `StudentExamVersion`:

$$\text{assignedVersion}_s = V_{s \bmod N} \quad (5.6)$$

với s là số thứ tự học sinh trong danh sách lớp. Phân bổ theo modulo đảm bảo các phiên bản được sử dụng đều nhau.

Bước 5.3, khi giáo viên phát hành đề, hệ thống gửi thông báo đến học sinh. Học sinh có thể xem đề của mình trên web hoặc in ra giấy. Phiếu OMR được in kèm theo đề hoặc phát hành riêng tùy theo cấu hình của trường.

Bước 5.4, khi giáo viên bắt đầu chấm thi bằng điện thoại, ứng dụng gọi API `GET /api/exams/<id>/template` để lấy `OMRTemplate` của phiên bản tương ứng với học sinh đang chấm. Template này được truyền vào `OMREngine.initialize()` để thiết lập tọa độ bubble cho toàn bộ pipeline nhận dạng.

5.1.3 Kết quả đạt được

Pipeline sinh tối đa 50 phiên bản đề kèm phiếu OMR trong dưới 2 phút cho đề 40 câu. Tọa độ các ô chính xác dưới 1 pixel ở 300 DPI. Đã thử nghiệm với 5 bài kiểm tra, hơn 100 phiên bản đề và 300 phiếu OMR; tất cả nhận dạng thành công.

5.2 Giải pháp nhận dạng OMR trên thiết bị di động với khả năng phát hiện lỗi

5.2.1 Dẫn dắt bài toán

Yêu cầu chấm thi bằng ứng dụng di động (mục 2.2.8) đặt ra bài toán kỹ thuật khó: nhận dạng chính xác các ô được tô trên phiếu trả lời từ ảnh chụp bằng camera thông thường, đồng thời phát hiện các lỗi phổ biến như tô nhầm nhiều ô trong cùng một câu, tô quá nhẹ không đạt ngưỡng, hoặc ảnh chụp bị nghiêng, mờ do điều kiện thực tế tại lớp học. Mục 4.4 đã trình bày kết quả kiểm thử; phần này trình bày chi tiết thuật toán.

5.2.2 Giải pháp chi tiết

Module `OMREngine` trong ứng dụng Flutter thực hiện pipeline nhận diện OMR gồm sáu giai đoạn chính. Mỗi giai đoạn được thiết kế độc lập, cho phép tinh chỉnh tham số mà không ảnh hưởng đến các bước khác.

a, Giai đoạn 1 – Tiền xử lý ảnh

Ảnh chụp từ camera được đưa vào chuỗi xử lý chuẩn hóa nhằm giảm nhiễu và tăng độ tương phản trước khi phân tích. Các phép biến đổi được áp dụng theo thứ tự sau.

Bước 1.1, chuyển đổi không gian màu từ RGB sang grayscale để giảm kích thước dữ liệu và loại bỏ thông tin màu không cần thiết cho nhận dạng bubble:

$$I_{gray}(x, y) = 0,299 \cdot I_R(x, y) + 0,587 \cdot I_G(x, y) + 0,114 \cdot I_B(x, y) \quad (5.7)$$

Bước 1.2, lọc nhiễu bằng bộ lọc Gaussian kích thước 5×5 với độ lệch chuẩn $\sigma = 1,0$:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (5.8)$$

Phép tích chập $I_{blur} = I_{gray} * G$ giúp làm mịn nhiễu Gaussian và nhiễu muối tiêu, giữ lại cạnh chính của hình ảnh.

Bước 1.3, nhị phân hóa bằng phương pháp Otsu. Thuật toán tự động chọn ngưỡng τ_{otsu} tối ưu bằng cách tối đa hóa phương sai giữa hai lớp pixel nền và nội dung:

$$\tau_{otsu} = \arg \max_{\tau} (\omega_0(\tau)\sigma_0^2(\tau) + \omega_1(\tau)\sigma_1^2(\tau)) \quad (5.9)$$

trong đó ω_0, ω_1 là tỉ lệ pixel của hai lớp và σ_0^2, σ_1^2 là phương sai của từng lớp. Kết quả nhị phân:

$$I_{bin}(x, y) = \begin{cases} 1 & \text{nếu } I_{blur}(x, y) > \tau_{otsu} \\ 0 & \text{nếu không} \end{cases} \quad (5.10)$$

Bước 1.4, phép toán morphological opening và closing để loại bỏ nhiễu nhỏ và lấp lỗ trống trong vùng bubble:

$$I_{open} = I_{bin} \circ SE = (I_{bin} \ominus SE) \oplus SE \quad (5.11)$$

$$I_{clean} = I_{open} \bullet SE = (I_{open} \oplus SE) \ominus SE \quad (5.12)$$

với SE là phần tử cấu trúc hình elip 3×3 .

Thuật toán tổng thể giai đoạn 1 được tóm tắt trong Pseudocode 4.

Algorithm 4: Tiền xử lý ảnh OMR

Input: Ảnh RGB I_{RGB} kích thước $H \times W \times 3$

Output: Ảnh nhị phân đã làm sạch I_{clean}

$I_{gray} \leftarrow \text{convertToGrayscale}(I_{RGB})$;

$I_{blur} \leftarrow \text{GaussianBlur}(I_{gray}, 5 \times 5, \sigma = 1.0)$;

$\tau \leftarrow \text{OtsuThreshold}(I_{blur})$;

$I_{bin} \leftarrow \text{applyThreshold}(I_{blur}, \tau)$;

$I_{clean} \leftarrow \text{morphologicalClosing}(\text{morphologicalOpening}(I_{bin}))$;

return I_{clean} ;

b, Giai đoạn 2 – Phát hiện và hiệu chỉnh góc nghiêng

Ảnh chụp từ camera thường bị nghiêng do góc cầm điện thoại không vuông góc với mặt phẳng phiếu. Để đảm bảo tọa độ bubble chính xác, thuật toán tự động phát hiện góc xoay và hiệu chỉnh.

Bước 2.1, phát hiện cạnh bằng toán tử Canny với ngưỡng thấp $t_{low} = 50$, ngưỡng cao $t_{high} = 150$, kích thước kernel Sobel 3×3 :

$$I_{edge} = \text{Canny}(I_{blur}, t_{low}, t_{high}) \quad (5.13)$$

Bước 2.2, biến đổi Hough xác định các đường thẳng chính của phiếu bằng cách tìm cực đại trong không gian accumulator:

$$r = x \cos \theta + y \sin \theta, \quad \theta \in [0^\circ, 180^\circ] \quad (5.14)$$

Hệ thống chọn hai đường thẳng có số phiếu bầu cao nhất và không gần song song với trục hoành để tính góc xoay α . Nếu $|\alpha| < 1^\circ$, bước này bị bỏ qua để tiết kiệm thời gian.

Bước 2.3, xoay ảnh ngược chiều $-\alpha$ bằng phép affine transformation:

$$I_{rot}(x', y') = I_{rot}(x \cos \alpha + y \sin \alpha, -x \sin \alpha + y \cos \alpha) \quad (5.15)$$

Sau xoay, ảnh được cắt biên thừa để đảm bảo kích thước khớp với template OMR.

Pseudocode 5 mô tả quy trình hiệu chỉnh góc nghiêng.

Algorithm 5: Phát hiện và hiệu chỉnh góc nghiêng

Input: Ảnh xám đã làm mờ I_{blur}
Output: Ảnh đã xoay thẳng I_{rot}
 $I_{edge} \leftarrow \text{Canny}(I_{blur}, 50, 150)$;
 $(r_1, \theta_1), (r_2, \theta_2) \leftarrow \text{HoughLinesP}(I_{edge}, \text{topK} = 2)$;
 $\alpha \leftarrow \text{computeRotationAngle}(\theta_1, \theta_2)$;
if $|\alpha| < 1^\circ$ **then**
 return I_{blur} ;
 $M \leftarrow \text{rotationMatrix}(\alpha, \text{center} = (W/2, H/2))$;
 $I_{rot} \leftarrow \text{warpAffine}(I_{blur}, M, (W, H))$;
return I_{rot} ;

c, Giai đoạn 3 – Phát hiện vùng phiếu

Sau khi ảnh đã thẳng, hệ thống cần xác định chính xác biên của phiếu trả lời để cắt vùng làm việc, loại bỏ nền và các vật thể nhiễu xung quanh.

Bước 3.1, tìm tất cả đường viền (contours) bằng hàm `findContours` trên ảnh nhị phân I_{clean} , sử dụng chuỗi lặp liên kế đơn giản với phương pháp trích xuất `RETR_EXTERNAL`.

Bước 3.2, lọc contours theo tiêu chí diện tích. Chỉ giữ lại contours có diện tích S nằm trong khoảng:

$$S_{min} \leq S \leq S_{max} \quad (5.16)$$

với $S_{min} = 0,15 \cdot H \cdot W$ và $S_{max} = 0,95 \cdot H \cdot W$. Các contours nhỏ hơn S_{min} được loại bỏ vì có thể là nhiễu điểm hoặc ký tự in nhỏ; contours lớn hơn S_{max} bị loại vì thường là viền khung hình ảnh.

Bước 3.3, từ các contours hợp lệ, chọn contour lớn nhất làm biên phiếu chính:

$$C_{sheet} = \arg \max_{C_i \in \mathcal{C}_{valid}} \text{Area}(C_i) \quad (5.17)$$

Bước 3.4, xấp xỉ C_{sheet} thành tứ giác bằng phương pháp `approxPolyDP` với sai số tối đa $\varepsilon = 0,02 \cdot \text{perimeter}(C_{sheet})$. Nếu số đỉnh khác 4, thuật toán thử tìm tứ giác ngoài cùng của contours cha bằng cấu trúc phân cấp `RETR_TREE`.

Bước 3.5, thực hiện phối cảnh biến đổi (perspective transform) để đưa tứ giác về hình chữ nhật chuẩn kích thước $W_{std} \times H_{std}$ (khớp với kích thước template OMR ở 300 DPI):

$$I_{warp} = \text{perspectiveTransform}(I_{gray}, C_{sheet}, P_{std}) \quad (5.18)$$

Pseudocode 6 tóm tắt quy trình phát hiện vùng phiếu.

Algorithm 6: Phát hiện vùng phiếu trả lời

Input: Ảnh nhị phân I_{clean} , ảnh gốc xám I_{gray}
Output: Ảnh phiếu đã chuẩn hóa I_{warp}
 $\mathcal{C} \leftarrow \text{findContours}(I_{clean}, \text{RETR_EXTERNAL})$;
 $\mathcal{C}_{valid} \leftarrow \{C \in \mathcal{C} \mid S_{min} \leq \text{Area}(C) \leq S_{max}\}$;
 $C_{sheet} \leftarrow \arg \max_{C \in \mathcal{C}_{valid}} \text{Area}(C)$;
 $P_{sheet} \leftarrow \text{approxPolyDP}(C_{sheet}, \varepsilon = 0.02 \cdot \text{peri}(C_{sheet}))$;
if $|P_{sheet}| \neq 4$ **then**
 $P_{sheet} \leftarrow \text{findLargestQuad}(P_{sheet})$;
end
 $P_{std} \leftarrow \text{standardRect}(W_{std}, H_{std})$;
 $I_{warp} \leftarrow \text{warpPerspective}(I_{gray}, P_{sheet}, P_{std})$;
return I_{warp} ;

d, Giai đoạn 4 – Phát hiện bubble và đọc mã

Vùng phiếu đã chuẩn hóa chứa hai loại thông tin cần trích xuất: (a) vùng cố định chứa mã đề và mã số sinh viên, (b) vùng câu trả lời gồm các bubble được sắp xếp theo lưới.

Bước 4.1, định vị vùng mã đề và mã số sinh viên dựa trên tọa độ tham chiếu từ template OMR. Mỗi vùng mã là một hình chữ nhật nhỏ được chia thành n cột (số câu/số ký tự), mỗi cột chứa 4 bubble tương ứng 4 lựa chọn.

Bước 4.2, với mỗi bubble có tọa độ chuẩn (x_i, y_i) , hệ thống trích xuất vùng ảnh nhỏ quanh tâm bubble:

$$R_i = I_{warp}[y_i - r : y_i + r, x_i - r : x_i + r] \quad (5.19)$$

với bán kính $r = 12$ pixel ở 300 DPI.

Bước 4.3, đánh giá mức độ tô của bubble bằng công thức tỉ lệ tô đậm (fill ratio):

$$f_i = \frac{\text{Area}(R_i \cap I_{bin})}{\text{Area}(R_i)} \quad (5.20)$$

trong đó I_{bin} là ảnh nhị phân của vùng R_i . Giá trị $f_i \in [0, 1]$ thể hiện tỉ lệ pixel đen bên trong vùng bubble.

Bước 4.4, xác định trạng thái bubble:

$$\text{status}_i = \begin{cases} \text{FILLED} & \text{nếu } f_i \geq \theta \\ \text{EMPTY} & \text{nếu } f_i < \theta_{light} \\ \text{LIGHT} & \text{nếu } \theta_{light} \leq f_i < \theta \end{cases} \quad (5.21)$$

với ngưỡng chính $\theta = 0,4$ và ngưỡng tô nhẹ $\theta_{light} = 0,15$.

Bước 4.5, đọc mã số sinh viên và mã đề bằng cách chọn bubble có f_i lớn nhất trong mỗi cột, trả về ký tự tương ứng (A, B, C, D). Nếu không có bubble nào vượt θ , cột đó được đánh dấu là không đọc được.

Pseudocode 7 mô tả quy trình đọc bubble và mã.

Algorithm 7: Phát hiện bubble và đọc mã

Input: Ảnh đã chuẩn hóa I_{warp} , danh sách bubble $\mathcal{B} = \{(x_i, y_i)\}$

Output: Map mã đề, Map mã sinh viên, Map đáp án

$codes \leftarrow \emptyset$; $answers \leftarrow \emptyset$;

foreach $codeRegion \in template.codeRegions$ **do**

foreach $col \in codeRegion.columns$ **do**

$b_{best} \leftarrow \arg \max_{b \in col} f(b)$;

if $f(b_{best}) \geq \theta$ **then**

$codes[region] \leftarrow col.indexOf(b_{best})$;

end

end

end

foreach $row \in template.answerRows$ **do**

$b_{best} \leftarrow \arg \max_{b \in row} f(b)$;

if $f(b_{best}) \geq \theta$ **then**

$answers[row.questionNo] \leftarrow row.choices[b_{best}]$;

end

else if $\exists b \in row : f(b) \geq \theta_{light}$ **then**

$answers[row.questionNo] \leftarrow FLAG_LIGHT$;

end

if $|\{b \in row : f(b) \geq \theta\}| > 1$ **then**

$answers[row.questionNo] \leftarrow FLAG_MULTI$;

end

end

return $codes, answers$;

e, Giai đoạn 5 – Đánh giá câu trả lời và phát hiện lỗi

Sau khi trích xuất đáp án thô từ phiếu, hệ thống thực hiện đánh giá so với đáp án chuẩn của phiên bản đề tương ứng và phát hiện các trường hợp bất thường.

Bước 5.1, so sánh đáp án đọc được với đáp án đúng từ ExamVersion.answerKey:

$$score = \sum_{q=1}^Q \mathbb{1}(\text{readAns}_q = \text{correctAns}_q) \times \frac{10}{Q} \quad (5.22)$$

với Q là tổng số câu hỏi.

Bước 5.2, phát hiện lỗi tô nhầm. Với mỗi câu q , đếm số bubble thỏa $f_i \geq \theta$:

$$n_{marked}(q) = |\{b \in \text{row}_q \mid f(b) \geq \theta\}| \quad (5.23)$$

Nếu $n_{marked}(q) > 1$, hệ thống ghi nhận lỗi `MULTI_MARK` và không tính điểm câu đó, đồng thời lưu danh sách bubble được tô để hiển thị lại cho giáo viên.

Bước 5.3, phát hiện lỗi tô nhẹ. Nếu không có bubble nào vượt ngưỡng θ nhưng tồn tại bubble với $\theta_{light} \leq f_i < \theta$, ghi nhận lỗi `LIGHT_MARK`. Câu này được đánh dấu cần kiểm tra thủ công.

Bước 5.4, phát hiện lỗi bỏ trống. Nếu $n_{marked}(q) = 0$ và không có bubble nào rơi vào khoảng $[\theta_{light}, \theta)$, ghi nhận lỗi `BANK_Graded`.

Bước 5.5, tổng hợp kết quả thành đối tượng `OMRResult` gồm: điểm số, danh sách đáp án đọc được, danh sách lỗi theo câu, ảnh đã xử lý có vẽ biên bubble để hiển thị minh họa. Giáo viên có thể chấp nhận kết quả hoặc điều chỉnh điểm thủ công trên giao diện web.

Pseudocode 8 mô tả quy trình đánh giá và phát hiện lỗi.

Algorithm 8: Đánh giá câu trả lời và phát hiện lỗi

Input: Đáp án đọc được $readAns$, đáp án chuẩn $correctAns$, bảng $\mathcal{B}$

Output: Điểm số, danh sách lỗi

$score \leftarrow 0$; $errors \leftarrow \emptyset$;

foreach $q \in [1, Q]$ **do**

$n_{marked} \leftarrow |\{b \in row_q : f(b) \geq \theta\}|$;

$n_{light} \leftarrow |\{b \in row_q : \theta_{light} \leq f(b) < \theta\}|$;

if $n_{marked} = 1$ **then**

if $readAns_q = correctAns_q$ **then**

$score \leftarrow score + \frac{10}{Q}$;

end

if $n_{marked} > 1$ **then**

$errors[q] \leftarrow \text{MULTI_MARK}$;

else if $n_{marked} = 0 \wedge n_{light} > 0$ **then**

$errors[q] \leftarrow \text{LIGHT_MARK}$;

end

else if $n_{marked} = 0 \wedge n_{light} = 0$ **then**

$errors[q] \leftarrow \text{BANK_Graded}$;

end

end

return $score, errors$;

f, Giai đoạn 6 – Tính điểm và lưu kết quả

Kết quả cuối cùng của pipeline là đối tượng `OMRResult` chứa: (a) điểm số tổng và điểm chi tiết từng câu, (b) mã đề và mã sinh viên đọc được, (c) danh sách lỗi kèm mức độ nghiêm trọng, (d) ảnh đã xử lý có annotation biên bubble và mã QR/nhận dạng. Ứng dụng hiển thị kết quả tạm thời cho giáo viên xem trước trong 5 giây, sau đó tự động gửi lên server qua API `POST /api/submissions/omr`. Server thực hiện kiểm tra quyền sở hữu đề thi, lưu vào MongoDB và kích hoạt thông báo cho phụ huynh nếu có bất tính năng.

5.2.3 Kết quả đạt được

Thuật toán đạt độ chính xác 96,2% trên 300 phiếu thử nghiệm thực tế. Thời gian xử lý trung bình 2,3 giây/phiếu trên điện thoại Android tầm trung. Phát hiện 100% trường hợp tô nhầm và tô nhẹ. Hệ thống đã xử lý thành công hơn 900 phiếu trong môi trường lớp học thực tế với đủ điều kiện ánh sáng và góc chụp khác nhau.

5.3 Giải pháp quản lý phiên làm việc và phân quyền theo vai trò trên đa nền tảng

5.3.1 Dẫn dắt bài toán

Hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR phục vụ năm vai trò người dùng khác nhau (admin, school-admin, teacher, student, parent) trên cả ba nền tảng (web, di động, API). Bài toán đặt ra là đảm bảo mỗi vai trò chỉ truy cập được các chức năng và dữ liệu phù hợp, đồng thời duy trì phiên làm việc ổn định trong thời gian dài mà không ảnh hưởng đến trải nghiệm người dùng.

5.3.2 Giải pháp chi tiết

Xác thực dùng JWT với cặp access token (15 phút) và refresh token (7 ngày). Khi access token hết hạn, ứng dụng tự gọi API refresh-token. Refresh token lưu trong cơ sở dữ liệu với thông tin thiết bị để thu hồi khi cần.

Phân quyền hai lớp: lớp định tuyến (middleware auth kiểm tra JWT, middleware permission kiểm tra vai trò) và lớp nghiệp vụ (service kiểm tra quyền sở hữu dữ liệu). Trên web, route guard trong React Router chuyển hướng theo vai trò (admin → /admin, school-admin → /school, teacher → /, student → /my-scores), sidebar lọc menu theo vai trò. Trên di động, BLoC auth quản lý điều hướng qua `go_router` với route có bảo vệ.

5.3.3 Kết quả đạt được

100% endpoint API được bảo vệ bằng kiểm tra quyền. Thời gian phản hồi tăng không đáng kể (khoảng 5ms). Cơ chế refresh tự động giảm phiền toái cho người dùng.

Chương 5 đã trình bày ba giải pháp kỹ thuật nổi bật: pipeline sinh đề-phiếu OMR với tọa độ chính xác, thuật toán OMR trên di động đạt 96,2%, và quản lý phiên làm việc và phân quyền. Chương 6 sẽ tổng kết, so sánh với sản phẩm tương tự và đề xuất hướng phát triển.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết toàn bộ quá trình thực hiện, so sánh kết quả với các sản phẩm tương tự, phân tích hạn chế và đóng góp, rút ra bài học kinh nghiệm và đề xuất hướng phát triển.

6.1 Kết luận

So với Remark Office OMR, Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR có ưu điểm tích hợp đầy đủ (quản lý đào tạo, phúc khảo, ứng dụng di động, miễn phí). So với Gradescope, có ưu điểm hỗ trợ tiếng Việt, tạo nhiều mã đề tự động, tích hợp AI. So với OMRChecker, có ưu điểm giao diện quản lý hoàn chỉnh, phúc khảo, báo cáo thống kê.

Hạn chế: chưa hỗ trợ phiếu multi-column phức tạp, chưa hỗ trợ dạng câu hỏi khác ngoài trắc nghiệm, độ chính xác OMR thấp hơn máy quét chuyên dụng khoảng 2-3%.

Đã hoàn thành: khảo sát và phân tích yêu cầu; thiết kế kiến trúc phân lớp; backend Node.js (17 model, 16 service, 17 nhóm API); web React (phân quyền, dashboard, đầy đủ chức năng); ứng dụng Flutter (OMR 96,2%); pipeline AMC sinh đề-phiếu OMR; tích hợp AI đa nhà cung cấp; thử nghiệm 5 bài kiểm tra, khoảng 300 bài làm.

Đóng góp nổi bật: pipeline tự động sinh đề nhiều phiên bản và phiếu OMR kèm tọa độ; thuật toán chấm OMR trên di động với phát hiện lỗi tô nhầm, tô nhẹ; tích hợp AI trong sinh câu hỏi và trợ lý ảo cho học sinh.

Chưa hoàn thành: hỗ trợ dạng câu hỏi khác ngoài trắc nghiệm; chế độ ngoại tuyến hoàn chỉnh trên di động; hỗ trợ phiếu nhiều hơn 4 đáp án; thanh toán trực tuyến; bản dịch tiếng Anh.

Bài học kinh nghiệm: lựa chọn công nghệ phù hợp ngay từ đầu có ý nghĩa quyết định; thiết kế cơ sở dữ liệu cần cẩn thận vì thay đổi schema rất tốn kém; kiểm thử cần xây dựng song song phát triển; cần kế hoạch rõ ràng và công cụ quản lý dự án (Git) để tránh mất kiểm soát.

6.2 Hướng phát triển

6.2.1 Hoàn thiện các chức năng hiện có

Cải thiện độ chính xác OMR lên trên 98% bằng học sâu (deep learning) để phát hiện ô tô; bổ sung chế độ ngoại tuyến cho di động với đồng bộ tự động; tối ưu backend cho khối lượng lớn trong các kỳ thi hàng nghìn thí sinh.

6.2.2 Mở rộng tính năng mới

Hỗ trợ thêm dạng câu hỏi khác (điền số, điền từ, ghép đôi, tự luận ngắn chấm bằng AI); hỗ trợ chấm thi kết hợp online/offline; tạo đề thông minh theo ma trận kiến thức; phân tích học tập cá nhân hóa cho học sinh; tích hợp thanh toán trực tuyến (SaaS); dịch giao diện tiếng Anh; tích hợp LMS qua chuẩn LTI (Moodle, Canvas).

6.2.3 Nâng cấp công nghệ

Chuyển đổi backend sang microservices (sinh đề, chấm thi, phân tích) khi quy mô tăng; áp dụng học sâu nâng cao nhận dạng OMR; tích hợp mô hình AI tiên tiến cho báo cáo và trợ lý; xây dựng hệ thống giám sát và cảnh báo sớm.

Chương 6 đã tổng kết toàn bộ quá trình thực hiện Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR, so sánh với các sản phẩm tương tự, phân tích hạn chế và đóng góp, rút ra bài học kinh nghiệm và đề xuất hướng phát triển. Đề án đạt mục tiêu đề ra và tạo nền tảng cho các phiên bản tiếp theo.

TÀI LIỆU THAM KHẢO

- [1] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [3] OMRChecker, *OMRChecker: OMR Sheet Scanning & Analysis*, <https://github.com/Udayraj123/OMRChecker>, Truy cập: 28-06-2026, 2022.
- [4] Auto Multiple Choice Project, *Auto Multiple Choice Documentation*, <https://www.auto-multiple-choice.net/>, Truy cập: 28-06-2026, 2024.
- [5] OpenJS Foundation, *Node.js Documentation*, <https://nodejs.org/en/docs>, Truy cập: 28-06-2026, 2024.
- [6] Express.js Authors, *Express - Fast, unopinionated, minimalist web framework for Node.js*, <https://expressjs.com/>, Truy cập: 28-06-2026, 2024.
- [7] MDN Web Docs, *Model-View-Controller (MVC)*, <https://developer.mozilla.org/en-US/docs/Glossary/MVC>, Truy cập: 28-06-2026, 2024.
- [8] IETF, *JSON Web Token (JWT)*. Internet Engineering Task Force, 2015, RFC 7519.
- [9] Passport.js, *Passport.js - Simple, unobtrusive authentication for Node.js*, <https://www.passportjs.org/>, Truy cập: 28-06-2026, 2024.
- [10] MongoDB Inc., *MongoDB: The Developer Data Platform*, <https://www.mongodb.com/docs>, Truy cập: 28-06-2026, 2024.
- [11] MongoDB Inc., *Mongoose ODM Documentation*, <https://mongoosejs.com/docs/>, Truy cập: 28-06-2026, 2024.
- [12] Meta Open Source, *React - A JavaScript library for building user interfaces*, <https://react.dev/>, Truy cập: 28-06-2026, 2024.
- [13] Poimandres, *Zustand - Bear necessities for state management in React*, <https://github.com/pmndrs/zustand>, Truy cập: 28-06-2026, 2024.
- [14] Google, *Flutter Documentation*, <https://docs.flutter.dev/>, Truy cập: 28-06-2026, 2024.
- [15] Felix Angelov, *BLoC Pattern for Flutter*, <https://bloclibrary.dev/>, Truy cập: 28-06-2026, 2024.
- [16] Google, *Gemini API Documentation*, <https://ai.google.dev/docs>, Truy cập: 28-06-2026, 2024.

- [17] OpenAI, *OpenAI API Reference*, <https://platform.openai.com/docs/api-reference>, Truy cập: 28-06-2026, 2024.
- [18] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is All you Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [19] Cloudinary Ltd., *Cloudinary Documentation*, <https://cloudinary.com/documentation>, Truy cập: 28-06-2026, 2024.

PHỤ LỤC

A. ĐẶC TẢ USE CASE

Phần này trình bày đặc tả chi tiết cho các use case phụ của hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR không được đặc tả trong mục 2.3 của Chương 2 do hạn chế về dung lượng. Mỗi đặc tả gồm mã use case, tên use case, tác nhân, điều kiện trước, luồng thực thi chính, luồng thực thi mở rộng và điều kiện kết thúc, theo đúng mẫu đặc tả đã sử dụng trong Chương 2.

A.0.1 Đặc tả use case Quản lý trường học

Thông tin chung	Chi tiết
Mã UC (UC #)	UC001
Tên use case	Quản lý trường học
Tác nhân	Quản trị viên hệ thống (admin)
Điều kiện trước	Quản trị viên hệ thống đã đăng nhập vào hệ thống với vai trò admin.

Bảng A.1: Thông tin chung use case Quản lý trường học

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Quản trị viên	Truy cập chức năng "Quản lý trường học" trên giao diện web.
2	Hệ thống	Hiển thị danh sách các trường hiện có kèm các thông tin: tên trường, mã trường, địa chỉ, số lượng giáo viên, số lượng học sinh.
3	Quản trị viên	Chọn chức năng "Tạo trường mới".
4	Hệ thống	Hiển thị form tạo trường với các trường: tên trường, mã trường, địa chỉ, số điện thoại, email, cấu hình thang điểm.
5	Quản trị viên	Nhập thông tin và nhấn nút "Lưu".
6	Hệ thống	Kiểm tra tính hợp lệ của thông tin, kiểm tra mã trường trùng lặp, lưu trường mới vào cơ sở dữ liệu và hiển thị thông báo thành công.
7	Quản trị viên	Chọn một trường trong danh sách để xem chi tiết, cập nhật thông tin hoặc xóa trường.
8	Hệ thống	Cập nhật thông tin trường và ghi nhận nhật ký thao tác vào cơ sở dữ liệu.

Bảng A.2: Luồng thực thi chính của use case Quản lý trường học

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
6a	Hệ thống	Thông báo lỗi: Mã trường đã tồn tại. Vui lòng nhập mã trường khác.
6b	Hệ thống	Thông báo lỗi: Vui lòng nhập đầy đủ các trường bắt buộc (tên trường, mã trường, địa chỉ).
7a	Hệ thống	Thông báo cảnh báo: Trường đang có giáo viên hoặc học sinh hoạt động. Bạn có chắc chắn muốn xóa?

Bảng A.3: Luồng thực thi mở rộng của use case Quản lý trường học

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, trường mới được tạo và có thể được gán cho quản trị viên trường. Nếu thất bại, trạng thái danh sách trường không thay đổi.

Bảng A.4: Điều kiện kết thúc use case Quản lý trường học

A.0.2 Đặc tả use case Quản lý học sinh

Thông tin chung	Chi tiết
Mã UC (UC #)	UC004
Tên use case	Quản lý học sinh
Tác nhân	Quản trị viên trường (school-admin), Giáo viên (teacher)
Điều kiện trước	Quản trị viên trường hoặc giáo viên đã đăng nhập vào hệ thống.

Bảng A.5: Thông tin chung use case Quản lý học sinh

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Người dùng	Truy cập chức năng "Quản lý học sinh" trên giao diện web.
2	Hệ thống	Hiển thị danh sách học sinh theo lớp hoặc theo trường kèm các điều khiển tìm kiếm, lọc.
3	Người dùng	Chọn "Thêm học sinh vào lớp".
4	Hệ thống	Hiển thị form cho phép thêm một học sinh mới (tạo tài khoản) hoặc thêm học sinh đã có tài khoản vào lớp.
5	Người dùng	Nhập thông tin cần thiết gồm: họ tên, mã học sinh, ngày sinh, lớp và nhấn nút "Lưu".
6	Hệ thống	Kiểm tra tính hợp lệ, kiểm tra mã học sinh trùng lặp, lưu thông tin và gửi thông báo cho học sinh.
7	Người dùng	Chọn một học sinh để xem chi tiết, cập nhật thông tin, chuyển lớp hoặc xóa khỏi lớp.
8	Hệ thống	Cập nhật thông tin học sinh tương ứng và ghi nhận nhật ký thao tác vào cơ sở dữ liệu.

Bảng A.6: Luồng thực thi chính của use case Quản lý học sinh

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
5a	Hệ thống	Thông báo lỗi: Mã học sinh đã tồn tại. Vui lòng nhập mã khác hoặc chọn học sinh đã có tài khoản.
5b	Hệ thống	Thông báo lỗi: Vui lòng nhập đầy đủ các trường bắt buộc (họ tên, mã học sinh, lớp).
7a	Hệ thống	Thông báo cảnh báo: Học sinh đang có bài thi chưa hoàn thành. Bạn có chắc chắn muốn xóa?
7b	Hệ thống	Hiện thị form yêu cầu chọn lớp đích khi chuyển lớp.

Bảng A.7: Luồng thực thi mở rộng của use case Quản lý học sinh

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, học sinh được thêm vào lớp và có thể tham gia các bài kiểm tra của lớp. Nếu thất bại, trạng thái danh sách học sinh không thay đổi.

Bảng A.8: Điều kiện kết thúc use case Quản lý học sinh**A.0.3 Đặc tả use case Quản lý các bài thi của học sinh**

Thông tin chung	Chi tiết
Mã UC (UC #)	UC007
Tên use case	Quản lý các bài thi của học sinh
Tác nhân	Giáo viên (teacher)
Điều kiện trước	Giáo viên đã đăng nhập vào hệ thống, có bài làm đã được chấm tự động trên ứng dụng.

Bảng A.9: Thông tin chung use case Quản lý các bài thi của học sinh**Luồng thực thi chính**

No.	Thực hiện	Hành động
1	Giáo viên	Truy cập chức năng "Bài nộp" trên giao diện web và chọn bài kiểm tra tương ứng.
2	Hệ thống	Hiện thị danh sách bài làm của học sinh với các thông tin: tên học sinh, mã số, điểm, trạng thái.
3	Giáo viên	Chọn một bài làm để xem chi tiết.
4	Hệ thống	Hiện thị thông tin chi tiết gồm: ảnh phiếu đã chấm, đáp án của học sinh, đáp án đúng, điểm từng câu, cảnh báo (nếu có).
5	Giáo viên	Chỉnh sửa thủ công các câu bị nhận dạng sai bằng cách chọn lại đáp án đúng trong giao diện.
6	Giáo viên	Nhấn nút "Lưu chỉnh sửa" để cập nhật điểm.
7	Hệ thống	Ghi nhận lịch sử chỉnh sửa, cập nhật điểm tự động và lưu vào cơ sở dữ liệu.
8	Giáo viên	Nhấn nút "Phê duyệt kết quả" để chuyển bài làm sang trạng thái sẵn sàng công bố.

Bảng A.10: Luồng thực thi chính của use case Quản lý các bài thi của học sinh

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
5a	Hệ thống	Thông báo cảnh báo: Phát hiện câu bị tô nhầm hoặc tô nhẹ. Yêu cầu giáo viên xác nhận lại đáp án.
7a	Hệ thống	Ghi nhận lịch sử chỉnh sửa gồm: người chỉnh sửa, thời gian, câu được chỉnh sửa, điểm cũ và điểm mới.
8a	Hệ thống	Thông báo: Bài làm chưa được phê duyệt. Học sinh chưa thể xem điểm.

Bảng A.11: Luồng thực thi mở rộng của use case Quản lý các bài thi của học sinh

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, bài làm được cập nhật và sẵn sàng công bố cho học sinh khi giáo viên phát hành kết quả. Nếu thất bại, trạng thái bài làm không thay đổi.

Bảng A.12: Điều kiện kết thúc use case Quản lý các bài thi của học sinh**A.0.4 Đặc tả use case Xem bài thi và điểm số**

Thông tin chung	Chi tiết
Mã UC (UC #)	UC010
Tên use case	Xem bài thi và điểm số
Tác nhân	Học sinh (student)
Điều kiện trước	Học sinh đã đăng nhập vào hệ thống, bài thi đã có kết quả được công bố.

Bảng A.13: Thông tin chung use case Xem bài thi và điểm số**Luồng thực thi chính**

No.	Thực hiện	Hành động
1	Học sinh	Truy cập chức năng "Bài thi của tôi" hoặc "Điểm của tôi" trên giao diện web hoặc ứng dụng di động.
2	Hệ thống	Hiển thị danh sách các bài thi đã tham gia với điểm tổng và trạng thái công bố.
3	Học sinh	Chọn một bài thi để xem chi tiết.
4	Hệ thống	Hiển thị thông tin chi tiết gồm: điểm tổng, điểm từng câu, đáp án của học sinh, đáp án đúng, ảnh phiếu đã chấm.
5	Học sinh	Xem các đơn phúc khảo đã gửi cho bài thi này và trạng thái của chúng.
6	Học sinh	Nhấn nút "Tải báo cáo" để tải về báo cáo điểm cá nhân dạng PDF.
7	Hệ thống	Sinh file PDF báo cáo điểm và tải về thiết bị của học sinh.

Bảng A.14: Luồng thực thi chính của use case Xem bài thi và điểm số

Luồng thực thi mở rộng

No.	Thực hiện	Hành động
4a	Hệ thống	Thông báo: Bài thi có câu bị giáo viên chỉnh sửa thủ công. Hiện thị danh sách các câu được chỉnh sửa để học sinh biết.
6a	Hệ thống	Thông báo lỗi: Tải báo cáo thất bại. Vui lòng thử lại.

Bảng A.15: Luồng thực thi mở rộng của use case Xem bài thi và điểm số

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, học sinh nắm được kết quả chi tiết bài thi của mình và có thể tải báo cáo điểm. Nếu thất bại, học sinh vẫn xem được thông tin cơ bản.

Bảng A.16: Điều kiện kết thúc use case Xem bài thi và điểm số

A.0.5 Đặc tả use case Đăng ký/Đăng nhập

Thông tin chung	Chi tiết
Mã UC (UC #)	UC002
Tên use case	Đăng ký/Đăng nhập
Tác nhân	Tất cả người dùng (admin, school-admin, teacher, student)
Điều kiện trước	Người dùng truy cập được vào ứng dụng web hoặc di động.

Bảng A.17: Thông tin chung use case Đăng ký/Đăng nhập

Luồng thực thi chính

No.	Thực hiện	Hành động
1	Người dùng	Truy cập trang đăng nhập của ứng dụng.
2	Người dùng	Nhập email và mật khẩu, nhấn nút "Đăng nhập".
3	Hệ thống	Kiểm tra thông tin đăng nhập: xác thực email và mật khẩu đã nhập.
4	Hệ thống	Nếu hợp lệ, cấp access token và refresh token, chuyển hướng đến dashboard phù hợp với vai trò người dùng.
5	Người dùng	Nếu chưa có tài khoản, chọn "Đăng ký" và nhập thông tin cá nhân gồm: họ tên, email, mật khẩu.
6	Hệ thống	Gửi email xác thực đến địa chỉ email đã đăng ký.
7	Người dùng	Truy cập link trong email để xác thực tài khoản.
8	Hệ thống	Kích hoạt tài khoản và cho phép người dùng đăng nhập.
9	Người dùng	Nếu quên mật khẩu, chọn "Quên mật khẩu" và nhập email đã đăng ký.
10	Hệ thống	Gửi email chứa link khôi phục mật khẩu.
11	Người dùng	Truy cập link và nhập mật khẩu mới để đặt lại.
12	Hệ thống	Cập nhật mật khẩu mới và cho phép đăng nhập.

Bảng A.18: Luồng thực thi chính của use case Đăng ký/Đăng nhập**Luồng thực thi mở rộng**

No.	Thực hiện	Hành động
3a	Hệ thống	Thông báo lỗi: Email hoặc mật khẩu không đúng. Vui lòng thử lại.
4a	Hệ thống	Thông báo: Tài khoản chưa được xác thực email. Vui lòng kiểm tra hộp thư và xác thực trước khi đăng nhập.
6a	Hệ thống	Thông báo lỗi: Email đã được sử dụng. Vui lòng đăng nhập hoặc sử dụng email khác.
10a	Hệ thống	Thông báo: Email không tồn tại trong hệ thống. Không gửi được email khôi phục.

Bảng A.19: Luồng thực thi mở rộng của use case Đăng ký/Đăng nhập

Điều kiện kết thúc	Chi tiết
Điều kiện sau	Nếu thành công, người dùng được đăng nhập và truy cập các chức năng theo vai trò. Nếu thất bại, người dùng quay lại trang đăng nhập và có thể thử lại.

Bảng A.20: Điều kiện kết thúc use case Đăng ký/Đăng nhập

B. THIẾT KẾ CHI TIẾT CÁC LỚP

B.1 Lớp AppealService (Backend – Node.js)

Vị trí. Gói `services/`, tầng Business Logic.

Mô tả. AppealService quản lý toàn bộ quy trình phúc khảo từ khi học sinh gửi yêu cầu đến khi giáo viên xem xét và cập nhật kết quả. Lớp kiểm tra tính hợp lệ của yêu cầu, cập nhật trạng thái và gửi thông báo đến học sinh khi có kết quả.

Các thuộc tính. Bảng C.1 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
appealModel	AppealModel	Model phúc khảo, quản lý dữ liệu yêu cầu phúc khảo trong MongoDB
submissionService	SubmissionService	Service bài nộp, dùng để ghi đề đáp án khi phúc khảo được chấp nhận
notificationService	NotificationService	Service thông báo, dùng để gửi thông báo kết quả phúc khảo đến học sinh
userModel	UserModel	Model người dùng để tra cứu thông tin học sinh và giáo viên

Bảng B.1: Các thuộc tính của lớp AppealService

Các phương thức. Bảng C.2 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
create	userId, data	Promise<Appeal>	Tạo yêu cầu phúc khảo mới từ dữ liệu của học sinh
findById	id: string	Promise<Appeal>	Tìm yêu cầu phúc khảo theo ID
findByStudent	studentId, filters	Promise<Appeal[]>	Tìm tất cả yêu cầu phúc khảo của một học sinh, có hỗ trợ lọc
findByExam	examId, filters	Promise<Appeal[]>	Tìm tất cả yêu cầu phúc khảo của một đề thi, có hỗ trợ lọc
findPending	filters	Promise<Appeal[]>	Tìm các yêu cầu phúc khảo đang trong trạng thái chờ xử lý
review	id, reviewData	Promise<Appeal>	Giáo viên xem xét và phê duyệt yêu cầu phúc khảo
reject	id, reason	Promise<Appeal>	Từ chối yêu cầu phúc khảo với lý do cụ thể
cancel	id, userId	Promise<Appeal>	Học sinh hủy yêu cầu phúc khảo của mình
getStatistics	examId: string	Promise<AppealStatistics>	Lấy thống kê phúc khảo của một đề thi

Bảng B.2: Các phương thức của lớp AppealService

Mối quan hệ. AppealService có composition với AppealModel (tạo, lưu và truy vấn yêu cầu phúc khảo), dependency với SubmissionService (gọi phương thức `overrideAnswer` khi phúc khảo được chấp nhận), dependency với

NotificationService (gửi thông báo cho học sinh khi có kết quả), và sử dụng UserModel để tra cứu thông tin người dùng.

B.2 Lớp NotificationService (Backend – Node.js)

Vị trí. Gói `services/`, tầng Business Logic.

Mô tả. NotificationService quản lý việc gửi thông báo đến người dùng qua nhiều kênh: email, in-app notification và push notification. Lớp đồng thời quản lý trạng thái đọc/chưa đọc của thông báo trong hệ thống.

Các thuộc tính. Bảng C.3 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
notificationModel	NotificationModel	Model thông báo, quản lý dữ liệu thông báo trong MongoDB
emailService	EmailService	Service gửi email, dùng để gửi thông báo qua hộp thư điện tử
pushService	PushService	Service push notification, dùng để gửi thông báo đến thiết bị di động
userModel	UserModel	Model người dùng để tra cứu thông tin và kênh liên lạc của người nhận

Bảng B.3: Các thuộc tính của lớp NotificationService

Các phương thức. Bảng C.4 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
send	notification	Promise<Notification>	Gửi thông báo đơn lẻ đến một người dùng
sendScoreNotification	submission	Promise<void>	Gửi thông báo điểm số khi bài thi được chấm xong
sendBulkNotifications	userIds, notification	Promise<void>	Gửi thông báo đến nhiều người dùng cùng lúc
sendToRole	role, notification	Promise<void>	Gửi thông báo đến tất cả người dùng có vai trò cụ thể
markAsRead	notificationId, userId	Promise<Notification>	Đánh dấu một thông báo là đã đọc
markAllAsRead	userId: string	Promise<void>	Đánh dấu tất cả thông báo của một người dùng là đã đọc
getUnread	userId: string	Promise<Notification[]>	Lấy danh sách thông báo chưa đọc của người dùng
getAll	userId, filters	Promise<Notification[]>	Lấy tất cả thông báo của người dùng, có hỗ trợ lọc

Bảng B.4: Các phương thức của lớp NotificationService

Mối quan hệ. NotificationService có composition với NotificationModel (lưu trữ và truy vấn thông báo), dependency với EmailService (gửi thông báo qua email), dependency với PushService (gửi push notification), và sử dụng UserModel để tra cứu thông tin người nhận.

B.3 Lớp AuthService (Backend – Node.js)

Vị trí. Gói `services/`, tầng Business Logic.

Mô tả. AuthService quản lý xác thực và phân quyền người dùng, bao gồm đăng ký, đăng nhập, đăng xuất, quản lý token, xác thực email và quản lý mật khẩu. Lớp sử dụng JWT để tạo và xác thực access token, đồng thời dùng bcrypt để mã hóa mật khẩu.

Các thuộc tính. Bảng C.5 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
userModel	UserModel	Model người dùng để quản lý tài khoản trong MongoDB
schoolModel	SchoolModel	Model trường học để xác thực và gán trường cho tài khoản mới
emailService	EmailService	Service gửi email để xác thực tài khoản và đặt lại mật khẩu
jwtService	JwtService	Service JWT để tạo và xác thực access/refresh token
bcrypt	BcryptService	Service mã hóa mật khẩu bằng thuật toán bcrypt

Bảng B.5: Các thuộc tính của lớp AuthService

Các phương thức. Bảng C.6 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
register	data	Promise<User>	Đăng ký tài khoản mới với thông tin người dùng
login	email, password	Promise<AuthResult>	Đăng nhập, trả về access token và refresh token
logout	userId, token	Promise<void>	Đăng xuất, vô hiệu hóa token hiện tại
refreshToken	refreshToken	Promise<AuthResult>	Làm mới access token bằng refresh token
verifyEmail	token: string	Promise<User>	Xác thực email bằng token gửi qua mail
forgotPassword	email: string	Promise<void>	Gửi email đặt lại mật khẩu đến người dùng
resetPassword	token, newPassword	Promise<User>	Đặt lại mật khẩu mới sau khi xác thực token
changePassword	userId, oldPassword, newPassword	Promise<User>	Đổi mật khẩu với xác thực mật khẩu cũ
updateProfile	userId, data	Promise<User>	Cập nhật thông tin cá nhân của người dùng
getProfile	userId: string	Promise<User>	Lấy thông tin cá nhân của người dùng
validateToken	token: string	Promise<TokenPayload>	Kiểm tra tính hợp lệ của access token

Bảng B.6: Các phương thức của lớp AuthService

Mối quan hệ. AuthService có composition với UserModel (quản lý tài khoản người dùng), dependency với JwtService (tạo và xác thực token), dependency với BcryptService (mã hóa và so sánh mật khẩu), dependency với EmailService (gửi email xác thực và khôi phục), và sử dụng SchoolModel để xác thực trường học.

B.4 Lớp ClassService (Backend – Node.js)

Vị trí. Gói `services/`, tầng Business Logic.

Mô tả. ClassService quản lý các lớp học trong hệ thống, bao gồm tạo lớp, thêm học sinh, phân công giáo viên chủ nhiệm và giáo viên bộ môn, cũng như quản lý danh sách lớp theo trường học.

Các thuộc tính. Bảng C.7 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
classModel	ClassModel	Model lớp học, quản lý dữ liệu lớp trong MongoDB
userModel	UserModel	Model người dùng để tra cứu và quản lý học sinh, giáo viên
subjectModel	SubjectModel	Model môn học để liên kết lớp với các môn giảng dạy

Bảng B.7: Các thuộc tính của lớp ClassService

Các phương thức. Bảng C.8 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
create	userId, data	Promise<Class>	Tạo lớp học mới với thông tin cơ bản
findById	id: string	Promise<Class>	Tìm lớp học theo ID
findBySchool	schoolId, filters	Promise<Class[]>	Tìm tất cả lớp học thuộc một trường, có hỗ trợ lọc
findByTeacher	teacherId: string	Promise<Class[]>	Tìm các lớp mà một giáo viên được phân công
findByStudent	studentId: string	Promise<Class[]>	Tìm các lớp mà một học sinh tham gia
update	id, data	Promise<Class>	Cập nhật thông tin lớp học
delete	id: string	Promise<void>	Xóa lớp học khỏi hệ thống
addStudents	classId, studentIds	Promise<Class>	Thêm danh sách học sinh vào lớp
removeStudents	classId, studentIds	Promise<Class>	Xóa danh sách học sinh khỏi lớp
setHomeroomTeacher	classId, teacherId	Promise<Class>	Đặt giáo viên chủ nhiệm cho lớp
assignSubjectTeachers	classId, assignments	Promise<Class>	Phân công giáo viên bộ môn cho lớp
getStudents	classId: string	Promise<User[]>	Lấy danh sách học sinh trong lớp
getTeachers	classId: string	Promise<User[]>	Lấy danh sách giáo viên phụ trách lớp

Bảng B.8: Các phương thức của lớp ClassService

Mối quan hệ. ClassService có composition với ClassModel (tạo, lưu và truy vấn lớp học), dependency với UserModel (quản lý học sinh và giáo viên trong lớp), và dependency với SubjectModel (liên kết môn học với lớp học).

B.5 Lớp AuthStore (Frontend – React/Zustand)

Vị trí. Gói `stores/`, tầng State Management.

Mô tả. AuthStore quản lý trạng thái xác thực trên ứng dụng web sử dụng Zustand, bao gồm thông tin user, token, trạng thái đăng nhập và các action đăng nhập, đăng xuất, quản lý token. Store lưu trữ token trong localStorage để duy trì đăng nhập khi reload trang.

Các thuộc tính. Bảng C.9 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
user	User null	Thông tin người dùng hiện tại đã đăng nhập
token	string null	Access token JWT dùng để xác thực API
refreshToken	string null	Refresh token dùng để làm mới access token
isAuthenticated	boolean	Trạng thái đăng nhập, true khi có token hợp lệ
isLoading	boolean	Trạng thái loading khi đang xử lý đăng nhập hoặc đăng ký
error	string null	Thông báo lỗi từ quá trình xác thực

Bảng B.9: Các thuộc tính của lớp AuthStore

Các phương thức. Bảng C.10 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
login	email, password	Promise<void>	Đăng nhập với email và mật khẩu, lưu token vào store
register	data	Promise<void>	Đăng ký tài khoản mới với thông tin người dùng
logout	–	void	Đăng xuất, xóa token và thông tin người dùng khỏi store
refreshToken	–	Promise<void>	Làm mới access token bằng refresh token
updateProfile	data	Promise<void>	Cập nhật thông tin cá nhân của người dùng
changePassword	oldPassword, newPassword	Promise<void>	Đổi mật khẩu với xác thực mật khẩu cũ
forgotPassword	email	Promise<void>	Gửi yêu cầu khôi phục mật khẩu qua email
resetPassword	token, newPassword	Promise<void>	Đặt lại mật khẩu mới sau khi xác thực token
verifyEmail	token	Promise<void>	Xác thực email bằng token từ đường dẫn trong email
initialize	–	Promise<void>	Khởi tạo store từ localStorage khi ứng dụng khởi động

Bảng B.10: Các phương thức của lớp AuthStore

Mối quan hệ. AuthStore sử dụng ApiService để gọi các API xác thực (login, register, logout, refreshToken, verifyEmail, forgotPassword, resetPassword, changePassword, updateProfile) và lưu trữ thông tin user cùng token vào localStorage.

B.6 Lớp SubmissionStore (Frontend – React/Zustand)

Vị trí. Gói stores/, tầng State Management.

Mô tả. SubmissionStore quản lý trạng thái liên quan đến bài nộp và kết quả thi trên ứng dụng web. Store cung cấp các phương thức để lấy danh sách bài nộp, xem chi tiết bài nộp, lấy điểm theo đề thi và cập nhật điểm thủ công.

Các thuộc tính. Bảng C.11 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
submissions	Submission[]	Danh sách bài nộp đã được tải từ server
currentSubmission	Submission null	Bài nộp đang được xem chi tiết
examScores	Map	Dữ liệu điểm theo đề thi, key là examId
isLoading	boolean	Trạng thái loading khi đang tải dữ liệu
error	string null	Thông báo lỗi khi tải dữ liệu thất bại
filters	SubmissionFilters	Bộ lọc hiện tại để tìm kiếm và phân trang bài nộp

Bảng B.11: Các thuộc tính của lớp SubmissionStore

Các phương thức. Bảng C.12 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
fetchSubmissions	filters	Promise<void>	Lấy danh sách bài nộp với bộ lọc tùy chọn
fetchSubmissionById	id	Promise<void>	Lấy chi tiết một bài nộp theo ID
fetchMySubmissions	–	Promise<void>	Lấy tất cả bài nộp của người dùng hiện tại
fetchExamScores	examId	Promise<ExamScoreData>	Lấy điểm và thống kê của một đề thi
getStatistics	examId	Promise<ExamStatistics>	Lấy thống kê điểm của một đề thi
updateScore	submissionId, score	Promise<Submission>	Cập nhật điểm thủ công cho một bài nộp

Bảng B.12: Các phương thức của lớp SubmissionStore

Mối quan hệ. SubmissionStore sử dụng ApiService để gọi các API liên quan đến bài nộp (GET submissions, GET submissions/:id, GET exam-scores, GET statistics, PUT submissions/:id/score). Store quản lý các model Submission, ExamScoreData và ExamStatistics.

B.7 Lớp ImageProcessor (Mobile – Flutter)

Vị trí. Gói engine/, tầng Business Logic.

Mô tả. ImageProcessor thực hiện các phép biến đổi trên ảnh đầu vào để chuẩn bị cho việc nhận diện OMR. Lớp cung cấp các phương thức để tăng tương phản, áp dụng ngưỡng nhị phân, xoay, cắt, chống nghiêng, chuyển đổi grayscale và lọc nhiễu.

Các thuộc tính. Bảng C.13 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
imageLib	ImageProcessingLib	Thư viện xử lý ảnh (OpenCV/Dart) dùng để thực hiện các phép biến đổi trên ảnh

Bảng B.13: Các thuộc tính của lớp ImageProcessor

Các phương thức. Bảng C.14 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
preprocess	raw: Uint8List	ProcessedImage	Tiền xử lý đầy đủ bao gồm grayscale, tăng tương phản và áp dụng ngưỡng
enhanceContrast	image, factor	Uint8List	Tăng độ tương phản của ảnh theo hệ số cho trước
applyThreshold	image, method	Uint8List	Áp dụng ngưỡng nhị phân với phương pháp: Simple, Otsu, Adaptive Mean, Adaptive Gaussian
rotate	image, angle	Uint8List	Xoay ảnh theo góc (đơn vị độ) cho trước
resize	image, width, height	Uint8List	Thay đổi kích thước ảnh về chiều rộng và chiều cao chỉ định
crop	image, rect	Uint8List	Cắt vùng ảnh theo hình chữ nhật rect từ ảnh gốc
deskew	image: Uint8List	Uint8List	Chống nghiêng tự động bằng cách phát hiện và xoay ảnh về góc ngang
convertToGrayscale	image	Uint8List	Chuyển ảnh màu sang ảnh xám (grayscale)
removeNoise	image	Uint8List	Lọc nhiễu khỏi ảnh bằng các bộ lọc như Gaussian blur
detectSkewAngle	image	double	Phát hiện góc nghiêng của ảnh và trả về giá trị góc (đơn vị độ)

Bảng B.14: Các phương thức của lớp ImageProcessor

Mối quan hệ. ImageProcessor sử dụng ImageProcessingLib để thực hiện các phép biến đổi trên ảnh. Lớp sản sinh và trả về đối tượng ProcessedImage chứa dữ liệu ảnh đã xử lý cùng các thông tin metadata (width, height, format, processingSteps).

B.8 Lớp BubbleDetector (Mobile – Flutter)

Vị trí. Gói engine/, tầng Business Logic.

Mô tả. BubbleDetector phát hiện và phân loại các bong bóng (vùng tròn được tô đậm) trên ảnh phiếu trả lời OMR. Lớp tìm các đường viền trong ảnh, lọc theo diện tích để xác định vùng bong bóng, tính tỷ lệ tô đầy (fill ratio) và xác định bong bóng nào đã được tô.

Các thuộc tính. Bảng C.15 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
minBubbleArea	double	Diện tích tối thiểu của bong bóng để được coi là vùng tô trả lời
maxBubbleArea	double	Diện tích tối đa của bong bóng để tránh nhầm với các vùng lớn hơn
fillThreshold	double	Ngưỡng fill ratio (từ 0 đến 1) để xác định bong bóng đã được tô đầy

Bảng B.15: Các thuộc tính của lớp BubbleDetector

Các phương thức. Bảng C.16 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
detect	image	List<Bubble>	Phát hiện tất cả bong bóng trong ảnh, trả về danh sách đối tượng Bubble
findContours	image	List<Contour>	Tìm tất cả đường viền (contour) trong ảnh đã xử lý
filterByArea	contours, minArea, maxArea	List<Contour>	Lọc các contour theo ngưỡng diện tích tối thiểu và tối đa
groupByRow	contours	List<List<Contour>>	Nhóm các contour thành các hàng dựa trên tọa độ y
sortByPosition	bubbles	List<Bubble>	Sắp xếp bong bóng theo vị trí (trái sang phải, trên xuống dưới)
calculateFillRatio	contour, image	double	Tính tỷ lệ diện tích tô đầy của một contour trên ảnh gốc
isFilled	contour, image, threshold	boolean	Kiểm tra một contour đã được tô đầy hay chưa dựa trên ngưỡng

Bảng B.16: Các phương thức của lớp BubbleDetector

Mối quan hệ. BubbleDetector sử dụng Contour để phát hiện các vùng có thể là bong bóng và tạo ra các đối tượng Bubble chứa thông tin vị trí, kích thước và trạng thái tô của mỗi bong bóng.

B.9 Lớp AnswerReader (Mobile – Flutter)

Vị trí. Gói engine/, tầng Business Logic.

Mô tả. AnswerReader đọc mã học sinh, mã phiên bản đề và các đáp án từ danh sách bong bóng đã được phát hiện. Lớp sử dụng OMRTemplate để xác định cấu trúc layout của phiếu trả lời (số hàng, số cột, vị trí mã học sinh, vị trí mã đề, vị trí đáp án).

Các thuộc tính. Bảng C.17 mô tả chi tiết các thuộc tính của lớp.

Thuộc tính	Kiểu	Mô tả
template	OMRTemplate	Template OMR định nghĩa cấu trúc layout phiếu trả lời để mapping bong bóng đến câu hỏi và đáp án

Bảng B.17: Các thuộc tính của lớp AnswerReader

Các phương thức. Bảng C.18 mô tả chi tiết các phương thức của lớp.

Phương thức	Tham số	Kiểu trả về	Mô tả
readStudentCode	bubbles	String	Đọc mã học sinh từ vùng bong bóng dành cho mã số trên phiếu
readVersionCode	bubbles	String	Đọc mã phiên bản đề thi từ vùng bong bóng dành cho mã đề
readAnswers	bubbles, numQuestions	Map<int, String>	Đọc toàn bộ đáp án từ các bong bóng, trả về map từ số câu đến đáp án (A, B, C, D)
mapBubbleToQuestion	bubble	int	Map một bong bóng đến số thứ tự câu hỏi tương ứng
mapBubbleToOption	bubble	String	Map một bong bóng đến đáp án tương ứng (A, B, C, D hoặc E)
_groupBubblesByRow	bubbles	List<List<Bubble>	Nhóm các bong bóng theo hàng dựa trên tọa độ y
_decodeFilledBubbles	rowBubbles	String	Giải mã nhóm bong bóng trong một hàng để xác định đáp án được chọn
_parseStudentCode	codeString	String	Parse và kiểm tra tính hợp lệ của mã học sinh đọc được

Bảng B.18: Các phương thức của lớp AnswerReader

Mối quan hệ. AnswerReader sử dụng OMRTemplate để biết cấu trúc layout phiếu (số câu hỏi, số đáp án mỗi câu, vị trí mã học sinh, vị trí mã đề, vị trí đáp án) và sử dụng Bubble để trích xuất thông tin đáp án từ danh sách bong bóng đã phát hiện.

B.10 Tổng hợp các lớp trong phụ lục

Bảng B.19 tóm tắt tất cả 9 lớp được mô tả trong phụ lục này cùng với nền tảng, gói chứa và số lượng phương thức của mỗi lớp.

#	Lớp	Nền tảng	Gói	UC liên quan
1	AppealService	Backend	services	UC-08 (Phức khảo)
2	NotificationService	Backend	services	UC-07 (Thông báo)
3	AuthService	Backend	services	UC-00 (Đăng nhập)
4	ClassService	Backend	services	UC-03 (Quản lý lớp)
5	AuthStore	Frontend	stores	UC-00 (Xác thực)
6	SubmissionStore	Frontend	stores	UC-07 (Xem kết quả)
7	ImageProcessor	Mobile	engine	UC-02 (Xử lý ảnh)
8	BubbleDetector	Mobile	engine	UC-02 (Phát hiện bong bóng)
9	AnswerReader	Mobile	engine	UC-02 (Đọc đáp án)

Bảng B.19: Tổng hợp các lớp trong Phụ lục C

Bảng B.20 thống kê phân bố các lớp theo nền tảng.

Nền tảng	Số lớp	Danh sách lớp
Backend (Node.js)	4	AppealService, NotificationService, AuthService, ClassService
Frontend Web (React)	2	AuthStore, SubmissionStore
Mobile (Flutter)	3	ImageProcessor, BubbleDetector, AnswerReader
Tổng cộng	9	

Bảng B.20: Phân bố các lớp theo nền tảng

C. SINH CÂU HỎI BẰNG AI

Phần này trình bày đặc tả chi tiết chức năng sinh câu hỏi bằng AI trong hệ thống Xây dựng hệ thống quản lý ngân hàng câu hỏi và chấm thi trắc nghiệm tự động sử dụng công nghệ OMR. Chức năng gồm hai phần: sinh câu hỏi mới từ chủ đề và sinh câu hỏi tương tự từ câu hỏi có sẵn.

C.1 Tổng quan chức năng sinh câu hỏi bằng AI

Chức năng sinh câu hỏi bằng AI cho phép giáo viên tạo câu hỏi trắc nghiệm nhanh chóng thông qua hai phương thức: (i) sinh câu hỏi mới từ chủ đề hoặc nội dung cho trước, và (ii) sinh câu hỏi tương tự từ một câu hỏi đã có trong ngân hàng câu hỏi. Hệ thống sử dụng dịch vụ AI để phân tích yêu cầu và sinh ra các câu hỏi theo cấu trúc chuẩn trắc nghiệm.

Giáo viên có thể chọn loại câu hỏi (một đáp án đúng, nhiều đáp án đúng, đúng sai), mức độ khó, chủ đề và số lượng câu hỏi cần sinh. Kết quả sinh ra sẽ được hiển thị để giáo viên kiểm tra, chỉnh sửa trước khi lưu vào ngân hàng câu hỏi.

C.2 Sinh câu hỏi mới

C.2.1 Mục đích

Chức năng sinh câu hỏi mới cho phép giáo viên tạo các câu hỏi trắc nghiệm hoàn toàn mới dựa trên chủ đề, nội dung bài giảng hoặc tài liệu tham khảo do giáo viên cung cấp.

C.2.2 Điều kiện trước

Giáo viên đã đăng nhập vào hệ thống, có quyền quản lý ngân hàng câu hỏi.

C.2.3 Luồng thực thi chính

1. Giáo viên truy cập chức năng "Ngân hàng câu hỏi" và chọn "Sinh câu hỏi bằng AI".
2. Hệ thống hiển thị form sinh câu hỏi mới với các trường: chủ đề, nội dung/tài liệu tham khảo, loại câu hỏi, mức độ khó, số lượng câu hỏi.
3. Giáo viên nhập thông tin và nhấn nút "Sinh câu hỏi".
4. Hệ thống gửi yêu cầu đến dịch vụ AI kèm prompt chứa thông tin cấu hình.
5. Dịch vụ AI phân tích nội dung và sinh danh sách câu hỏi theo yêu cầu.
6. Hệ thống nhận kết quả và hiển thị danh sách câu hỏi đã sinh.
7. Giáo viên kiểm tra từng câu hỏi, chỉnh sửa nội dung, đáp án hoặc mức độ khó nếu cần.
8. Giáo viên nhấn "Lưu vào ngân hàng" để thêm các câu hỏi đã duyệt.
9. Hệ thống lưu câu hỏi vào ngân hàng với trạng thái "chờ phê duyệt".

C.2.4 Luồng thực thi mở rộng

- Tại bước 5, nếu dịch vụ AI gặp lỗi: hệ thống thông báo và cho phép thử lại hoặc chuyển sang tạo câu hỏi thủ công.
- Tại bước 5, nếu nội dung đầu vào quá ngắn: hệ thống thông báo yêu cầu nhập thêm thông tin.
- Tại bước 7, nếu câu hỏi sinh ra không phù hợp: giáo viên có thể nhấn "Tạo lại" để yêu cầu AI sinh lại.
- Tại bước 8, nếu câu hỏi trùng lặp với câu hỏi đã có: hệ thống cảnh báo và cho phép bỏ qua hoặc lưu.

C.2.5 Điều kiện sau

Nếu thành công, các câu hỏi được thêm vào ngân hàng câu hỏi và sẵn sàng sử dụng để tạo đề thi.

C.2.6 Cấu trúc prompt gửi đến AI

Trường	Nội dung
System role	Bạn là một giáo viên có kinh nghiệm, chuyên tạo câu hỏi trắc nghiệm chất lượng cao.
Loại câu hỏi	single-choice (một đáp án đúng), multiple-choice (nhiều đáp án đúng), true-false (đúng sai)
Số lượng	Số câu hỏi cần sinh
Mức độ khó	easy, medium, hard
Chủ đề	Chủ đề chính của câu hỏi
Nội dung gốc	Nội dung bài giảng hoặc tài liệu tham khảo (nếu có)
Yêu cầu output	JSON array với cấu trúc: question, options[], correctAnswer, explanation, difficulty

Bảng C.1: Cấu trúc prompt cho chức năng sinh câu hỏi mới

C.2.7 Cấu trúc câu hỏi sinh ra

Trường	Mô tả
question	Nội dung câu hỏi (dạng text)
options	Mảng các đáp án (tối thiểu 3, tối đa 5)
correctAnswer	Đáp án đúng (index hoặc ký tự A, B, C, D, E)
explanation	Giải thích ngắn gọn tại sao đáp án đúng
difficulty	Mức độ khó: easy, medium, hard
topic	Chủ đề của câu hỏi

Bảng C.2: Cấu trúc câu hỏi sinh ra bởi AI

C.3 Sinh câu hỏi tương tự

C.3.1 Mục đích

Chức năng sinh câu hỏi tương tự cho phép giáo viên tạo các phiên bản khác của một câu hỏi đã có, giúp tăng số lượng câu hỏi trong ngân hàng mà không cần tạo

mới hoàn toàn. Các câu hỏi tương tự có cùng đáp án đúng nhưng khác về cách diễn đạt, thứ tự đáp án hoặc giá trị số.

C.3.2 Điều kiện trước

Giáo viên đã đăng nhập, có quyền quản lý ngân hàng câu hỏi và đã chọn một câu hỏi có sẵn trong ngân hàng.

C.3.3 Luồng thực thi chính

1. Giáo viên truy cập ngân hàng câu hỏi và chọn một câu hỏi muốn sinh tương tự.
2. Giáo viên nhấn nút "Sinh câu tương tự" trên giao diện chi tiết câu hỏi.
3. Hệ thống hiển thị form với các tùy chọn: số lượng câu sinh ra, kiểu thay đổi (diễn đạt lại, đảo đáp án, thay số liệu, kết hợp).
4. Giáo viên chọn tùy chọn và nhấn "Sinh".
5. Hệ thống gửi yêu cầu đến dịch vụ AI kèm câu hỏi gốc và các tùy chọn.
6. Dịch vụ AI phân tích câu hỏi gốc và sinh các phiên bản tương tự.
7. Hệ thống nhận kết quả và hiển thị danh sách câu hỏi tương tự.
8. Giáo viên kiểm tra từng câu hỏi sinh ra, đảm bảo đáp án đúng được giữ nguyên.
9. Giáo viên nhấn "Lưu tất cả" hoặc chọn từng câu để lưu vào ngân hàng.
10. Hệ thống lưu các câu hỏi mới và liên kết với câu hỏi gốc trong metadata.

C.3.4 Luồng thực thi mở rộng

- Tại bước 6, nếu câu hỏi gốc quá phức tạp: hệ thống thông báo và đề xuất giảm số lượng sinh ra.
- Tại bước 8, nếu đáp án bị thay đổi: hệ thống cảnh báo và yêu cầu giáo viên xác nhận.
- Tại bước 8, nếu câu hỏi sinh ra trùng lặp với câu hỏi gốc: hệ thống tự động loại bỏ và thông báo.
- Tại bước 9, nếu câu hỏi đã tồn tại trong ngân hàng: hệ thống cảnh báo và cho phép bỏ qua.

C.3.5 Điều kiện sau

Nếu thành công, các câu hỏi tương tự được thêm vào ngân hàng và được đánh dấu là phiên bản của câu hỏi gốc.

C.3.6 Các kiểu thay đổi khi sinh tương tự

Kiểu	Mô tả	Ví dụ
Diễn đạt lại	Giữ nguyên nội dung, thay đổi cách diễn đạt	"Tốc độ là gì?" → "Thế nào là tốc độ của một vật?"
Đảo đáp án	Giữ nguyên câu hỏi và đáp án, thay đổi thứ tự các lựa chọn	Đáp án đúng từ vị trí A chuyển sang vị trí C
Thay số liệu	Giữ nguyên cấu trúc, thay đổi giá trị số	"Một vật 10kg..." → "Một vật 15kg..."
Kết hợp	Áp dụng nhiều kiểu thay đổi cùng lúc	Diễn đạt lại + thay số liệu + đảo đáp án

Bảng C.3: Các kiểu thay đổi khi sinh câu hỏi tương tự

C.3.7 Cấu trúc prompt cho chức năng sinh tương tự

Trường	Nội dung
System role	Bạn là một giáo viên có kinh nghiệm, chuyên tạo các phiên bản tương tự của câu hỏi trắc nghiệm.
Câu hỏi gốc	Câu hỏi ban đầu cần sinh tương tự
Đáp án đúng	Đáp án đúng của câu hỏi gốc (phải giữ nguyên trong tất cả phiên bản)
Kiểu thay đổi	Loại thay đổi muốn áp dụng (xem Bảng C.3)
Số lượng	Số phiên bản cần sinh
Yêu cầu output	JSON array với cấu trúc: question, options[], correctAnswer

Bảng C.4: Cấu trúc prompt cho chức năng sinh câu hỏi tương tự

C.4 Xử lý lỗi và giới hạn

C.4.1 Các lỗi thường gặp

Lỗi	Nguyên nhân	Xử lý
Timeout khi gọi AI	Dịch vụ AI phản hồi chậm	Thử lại hoặc giảm số lượng câu
Nội dung không phù hợp	Prompt không đủ thông tin	Bổ sung nội dung chi tiết hơn
Số lượng không đủ	AI không sinh đủ số câu yêu cầu	Tăng nội dung đầu vào, giảm số lượng
Đáp án không hợp lệ	AI sinh ra đáp án ngoài phạm vi	Giáo viên chỉnh sửa thủ công

Bảng C.5: Các lỗi thường gặp và cách xử lý

C.4.2 Giới hạn của hệ thống

- Số lượng câu hỏi sinh ra tối đa trong một lần: 20 câu.
- Thời gian chờ phản hồi tối đa từ dịch vụ AI: 30 giây.

- Độ dài nội dung đầu vào tối đa: 5000 ký tự.
- Số lần thử lại tối đa khi gặp lỗi: 3 lần.
- Tất cả câu hỏi sinh ra đều được đánh dấu ”chờ phê duyệt” trước khi sử dụng.

C.5 Bảo mật và đạo đức

C.5.1 Các câu hỏi do AI sinh ra

- Câu hỏi sinh ra bởi AI được đánh dấu trong metadata để phân biệt với câu hỏi do giáo viên tạo thủ công.
- Giáo viên có trách nhiệm kiểm tra và phê duyệt tất cả câu hỏi do AI sinh trước khi đưa vào sử dụng.
- Hệ thống không sử dụng câu hỏi do AI sinh ra để huấn luyện mô hình mà không có sự đồng ý của giáo viên.

C.5.2 Xác thực nội dung

- Hệ thống có bộ lọc nội dung để ngăn chặn việc sinh ra câu hỏi chứa thông tin nhạy cảm, phân biệt đối xử hoặc không phù hợp.
- Nếu phát hiện nội dung vi phạm, hệ thống sẽ từ chối sinh và thông báo cho giáo viên.